HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



ĐỀ TÀI: HỆ THỐNG HỖ TRỢ TIỀN CHẨN ĐOÁN BỆNH VỀ DA DỰA TRÊN MULTI-SCALE CNN, KNOWLEDGE GRAPH VÀ RAG

Giảng viên hướng dẫn: PGS.TS. Trần Đình Quế

Thành viên nhóm

Đoàn Thảo Vân	B22DCCN890
Lê Đình Hiệp	B22DCCN295
Phan Thị Hồng Thắm	B22DCCN806

Hà Nội – 2026

MỤC LỤC

DANH MỤC HÌNH VẼ vi

DANH MỤC BẢNG BIỂU ix

DANH MỤC THUẬT NGỮ VIẾT TẮT x

MỞ ĐẦU 1

CHƯƠNG 1: TỔNG QUAN HỆ THỐNG HỖ TRỢ TIỀN CHẨN ĐOÁN BỆNH VỀ DA DỰA TRÊN MULTI-SCALE CNN, KNOWLEDGE GRAPH VÀ RAG 8

1.1 Đặt vấn đề và phạm vi bài toán 8

1.1.1 Đặt vấn đề 8

1.1.2 Đối tượng và chức năng 8

1.1.3 Giới hạn 9

1.2 Đề xuất giải pháp 10

1.2.1 Ý tưởng tổng thể 10

1.2.2 Đặc điểm của giải pháp 10

1.3 Kiến trúc hệ thống 11

1.3.1 Kiến trúc tổng quan 11

1.3.2 Luồng xử lý một lượt tư vấn 11

1.4 Các công nghệ phát triển ứng dụng 12

1.5 Các kỹ thuật và mô hình trí tuệ nhân tạo 13

1.5.1 Phân loại ảnh da liễu bằng CNN 13

1.5.2 Truy xuất tăng cường sinh văn bản (RAG) 13

1.5.3 Đồ thị tri thức 13
1.5.4 Tác nhân hội thoại dựa trên LLM 14
1.5.5 Các mô hình sử dụng trong hệ thống 14
1.6 Kết luận 15

CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 16

2.1 Phân tích yêu cầu 17
2.1.1 Tác nhân và yêu cầu của hệ thống 17
2.1.2 Biểu đồ use case tổng quan 17
2.1.3 Use case chi tiết 17
2.2 Thiết kế thực thể 17
2.2.1 Các lớp thực thể liên quan 17
2.2.2 Quan hệ giữa các lớp thực thể 17
2.3 Thiết kế kiến trúc microservice 17
2.3.1 Phân rã service 17
2.3.2 Thiết kế dịch vụ `user-service` 17
2.3.3 Thiết kế dịch vụ `chat-service` 17
2.3.4 Thiết kế dịch vụ `agent-service` 17
2.3.5 Thiết kế dịch vụ `appointment-service` 17
2.3.6 Thiết kế dịch vụ `record-service` 17
2.3.7 Thiết kế dịch vụ `knowledge-service` 17
2.4 Luồng xử lý các chức năng chính 17
2.4.1 Đăng nhập và đăng ký 17
2.4.2 Hỏi đáp với chatbot tiền chẩn đoán 17
2.4.3 Yêu cầu tư vấn và báo cáo tiền khám 17

2.4.4 Tư vấn video call với bác sĩ 17
2.4.5 Chốt báo cáo kết luận và nạp ca bệnh 17
2.4.6 Số hóa giáo trình vào Knowledge Base 17
2.5 Kết luận 17

CHƯƠNG 3: KỸ THUẬT PHÂN LOẠI HÌNH ẢNH TỔN THƯƠNG DA DỰA TRÊN MULTI-SCALE CNN 18

3.1 Bài toán phân loại hình ảnh da liễu 18
3.1.1 Đặc điểm bài toán 18
3.1.2 Các nghiên cứu liên quan 19
3.2 Cơ sở lý thuyết mạng nơ-ron tích chập 20
3.2.1 Biểu diễn ảnh dưới dạng tensor 20
3.2.2 Phép tích chập 21
3.2.3 Hàm kích hoạt và pooling 22
3.2.4 Kiến trúc ResNet-18 và Residual Learning 22
3.3 Kiến trúc đề xuất AdaptiveCNN 24
3.3.1 Tổng quan kiến trúc 24
3.3.2 Cơ chế tích chập đa kernel thích nghi (AMKC) 25
3.3.3 Cài đặt kiến trúc mô hình AdaptiveCNN 29
3.4 Huấn luyện và tối ưu mô hình 31
3.4.1 Hàm mất mát phân loại 31
3.4.2 Lan truyền ngược và cập nhật tham số 33
3.4.3 Learning rate và bộ tối ưu 33
3.4.4 Tăng cường dữ liệu và xử lý mất cân bằng lớp 35
3.5 Thiết lập môi trường thực nghiệm 37
3.5.1 Các bộ dữ liệu thực nghiệm 37

3.5.2 Các mô hình đối sánh 39
3.5.3 Môi trường thực nghiệm và Tiêu chí đánh giá 40
3.6 Kết quả thực nghiệm và Phân tích đánh giá 42
3.6.1 Kết quả thực nghiệm trên bộ dữ liệu HAM10000 42
3.6.2 Kết quả thực nghiệm trên bộ dữ liệu quy mô lớn ISIC 2019 . . . 45
3.6.3 Kết quả thực nghiệm trên bộ dữ liệu ảnh lâm sàng SkinDisease . 48
3.6.4 Kết quả thực nghiệm trên bộ dữ liệu ảnh điện thoại PAD-UFES-20 50
3.6.5 Bảng tổng hợp đối sánh và Phân tích khoa học khách quan 53
3.6.6 Phân tích đánh đổi (Trade-off Analysis) và Lựa chọn mô hình cho Hệ thống Chatbot 54
3.7 Kết luận 56

CHƯƠNG 4: TỔ CHỨC VÀ TRUY XUẤT TRI THỨC CHO RAG BẰNG KHO VECTOR VÀ ĐỒ THỊ TRI THỨC 57

4.1 Cơ sở lý thuyết về RAG 57
4.1.1 Hạn chế của mô hình ngôn ngữ lớn khi dùng độc lập 57
4.1.2 Nguyên lý của RAG 57
4.1.3 Các thành phần RAG trong hệ thống 58
4.2 Nguồn tri thức của chatbot da liễu 59
4.2.1 Ba nguồn tri thức và nơi lưu trữ 59
4.2.2 Giáo trình da liễu số hóa 60
4.2.3 Ca bệnh đã kết luận của bác sĩ 70
4.2.4 Đồ thị tri thức có sẵn: PrimeKG và DermO 73
4.3 Các luồng truy xuất tri thức của chatbot 80
4.3.1 Truy xuất trong luồng RAG của chatbot 80
4.3.2 Các luồng truy xuất trên văn bản giáo trình 81

4.3.3 Luồng tìm kiếm ngữ nghĩa 82
4.3.4 Luồng tìm kiếm từ khóa 83
4.3.5 Luồng tìm kiếm đồ thị 85
4.3.6 Luồng hỗn hợp (`hybrid_retrieval`) 87
4.3.7 Truy xuất trực tiếp trên PrimeKG và DermO 91
4.4 Kết luận 93

CHƯƠNG 5: THIẾT KẾ CHATBOT TIỀN CHẨN ĐOÁN DA LIỄU 94

5.1 Luồng xử lý tổng quát của một lượt chat 94
5.2 Luồng tiền chẩn đoán: xử lý mô tả triệu chứng và hình ảnh 94
5.2.1 Luồng xử lý mô tả triệu chứng 94
5.2.2 Luồng xử lý hình ảnh 94
5.2.3 Phối hợp các luồng 94
5.3 Dữ kiện liên quan đến bệnh nhân 94
5.3.1 Trích dữ kiện lâm sàng từ lời người bệnh 94
5.3.2 Trạng thái giữ qua các lượt hội thoại 94
5.3.3 Bộ nhớ dài hạn của bệnh nhân 94
5.4 Kết luận 94

CHƯƠNG 6: CÀI ĐẶT, TRIỂN KHAI VÀ KẾT QUẢ HỆ THỐNG 95

6.1 Yêu cầu hệ thống 95
6.2 Cài đặt hệ thống 95
6.3 Giao diện hệ thống 95
6.3.1 Đăng nhập và đăng ký 95
6.3.2 Bệnh nhân hỏi đáp với chatbot tiền chẩn đoán 95
6.3.3 Bệnh nhân yêu cầu tư vấn 95

6.3.4 Bác sĩ tiếp nhận yêu cầu tư vấn và xem báo cáo tiền khám 95
6.3.5 Bệnh nhân và bác sĩ tư vấn video call 95
6.3.6 Bác sĩ chốt báo cáo kết luận 95
6.3.7 Quản trị viên quản lý Knowledge Base 95
6.4 Kết luận 95

KẾT LUẬN **96**

DANH MỤC HÌNH VẼ

3.1 Cấu trúc khối Residual Block với đường truyền tắt (Skip Connection) trong ResNet 23

3.2 Sơ đồ tổng thể luồng xử lý của kiến trúc mạng AdaptiveCNN tích hợp khối AMKC 25

3.3 Sơ đồ khối cấu trúc mô-đun tích chập đa kernel thích nghi (AMKC) . . 27

3.4 Đường cong Training Loss và Validation Accuracy của ResNet-18 + AMKC trên HAM10000 43

3.5 Tiến trình giá trị cổng thích nghi $\tanh(g)$ và chuẩn trọng số tích chập AMKC trên HAM10000 43

3.6 Ma trận nhầm lẫn của mô hình ResNet-18 + AMKC trên tập kiểm thử HAM10000 44

3.7 Phân bố điểm F1 trên từng lớp bệnh của mô hình ResNet-18 + AMKC trên HAM10000 44

3.8 Đường cong Training Loss và Validation Accuracy trên tập dữ liệu ISIC 2019 46

3.9 Tiến trình giá trị cổng thích nghi $\tanh(g)$ và chuẩn trọng số tích chập AMKC trên ISIC 2019 46

3.10 Ma trận nhầm lẫn của mô hình ResNet-18 + AMKC trên tập kiểm thử ISIC 2019 47

3.11 Phân bố điểm F1 trên từng lớp bệnh của mô hình ResNet-18 + AMKC trên ISIC 2019 47

3.12 Đường cong Training Loss và Validation Accuracy trên tập SkinDisease (22 lớp) 48

3.13 Tiến trình giá trị cổng thích nghi tanh(g) và chuẩn trọng số tích chập AMKC trên SkinDisease 49

3.14 Ma trận nhầm lẫn 22 lớp của mô hình ResNet-18 + AMKC trên tập kiểm thử SkinDisease 49

3.15 Phân bố điểm F1 trên 22 lớp bệnh của mô hình ResNet-18 + AMKC trên SkinDisease 50

3.16 Đường cong Training Loss và Validation Accuracy trên tập PAD-UFES-20 51

3.17 Tiến trình giá trị cổng thích nghi tanh(g) và chuẩn trọng số tích chập AMKC trên PAD-UFES-20 51

3.18 Ma trận nhầm lẫn của mô hình ResNet-18 + AMKC trên tập kiểm thử PAD-UFES-20 52

3.19 Phân bố điểm F1 trên 6 nhóm bệnh của mô hình ResNet-18 + AMKC trên PAD-UFES-20 52

3.20 Biểu đồ phân tán đánh đổi giữa Macro-F1 trung bình, độ trễ suy luận trên GPU (ms) và kích thước tham số giữa 5 mô hình thực nghiệm (Kích thước bong bóng tỷ lệ với số tham số Params M) 55

4.1 Hai giai đoạn của hệ thống RAG 58

4.2 Các nguồn tri thức của hệ thống 59

4.3 Ba giáo trình da liễu được số hóa 60

4.4 Tổng quan quy trình số hóa giáo trình 65

4.5 Kết quả đọc nội dung của một trang 66

4.6 Kết quả đọc mục lục của một sách 66

4.7 Kết quả chia đoạn: chunk mẫu và danh sách cần soát 67

4.8 Kết quả lưu vào kho tri thức: số đoạn đã lưu, số chiều vector, kho lưu trữ và mô hình tạo vector 67

4.9 Kết quả trích quan hệ: giao diện duyệt 68

4.10 Số chunk của ba giáo trình trong Qdrant 69

4.11 Số cạnh trích từ giáo trình theo loại quan hệ và theo sách (tổng 8.802 cạnh, ba sách) 70

4.12 Thông tin đi kèm của một ca bệnh trong Qdrant 72

4.13 Dữ liệu PrimeKG sau khi lọc: bảng cạnh và bảng nút 74

4.14 Dữ liệu nguồn của DermO: một khối thuật ngữ 74

4.15 Lược đồ đồ thị tri thức trong Neo4j Browser (sáu loại nút, không gồm đồ thị hành trình bệnh nhân) 78

4.16 Đồ thị quanh bệnh *dermatitis, atopic* trong Neo4j Browser: thuốc chỉ định (cạnh `INDICATION`) và triệu chứng, dấu hiệu lâm sàng của bệnh (cạnh `DISEASE_PHENOTYPE_POSITIVE`) 78

4.17 Cạnh PrimeKG và cạnh trích từ hai giáo trình quanh bệnh psoriasis . . . 79

4.18 Số nút của PrimeKG theo loại (bệnh gồm cả nhóm `MONDO_grouped`) . . 79

4.19 Số cạnh của PrimeKG theo loại quan hệ 80

4.20 Luồng truy xuất hỗn hợp 88

DANH MỤC BẢNG BIỂU

1.1 Khó khăn và thành phần xử lý tương ứng 10

1.2 Các mô hình sử dụng trong hệ thống 15

3.1 Tổng hợp đặc tính các bộ dữ liệu hình ảnh da liễu thực nghiệm trong đề tài 39

3.2 Bảng đặc tả tổng hợp cấu hình kiến trúc, siêu tham số và thiết lập huấn luyện của 5 mô hình nghiên cứu 41

3.3 Kết quả đối sánh các mô hình trên tập kiểm thử HAM10000 (TTA, 3 seeds: 42, 123, 2026) 42

3.4 Kết quả đối sánh các mô hình trên tập kiểm thử ISIC 2019 (Seed 42, TTA) 45

3.5 Kết quả đối sánh các mô hình trên tập kiểm thử SkinDisease (22 lớp, Seed 42, TTA) 48

3.6 Kết quả đối sánh các mô hình trên tập kiểm thử PAD-UFES-20 (Seed 42, TTA) 50

3.7 Bảng tổng hợp kết quả thực nghiệm độc lập trên bốn bộ dữ liệu (Accuracy / Macro-F1, %) 53

3.8 Ma trận đánh đổi giữa hiệu năng, quy mô mô hình và độ trễ suy luận thực tế trên GPU Kaggle 54

4.1 Các nguồn tri thức và cách truy xuất 59

4.2 Thống kê ba giáo trình đã nạp vào kho 69

4.3 Thông tin đi kèm mỗi ca bệnh trong Qdrant 73

4.4 Ba hướng tìm kiếm bổ sung cho nhau 81

4.5 Kết quả đánh giá các luồng truy xuất trên bộ kiểm thử (n = 24) 90

4.6 Ý định truy vấn đồ thị 93

CHƯƠNG 3: KỸ THUẬT PHÂN LOẠI HÌNH ẢNH TỔN THƯƠNG DA DỰA TRÊN MULTI-SCALE CNN

Chương này trình bày nghiên cứu và xây dựng mô hình thị giác máy tính phục vụ bài toán phân loại tổn thương da từ ảnh chụp lâm sàng. Trước hết, chương phân tích đặc điểm riêng của bài toán phân loại ảnh da liễu và khảo sát các công trình liên quan về kỹ thuật tích chập đa kernel, đa tỷ lệ. Tiếp đó, chương hệ thống hóa cơ sở lý thuyết về mạng nơ-ron tích chập (CNN), kiến trúc ResNet-18 và cơ chế Residual Learning. Trên nền tảng đó, chương trình bày kiến trúc đề xuất AdaptiveCNN — mô hình kết hợp ResNet-18 với cơ chế tích chập đa kernel thích nghi (AMKC) được đặt trước tầng stem nhằm làm giàu biểu diễn đầu vào ở nhiều phạm vi không gian. Cuối cùng, chương mô tả quy trình huấn luyện, tối ưu mô hình và hệ thống các chỉ số đánh giá định lượng.

3.1 Bài toán phân loại hình ảnh da liễu

3.1.1 Đặc điểm bài toán

Phân loại ảnh da liễu là bài toán thị giác máy tính nhằm xác định lớp bệnh tương ứng với hình ảnh tổn thương da đầu vào. Với sự phát triển của học sâu, mạng nơ-ron tích chập (Convolutional Neural Network – CNN) được sử dụng rộng rãi nhờ khả năng tự động học các đặc trưng trực quan từ dữ liệu ảnh, thay vì yêu cầu thiết kế thủ công như phương pháp truyền thống.

Bài toán phân loại ảnh da liễu mang một số thách thức riêng biệt so với phân loại ảnh tự nhiên thông thường:

- **Đa dạng về hình thái tổn thương:** Các tổn thương da khác nhau về kích thước, hình dạng, đường biên và cấu trúc sắc tố. Cùng một bệnh có thể biểu hiện với nhiều hình thái khác nhau (đa hình thái), trong khi các bệnh lý khác nhau lại có thể có biểu hiện trực quan tương tự.
- **Phụ thuộc vào điều kiện chụp ảnh:** Ảnh chụp từ người bệnh trong thực tế có thể

khác biệt đáng kể về ánh sáng, góc chụp, độ phân giải và tỷ lệ vùng tổn thương trong khung hình, khác với ảnh soi da chuẩn hóa trong phòng khám.

- **Mất cân bằng phân bố lớp:** Một số bệnh lý phổ biến (như nốt ruồi lành tính) chiếm tỷ lệ áp đảo trong tập dữ liệu, trong khi các bệnh nguy hiểm hơn (như ung thư hắc tố) có số mẫu ít hơn nhiều.

Trong phạm vi đồ án, bài toán phân loại tổn thương da được nghiên cứu và đánh giá trên nhiều miền dữ liệu đa dạng: từ ảnh soi da chuyên sâu (Dermoscopy) với 7 nhóm tổn thương (tập HAM10000) và 8 nhóm tổn thương quy mô lớn (tập ISIC 2019), đến ảnh chụp lâm sàng thông thường (Clinical Photography) với 22 lớp bệnh (tập SkinDisease), cũng như ảnh chụp thực tế bằng điện thoại thông minh (Smartphone) với 6 nhóm bệnh (tập PAD-UFES-20). Mô hình thị giác máy tính tiếp nhận ảnh da đầu vào, trích xuất đặc trưng đa tỷ lệ, tổng hợp không gian qua tầng Global Average Pooling và ánh xạ sang phân bố xác suất của các lớp bệnh mục tiêu qua hàm Softmax. Kết quả phân loại này đóng vai trò là một công cụ tính toán (`predict_skin_lesion` tool), cung cấp các giả thuyết bệnh học kèm độ tin cậy định lượng cho tác nhân trí tuệ nhân tạo (LangGraph Agent Worker) kết hợp cùng ngữ cảnh người bệnh và đồ thị tri thức da liễu.

3.1.2 Các nghiên cứu liên quan

Các nghiên cứu về tích chập đa kernel, đặc trưng đa tỷ lệ và tích chập thích nghi cho thấy việc mở rộng cách biểu diễn đặc trưng là một hướng tiếp cận quan trọng trong thị giác máy tính và xử lý ảnh y khoa.

Tích chập đa kernel và đa tỷ lệ. Khan và cộng sự đề xuất MKIS-Net, một mạng nhẹ sử dụng thiết kế multi-kernel để tạo receptive field hiệu quả cho bài toán phân đoạn ảnh y khoa, bao gồm phân đoạn tổn thương da, mạch máu võng mạc và ảnh X-quang ngực. Kết quả cho thấy thiết kế đa kernel có thể hỗ trợ xây dựng mạng gọn nhẹ mà vẫn duy trì hiệu quả cạnh tranh [1]. Han và cộng sự đề xuất DM-CNN cho bài toán phân loại ảnh y khoa, trong đó có dữ liệu da liễu, sử dụng mô-đun Dynamic Multi-scale Feature Fusion để lựa chọn kernel dựa trên đặc trưng ở các tầng khác nhau, kết hợp cơ chế attention và pooling đa tỷ lệ [2].

Tích chập thích nghi và lựa chọn kernel. Hu và cộng sự đề xuất Domain and Content Adaptive Convolution (DCAC) nhằm cải thiện khả năng tổng quát hóa giữa các miền dữ liệu y khoa, điều chỉnh phép tích chập dựa trên thông tin miền và đặc trưng toàn cục của ảnh đầu vào [3]. Chen và cộng sự phát triển ESKNet cho phân đoạn khối u vú trên ảnh siêu âm, trong đó mô-đun Enhanced Selective Kernel Convolution kết hợp biểu diễn từ nhiều vùng đặc trưng và tái hiệu chỉnh thông tin theo chiều kênh lẫn không gian [4].

Kết hợp convolution và attention. AMCANet sử dụng các nhánh convolution attention với những mức lấy mẫu khác nhau nhằm kết hợp thông tin cục bộ và ngữ cảnh không gian cho bài toán khôi phục độ phân giải ảnh [5]. Do nhiệm vụ chính là super-resolution, nghiên cứu này chỉ là tài liệu tham khảo gián tiếp về thiết kế convolution kết hợp attention.

Nhận xét và định hướng. Các nghiên cứu trên thể hiện những hướng tiếp cận khác nhau: thiết kế nhiều kernel, kết hợp đặc trưng đa tỷ lệ, lựa chọn kernel thích nghi và kết hợp convolution với attention. Cần phân biệt ba khái niệm liên quan nhưng không đồng nhất: *multi-kernel* sử dụng nhiều kích thước kernel; *multi-scale* tập trung khai thác đặc trưng ở nhiều phạm vi không gian; còn *adaptive/selective convolution* có thêm cơ chế điều chỉnh hoặc lựa chọn đặc trưng dựa trên đầu vào. Từ cơ sở đó, đồ án sử dụng ResNet-18 làm backbone và tích hợp cơ chế tích chập đa kernel thích nghi (AMKC) trước tầng stem, cho phép mô hình khai thác đặc trưng ở nhiều kích thước ngay từ đầu vào.

3.2 Cơ sở lý thuyết mạng nơ-ron tích chập

3.2.1 Biểu diễn ảnh dưới dạng tensor

Ảnh màu thường được biểu diễn dưới dạng tensor ba chiều gồm số kênh màu, chiều cao và chiều rộng. Với ảnh RGB có kích thước $H \times W$, tensor ảnh có dạng:

$$X \in \mathbb{R}^{3\times H\times W} \tag{3.1}$$

Khi xử lý theo lô (batch), dữ liệu có dạng:

$$X \in \mathbb{R}^{N \times C \times H \times W} \tag{3.2}$$

Trong đó, N là số ảnh trong một batch, C là số kênh, H là chiều cao và W là chiều rộng.

Trước khi đưa vào mạng, ảnh thường được chuyển về kích thước thống nhất (224×224 trong đồ án này) và chuẩn hóa giá trị pixel theo trung bình và độ lệch chuẩn của tập ImageNet. Việc chuẩn hóa giúp dữ liệu có thang giá trị phù hợp với quá trình tối ưu khi sử dụng mô hình pretrained.

3.2.2 Phép tích chập

Phép tích chập (convolution) là thành phần cốt lõi của CNN. Một kernel có kích thước xác định được áp dụng trên các vùng cục bộ của đầu vào để tạo ra feature map. Các trọng số kernel được cập nhật trong quá trình huấn luyện để học những đặc trưng hữu ích cho bài toán.

Dạng tổng quát của phép tích chập hai chiều có thể biểu diễn như sau:

$$Y_{i,j} = \sum_{c=1}^{C_{\text{in}}} \sum_{u=0}^{K_h-1} \sum_{v=0}^{K_w-1} W_{c,u,v} X_{c,i+u,j+v} + b \tag{3.3}$$

Trong đó, X là tensor đầu vào, W là trọng số kernel, b là bias và Y là feature map đầu ra.

Kích thước kernel quyết định phạm vi không gian được xét trong mỗi phép tính. Kernel 3×3 tập trung vào vùng lân cận nhỏ, trong khi kernel 5×5 thu nhận thông tin từ vùng rộng hơn. Việc sử dụng nhiều kích thước kernel là cơ sở cho cơ chế AMKC được trình bày ở mục 3.3.2.

3.2.3 Hàm kích hoạt và pooling

Sau phép tích chập, hàm kích hoạt phi tuyến cho phép mạng học những quan hệ phức tạp hơn so với phép biến đổi tuyến tính đơn thuần. Hàm ReLU được sử dụng phổ biến nhất:

$$\text{ReLU}(x) = \max(0, x) \tag{3.4}$$

Pooling thực hiện giảm kích thước không gian của feature map hoặc tổng hợp thông tin từ các vùng lân cận. Max Pooling lấy giá trị lớn nhất trong mỗi vùng, trong khi Global Average Pooling (GAP) tính trung bình các giá trị trên toàn bộ không gian của từng kênh:

$$f_c = \frac{1}{|\Omega|} \sum_{i \in \Omega} x_{c,i} \tag{3.5}$$

Trong đó, Ω là tập các vị trí không gian và $x_{c,i}$ là giá trị đặc trưng tại vị trí i của kênh c. Trong AdaptiveCNN, GAP được sử dụng để tổng hợp feature map cuối cùng thành vector đặc trưng trước lớp phân loại.

3.2.4 Kiến trúc ResNet-18 và Residual Learning

ResNet (Residual Network) là kiến trúc CNN sử dụng kết nối tắt (skip connection) nhằm hỗ trợ quá trình huấn luyện mạng sâu. ResNet-18 là biến thể gồm 18 lớp có trọng số, được tổ chức theo nhiều giai đoạn (layer1 đến layer4), mỗi giai đoạn chứa các residual block.

Trong mạng CNN truyền thống, các lớp liên tiếp phải học trực tiếp ánh xạ từ đầu vào sang đầu ra. Khi mạng sâu hơn, gradient có thể suy giảm qua nhiều lớp, gây khó khăn cho quá trình tối ưu. ResNet giải quyết vấn đề này bằng cách học phần dư (residual mapping):

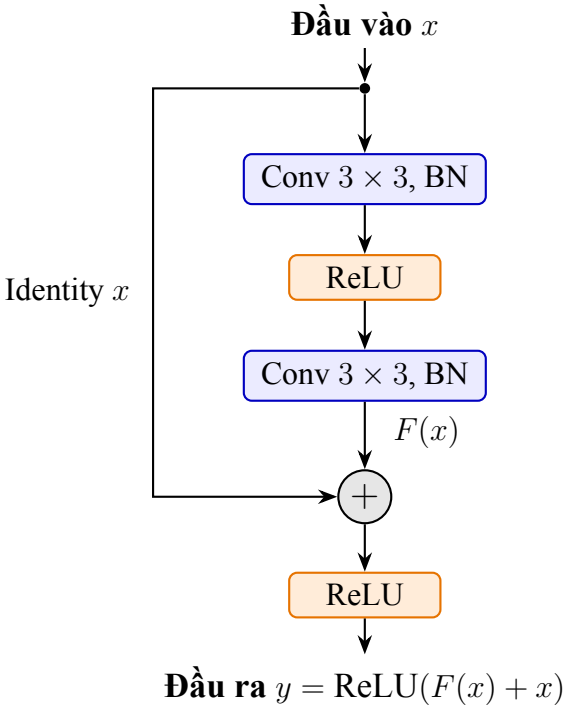
$$y = F(x, W) + x \tag{3.6}$$

Trong đó:

- x là đầu vào của khối.

- $F(x, W)$ là phép biến đổi được học bởi các lớp bên trong khối.
- y là đầu ra sau khi cộng với nhánh tắt.

Hình 3.1 minh họa cơ chế hoạt động của một khối Residual Block cơ bản với kết nối tắt (Skip Connection) trong kiến trúc ResNet-18.



Hình 3.1: Cấu trúc khối Residual Block với đường truyền tắt (Skip Connection) trong ResNet

Skip connection tạo đường truyền trực tiếp cho thông tin và gradient từ đầu vào đến đầu ra. Khi kích thước tensor giữa hai nhánh không tương thích, kiến trúc sử dụng phép chiếu 1×1 convolution để điều chỉnh số kênh. Cơ chế này giúp huấn luyện mạng sâu thuận lợi hơn và là nền tảng quan trọng của AdaptiveCNN.

ResNet-18 được lựa chọn làm backbone cho AdaptiveCNN vì cung cấp cơ chế trích xuất đặc trưng phân cấp hiệu quả, đồng thời có quy mô tham số tương đối gọn (khoảng 11 triệu tham số), phù hợp với tập dữ liệu da liễu có kích thước trung bình và giảm nguy cơ overfitting.

3.3 Kiến trúc đề xuất AdaptiveCNN

3.3.1 Tổng quan kiến trúc

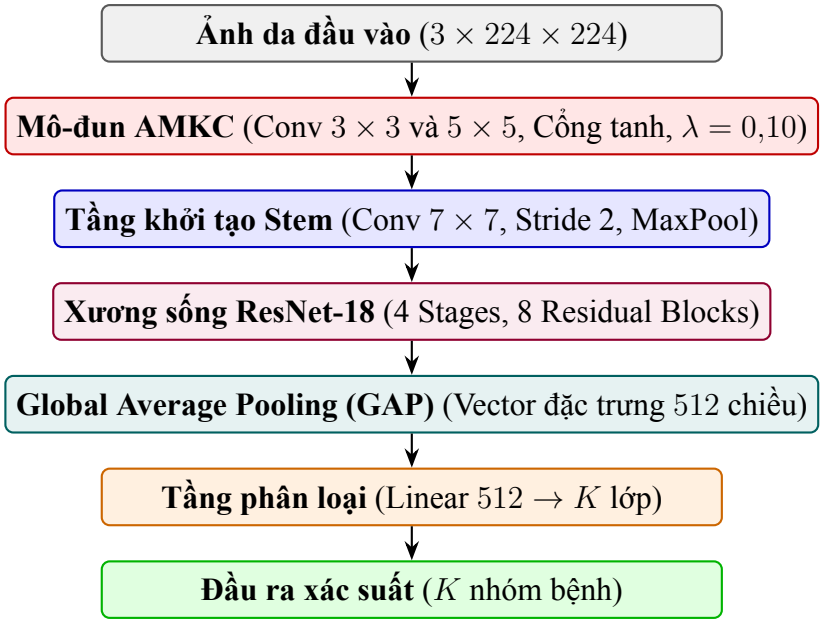
AdaptiveCNN được xây dựng trên nền ResNet-18 pretrained (ImageNet), bổ sung mô-đun AMKC (Adaptive Multi-Kernel Convolution) trước tầng stem. Luồng xử lý tổng thể được mô tả như sau:

$$\text{Input } (3 \times 224 \times 224) \xrightarrow{\text{AMKC}} \xrightarrow{\text{Stem}} \xrightarrow{\text{Layer1–4}} \xrightarrow{\text{GAP}} \xrightarrow{\text{Head}} \text{Output } (K \text{ classes}) \tag{3.7}$$

Các thành phần chính:

- **AMKC (Adaptive Multi-Kernel Convolution):** Mô-đun đặt trước stem, nhận ảnh đầu vào RGB ($3 \times 224 \times 224$) và trả về tensor cùng kích thước. AMKC kết hợp thông tin từ hai nhánh convolution 3×3 và 5×5 thông qua các cổng thích nghi, bổ sung đặc trưng đa tỷ lệ ngay từ không gian pixel. Chi tiết được trình bày tại mục 3.3.2.
- **Stem:** Lớp convolution 7×7, Batch Normalization, ReLU và Max Pooling 3×3 — giữ nguyên từ ResNet-18 chuẩn.
- **Layer1–Layer4:** Bốn giai đoạn residual block của ResNet-18, trích xuất đặc trưng phân cấp từ cấp thấp (cạnh, kết cấu) đến cấp cao (hình thái tổn thương).
- **GAP (Global Average Pooling):** Tổng hợp feature map cuối ($512 \times 7 \times 7$) thành vector 512 chiều.
- **Head:** Khối phân loại gồm BatchNorm $\rightarrow$ Dropout(0.4) $\rightarrow$ Linear(512, 512) $\rightarrow$ ReLU $\rightarrow$ BatchNorm $\rightarrow$ Dropout(0.2) $\rightarrow$ Linear(512, K), trong đó K là số lớp bệnh mục tiêu của bộ dữ liệu tương ứng.

Sơ đồ tổng thể kiến trúc dòng xử lý từ ảnh da đầu vào qua khối AMKC, xương sống ResNet-18 đến bộ phân loại được trực quan hóa trên Hình 3.2.



Hình 3.2: Sơ đồ tổng thể luồng xử lý của kiến trúc mạng AdaptiveCNN tích hợp khối AMKC

Điểm khác biệt then chốt so với ResNet-18 chuẩn là sự có mặt của AMKC — mô-đun nhẹ, được khởi tạo gần identity, cho phép mô hình khởi đầu từ biểu diễn pretrained rồi dần học thêm đặc trưng đa tỷ lệ trong quá trình fine-tune.

3.3.2 Cơ chế tích chập đa kernel thích nghi (AMKC)

a) Động cơ thiết kế

Trong ảnh da liễu, các tổn thương có thể khác nhau về kích thước, hình dạng, đường biên và cấu trúc sắc tố. Kernel 3×3 thu nhận thông tin từ vùng lân cận nhỏ, phù hợp để phát hiện chi tiết cục bộ như kết cấu bề mặt và biên tổn thương. Kernel 5×5 mở rộng vùng tiếp nhận không gian, giúp nắm bắt cấu trúc lớn hơn và mối quan hệ giữa các vùng lân cận.

AMKC được thiết kế nhằm kết hợp đồng thời đầu ra từ hai nhánh convolution có kích thước kernel khác nhau thông qua các cổng điều chỉnh có thể học, bổ sung đặc trưng đa phạm vi không gian ngay từ tầng đầu vào mà không thay đổi kiến trúc backbone.

b) Biểu diễn toán học

Gọi x là tensor đầu vào ($3 \times H \times W$). Hai nhánh tích chập song song được ký hiệu:

$$F_3(x) = \text{Conv}_{3\times3}(x), \quad F_5(x) = \text{Conv}_{5\times5}(x) \tag{3.8}$$

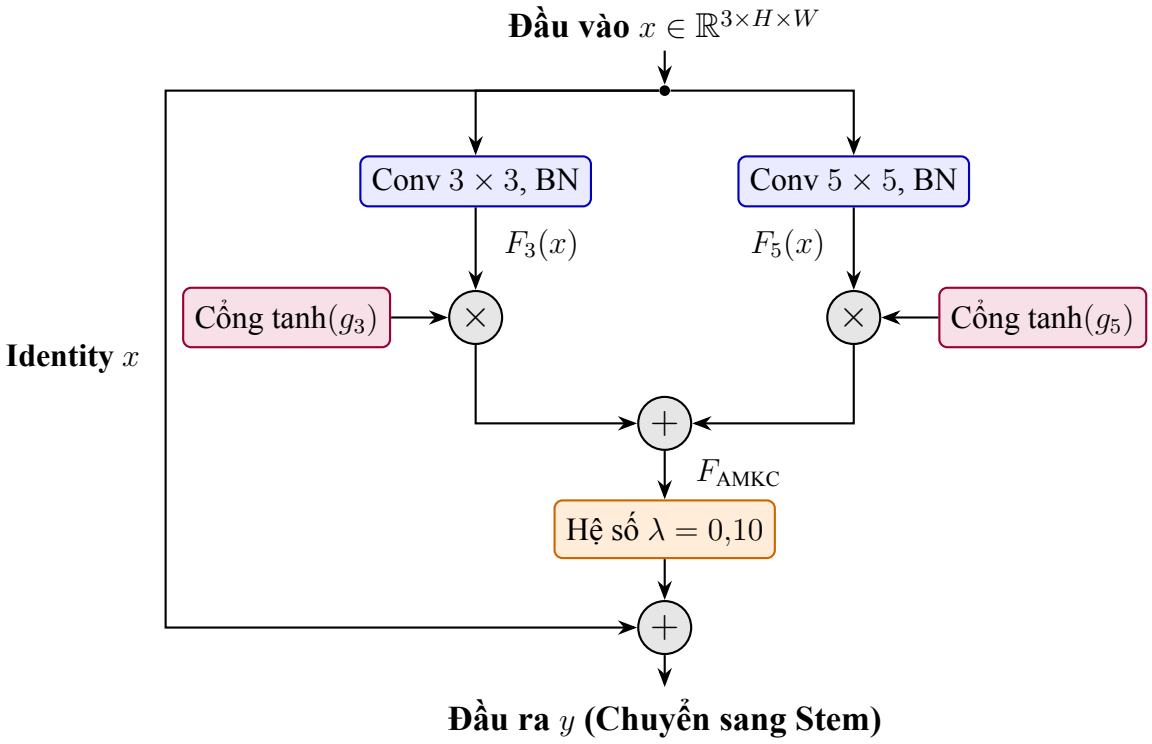
Đầu ra của mô-đun AMKC:

$$y = x + \lambda\,[\tanh(g_3) \cdot F_3(x) + \tanh(g_5) \cdot F_5(x)] \tag{3.9}$$

Trong đó:

- x là đầu vào, đồng thời đóng vai trò nhánh residual (identity shortcut).
- $F_3(x)$ và $F_5(x)$ là đặc trưng từ hai nhánh kernel 3×3 và 5×5, sử dụng `padding=k//2` để giữ nguyên kích thước không gian.
- g_3, g_5 là các tham số cổng (gate) có thể học. $\tanh(\cdot)$ giới hạn hệ số trong khoảng $(-1, 1)$, kiểm soát mức đóng góp và cho phép "tắt" một nhánh nếu cần.
- $\lambda = 0{,}10$ là hệ số tỷ lệ cố định, kiểm soát biên độ tổng thể của phần bổ sung so với nhánh identity.
- y là đầu ra, có cùng kích thước với x.

Chi tiết cấu trúc mô-đun AMKC, luồng dữ liệu song song của hai nhánh nhân tích chập và cơ chế cổng điều khiển được mô tả trực quan trong Hình 3.3.



Hình 3.3: Sơ đồ khối cấu trúc mô-đun tích chập đa kernel thích nghi (AMKC)

```python
class AdaptiveMultiKernelConv(nn.Module):
    def __init__(
        self, in_channels=3, out_channels=3,
        lambda_scale=0.10, kernels=None,
        learnable_gates=True
    ):
        super().__init__()
        self.kernels = kernels or [3, 5]
        self.lambda_scale = lambda_scale
        self.convs = nn.ModuleDict()
        self.gates = nn.ParameterDict()
        for k in self.kernels:
            conv = nn.Conv2d(
                in_channels, out_channels, k,
                padding=k // 2, bias=False
            )
            # Zero-init de ban dau la identity
            nn.init.zeros_(conv.weight)
            self.convs[f"conv{k}"] = conv
            self.gates[f"gate{k}"] = nn.Parameter(
                torch.ones(1),
                requires_grad=learnable_gates
            )

    def forward(self, x):
        residual = 0.0
        for k in self.kernels:
            gate = torch.tanh(self.gates[f"gate{k}"])
            out_k = self.convs[f"conv{k}"](x)
            residual = residual + gate * out_k
        return x + self.lambda_scale * residual
```

Mã nguồn 3.1: Cài đặt mô-đun tích chập đa kernel thích nghi (AMKC)

Về mặt cấu trúc, lớp `AdaptiveMultiKernelConv` kế thừa từ `nn.Module` nhằm thực hiện biến đổi và bổ sung đặc trưng đa tỷ lệ trực tiếp trên tensor ảnh đầu vào ($3 \times H \times W$):

- Trong phương thức khởi tạo `__init__`, danh sách `kernels` tiếp nhận hai kích thước $[3, 5]$, tương ứng với hai phạm vi quan sát không gian cục bộ (chi tiết biên, kết cấu sắc tố vi mô) và phạm vi mở rộng (cấu trúc hình thái tổng thể). Mỗi nhánh được cấu hình tích chập 2D với cơ chế đệm `padding = k // 2` để kích thước không gian feature map đầu ra luôn đồng nhất tuyệt đối với ảnh gốc ($H \times W$).
- Trọng số của các lớp tích chập được gán bằng không qua hàm `nn.init.zeros_(conv.weight)` (kỹ thuật Zero-Init). Nhờ đó, tại thời điểm khởi tạo, toàn bộ phần dư do tích chập sinh ra đều bằng 0, giúp mô-đun hoạt động chính xác như một ánh xạ đồng nhất (identity mapping) và bảo toàn trọn vẹn đặc trưng đã học từ ImageNet của xương sống ResNet-18. Mỗi nhánh đi kèm một tham số cổng vô hướng `gates[f"gate{k}"]` khởi tạo bằng 1 và có thể học (`requires_grad=True`).
- Trong hàm lan truyền xuôi `forward(x)`, ảnh đầu vào x lần lượt đi qua từng nhánh tích chập. Giá trị cổng được kích hoạt qua hàm `torch.tanh`, giới hạn trọng số điều tiết trong khoảng $(-1, 1)$, cho phép mạng tự động tăng cường, giảm nhẹ hoặc ức chế phản hồi từ một nhánh nếu nhánh đó phát sinh tín hiệu nhiễu. Tín hiệu tổng hợp của hai nhánh được nhân với hệ số co giãn $\lambda = 0{,}10$ (`lambda_scale`) nhằm giới hạn biên độ biến động, trước khi cộng trực tiếp vào đầu vào gốc x theo đường truyền tắt.

c) Khởi tạo zero-init và tính chất identity

Các trọng số convolution trong AMKC được khởi tạo bằng không (`nn.init.zeros_`). Kết hợp với hệ số $\lambda = 0{,}10$ nhỏ, điều này đảm bảo rằng tại thời điểm bắt đầu huấn luyện:

$$F_3(x) = 0, \quad F_5(x) = 0 \quad \Rightarrow \quad y = x \tag{3.10}$$

Mô-đun AMKC ban đầu hoạt động như ánh xạ đồng nhất (identity), không phá vỡ biểu diễn pretrained của ResNet-18. Trong quá trình fine-tune, các trọng số convolution và cổng dần được cập nhật để bổ sung đặc trưng đa kernel có lợi cho bài toán phân loại da liễu.

d) Vai trò trong AdaptiveCNN

AMKC được đặt trước tầng stem của ResNet-18, hoạt động như một "bộ tiền xử lý có thể học". So với việc chỉ đưa ảnh gốc vào backbone, AMKC bổ sung thêm thông tin đa phạm vi không gian — kernel 3×3 nắm bắt chi tiết cục bộ và kernel 5×5 nắm bắt cấu trúc rộng hơn — giúp backbone nhận được đầu vào phong phú hơn.

Tuy nhiên, mức cải thiện thực tế cần được kiểm chứng bằng thực nghiệm, bao gồm so sánh với ResNet-18 cơ sở (không có AMKC) trong cùng điều kiện huấn luyện và ablation study để xác định đóng góp của từng thành phần.

3.3.3 Cài đặt kiến trúc mô hình AdaptiveCNN

Mô hình hoàn chỉnh kết hợp AMKC đặt trước tầng stem của ResNet-18 pretrained, tiếp nối bởi tầng Global Average Pooling (GAP) và Standard Head được cài đặt như sau:

```python
def standard_head(
    feat_dim, num_classes, hidden=512, dropout=0.4
):
    return nn.Sequential(
        nn.BatchNorm1d(feat_dim),
        nn.Dropout(dropout),
        nn.Linear(feat_dim, hidden),
        nn.ReLU(inplace=True),
        nn.BatchNorm1d(hidden),
        nn.Dropout(dropout * 0.5),
        nn.Linear(hidden, num_classes),
    )

class ResNet18Variant(nn.Module):
    def __init__(
        self, num_classes, use_amkc=False,
        pretrained=True, hidden=512, dropout=0.4,
        lambda_scale=0.10, kernels=None
    ):
        super().__init__()
        if use_amkc:
            self.amkc = AdaptiveMultiKernelConv(
                3, 3, lambda_scale=lambda_scale,
                kernels=kernels
```

```
            )
        weights = (
            models.ResNet18_Weights.IMAGENET1K_V1
            if pretrained else None
        )
        backbone = models.resnet18(weights=weights)
        self.stem = nn.Sequential(
            backbone.conv1, backbone.bn1,
            backbone.relu, backbone.maxpool
        )
        self.layer1 = backbone.layer1
        self.layer2 = backbone.layer2
        self.layer3 = backbone.layer3
        self.layer4 = backbone.layer4
        self.pool = nn.AdaptiveAvgPool2d(1)  # GAP
        self.head = standard_head(
            backbone.fc.in_features,
            num_classes, hidden, dropout
        )

    def forward(self, x):
        if hasattr(self, "amkc"):
            x = self.amkc(x)
        x = self.stem(x)
        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        x = self.layer4(x)
        x = self.pool(x)
        return self.head(x.flatten(1))
```

Mã nguồn 3.2: Cài đặt kiến trúc AdaptiveCNN (ResNet-18 + AMKC + GAP)

Mô hình tổng thể AdaptiveCNN được tổ chức thành hai khối chức năng riêng biệt: tầng phân loại chuẩn hóa (`standard_head`) và lớp điều phối kiến trúc hoàn chỉnh (`ResNet18Variant`):

- Hàm `standard_head` thay thế cho lớp kết nối đầy đủ (`fc`) đơn tầng nguyên bản của ResNet-18 nhằm nâng cao năng lực khái quát hóa trên dữ liệu da liễu:
  - Vector đặc trưng 512 chiều đầu vào trước hết đi qua tầng `BatchNorm1d` và lớp `Dropout(0.4)` để chuẩn hóa phân bố dữ liệu và triệt tiêu bớt các liên kết dư thừa, hạn chế hiện tượng đồng thích nghi (co-adaptation).
  - Tiếp đó, tầng ẩn trung gian `Linear(512, 512)` kết hợp hàm kích hoạt phi tuyến `ReLU`, qua tầng `BatchNorm1d` thứ hai và `Dropout(0.2)`, trước khi ánh xạ về số lớp đầu ra `num_classes`. Cấu trúc phân tầng đa cấp kèm Dropout kép giúp mạng kiểm soát chặt chẽ nguy cơ quá khớp (overfitting) trên tập dữ liệu y tế có quy mô hữu hạn.
- Lớp `ResNet18Variant` chịu trách nhiệm ghép nối luồng dữ liệu liên tục:

- Khi cờ `use_amkc=True` được kích hoạt, mô-đun `AdaptiveMultiKernelConv` được chèn trực tiếp vào trước tầng stem.
- Xương sống ResNet-18 nạp sẵn bộ trọng số ImageNet-1K chuẩn. Các khối tính toán được phân tách tuần tự: tầng `stem` (tích chập 7×7, stride 2, MaxPool) và 4 giai đoạn residual block (`layer1` đến `layer4`) lần lượt trích xuất đặc trưng phân cấp từ đường nét, kết cấu cục bộ đến hình thái bệnh lý toàn cục.
- Tầng `AdaptiveAvgPool2d(1)` (Global Average Pooling - GAP) nén tensor đặc trưng $512 \times 7 \times 7$ thành vector 512 chiều mà không làm phát sinh tham số học, giúp biểu diễn không gian có tính bất biến cao với vị trí vùng tổn thương.
- Trong hàm `forward(x)`, dữ liệu ảnh được truyền tuần tự qua AMKC (nếu bật), stem, các khối residual, nén GAP, trải phẳng qua `x.flatten(1)` và phân loại bởi khối `head`.

3.4 Huấn luyện và tối ưu mô hình

3.4.1 Hàm mất mát phân loại

Với bài toán phân loại nhiều lớp, Cross-Entropy là hàm mất mát phổ biến. Gọi z_k là logit của lớp k, xác suất dự đoán được tính bằng Softmax:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}} \tag{3.11}$$

Trong đó, K là số lớp. Với nhãn đúng y, Cross-Entropy cho một mẫu là:

$$\mathcal{L} = -\log p_y \tag{3.12}$$

Đồ án sử dụng thêm kỹ thuật Label Smoothing ($\epsilon = 0{,}05$) để giảm overfitting. Thay vì one-hot cứng, nhãn được làm mềm thành:

$$y'_k = \begin{cases} 1-\epsilon & \text{nếu } k = y \\ \epsilon/(K-1) & \text{nếu } k \neq y \end{cases} \tag{3.13}$$

Ngoài ra, kỹ thuật Mixup ($\alpha = 0{,}2$) được áp dụng trong quá trình huấn luyện. Mixup tạo mẫu ảo bằng cách nội suy tuyến tính giữa hai mẫu huấn luyện và nhãn tương ứng, giúp mô hình học biểu diễn mượt mà hơn.

Đoạn mã hiện thực hóa kỹ thuật Mixup và cấu hình hàm mất mát Cross-Entropy với Label Smoothing trong PyTorch như sau:

```python
def mixup_data(x, y, alpha=0.2):
    lam = (
        np.random.beta(alpha, alpha)
        if alpha > 0 else 1.0
    )
    idx = torch.randperm(x.size(0), device=x.device)
    x_mix = lam * x + (1 - lam) * x[idx]
    return x_mix, y, y[idx], lam

def mixup_criterion(criterion, pred, y_a, y_b, lam):
    loss_a = criterion(pred, y_a)
    loss_b = criterion(pred, y_b)
    return lam * loss_a + (1 - lam) * loss_b

# Loss voi Label Smoothing epsilon = 0.05
criterion = nn.CrossEntropyLoss(label_smoothing=0.05)
```

Mã nguồn 3.3: Cài đặt kỹ thuật Mixup và Label Smoothing

Để tăng cường tính vững chắc cho mô hình trước hiện tượng ranh giới bệnh lý mờ nhạt và mất cân bằng dữ liệu, kỹ thuật nội suy tuyến tính mẫu (Mixup) và làm mịn nhãn (Label Smoothing) được tích hợp trong quy trình tối ưu:

- Hàm `mixup_data` sinh các mẫu huấn luyện ảo bằng cách hòa trộn giữa hai ảnh ngẫu nhiên trong cùng mini-batch:
  - Hệ số hòa trộn λ được lấy mẫu ngẫu nhiên từ phân phối Beta: $\lambda \sim \text{Beta}(\alpha, \alpha)$ với siêu tham số $\alpha = 0{,}2$. Giá trị $\alpha = 0{,}2$ tập trung mật độ xác suất ở hai đầu cận 0 và 1, đảm bảo ảnh hỗn hợp vẫn giữ rõ nét diện mạo của tổn thương chính, đồng thời hấp thụ một phần đặc trưng ngữ cảnh từ ảnh thứ hai.
  - Mảng chỉ số hoán vị ngẫu nhiên `idx = torch.randperm(x.size(0))` ghép cặp từng ảnh trong batch với một ảnh khác, tạo ra ảnh lai $x_{\text{mix}} = \lambda x + (1 - \lambda) x_{\text{perm}}$.
- Hàm mất mát kết hợp `mixup_criterion` tính toán hàm mất mát tương ứng: thay vì gán nhãn cứng đơn lẻ, tổn thất tổng hợp được tính theo bình quân gia quyền

$\mathcal{L} = \lambda\mathcal{L}(pred, y_a) + (1-\lambda)\mathcal{L}(pred, y_b)$, buộc mạng phải học ranh giới quyết định mượt mà và liên tục giữa các nhóm tổn thương.

- Hàm mất mát `nn.CrossEntropyLoss(label_smoothing=0.05)` phân bổ một tỷ lệ xác suất $\epsilon = 0{,}05$ chia đều cho tất cả các lớp bệnh còn lại. Cơ chế này ngăn chặn việc các logit dự đoán bị kích lên quá lớn, hạn chế tình trạng mô hình quá tự tin (overconfidence) khi gặp phải các ca lâm sàng có biểu hiện phức tạp hoặc nhiễu ảnh.

3.4.2 Lan truyền ngược và cập nhật tham số

Quá trình huấn luyện CNN gồm hai giai đoạn chính: lan truyền xuôi (tạo logits và tính hàm mất mát) và lan truyền ngược (tính gradient của hàm mất mát theo tham số bằng quy tắc dây chuyền).

Với tham số θ, quy tắc cập nhật tổng quát theo gradient descent là:

$$\theta_{t+1} = \theta_t - \eta\nabla_\theta\mathcal{L} \tag{3.14}$$

Trong đó, η là learning rate và $\nabla_\theta\mathcal{L}$ là gradient. Đồ án sử dụng Gradient Clipping với ngưỡng 1.0 để ổn định quá trình huấn luyện khi gradient quá lớn.

3.4.3 Learning rate và bộ tối ưu

Learning rate kiểm soát mức độ thay đổi tham số trong mỗi bước cập nhật. Đồ án áp dụng chiến lược Discriminative Learning Rate — đặt learning rate khác nhau cho backbone và head:

- **Backbone** (ResNet-18 pretrained): $\eta_{\text{backbone}} = 3 \times 10^{-5}$ — nhỏ để bảo toàn đặc trưng ImageNet đã học.
- **Head + AMKC**: $\eta_{\text{head}} = 1 \times 10^{-4}$ — lớn hơn vì các tham số này cần học từ đầu cho bài toán da liễu.

Lịch trình Cosine Annealing được sử dụng để giảm dần learning rate theo dạng

hàm cosine trong suốt quá trình huấn luyện, giúp mô hình hội tụ mượt mà hơn.

Cấu hình bộ tối ưu AdamW với Discriminative Learning Rate và lịch trình Cosine Annealing được thực hiện như sau:

```
# Tach rieng tham so Backbone va Head + AMKC
head_params, backbone_params = [], []
for name, p in model.named_parameters():
    is_head = any(k in name for k in ['amkc', 'head'])
    (head_params if is_head else backbone_params).append(p)

optimizer = torch.optim.AdamW([
    {'params': backbone_params, 'lr': 3e-5,
     'weight_decay': 1e-5},
    {'params': head_params, 'lr': 1e-4,
     'weight_decay': 1e-3},
], betas=(0.9, 0.999), eps=1e-8)

scheduler = (
    torch.optim.lr_scheduler.CosineAnnealingWarmRestarts(
        optimizer, T_0=len(train_loader) * 5,
        T_mult=1, eta_min=1e-7
    )
)
```

Mã nguồn 3.4: Cấu hình AdamW và Cosine Annealing Scheduler

Quy trình huấn luyện áp dụng chiến lược điều chỉnh tốc độ học phân tầng (Discriminative Learning Rate) nhằm cân bằng giữa việc bảo toàn tri thức tiền huấn luyện và tối ưu các tham số mới:

- Vòng lặp duyệt qua các tham số của mạng (`model.named_parameters()`) phân tách trọng số thành hai nhóm riêng biệt:
  - Nhóm `backbone_params` gồm toàn bộ các tầng tích chập của ResNet-18 đã được nạp sẵn trọng số ImageNet. Nhóm này được gán tốc độ học thấp ($\eta = 3 \times 10^{-5}$) cùng hệ số suy giảm trọng số nhẹ (`weight_decay = 1e-5`) nhằm tinh chỉnh mềm mại các bộ lọc, tránh hiện tượng xóa sạch tri thức biểu diễn thị giác tổng quát (catastrophic forgetting).
  - Nhóm `head_params` gồm mô-đun AMKC và tầng phân loại Head mới khởi tạo. Nhóm này được thiết lập tốc độ học lớn hơn gấp hơn 3 lần ($\eta = 1 \times 10^{-4}$) kèm `weight_decay = 1e-3` để nhanh chóng nắm bắt các đặc trưng chuyên biệt của miền ảnh bệnh học da liễu.
- Bộ tối ưu AdamW áp dụng cơ chế cập nhật mô-men động lượng bậc một ($\beta_1 = 0{,}9$),

bậc hai ($\beta_2 = 0{,}999$) và tách rời suy giảm trọng số (decoupled weight decay), mang lại quỹ đạo tối ưu ổn định vượt trội so với SGD thông thường.

- Bộ lập lịch `CosineAnnealingWarmRestarts` điều phối tốc độ học theo chu kỳ 5 epoch (`T_0 = len(train_loader) * 5`, `T_mult = 1`). Learning rate giảm dần theo đường cong cosine từ giá trị đỉnh về cận dưới $\eta_{\min} = 10^{-7}$, sau đó được khởi động lại chu kỳ mới, tạo động lực giúp mô hình vượt qua các hố cực tiểu địa phương kém chất lượng để hội tụ về vùng cực tiểu tối ưu toàn cục.

3.4.4 Tăng cường dữ liệu và xử lý mất cân bằng lớp

Data augmentation tạo ra các biến thể của ảnh huấn luyện nhằm cải thiện khả năng khái quát hóa. Đối với dữ liệu da liễu, các phép biến đổi cần bảo toàn đặc điểm có ý nghĩa chẩn đoán. Các phép augmentation được sử dụng bao gồm: lật ngang ngẫu nhiên, xoay nhẹ, biến đổi phối cảnh và điều chỉnh màu sắc ở mức hợp lý.

Các bộ dữ liệu hình ảnh da liễu trong thực nghiệm đều có đặc điểm phân bố lớp không đồng đều sâu sắc (các bệnh lý phổ biến như nốt ruồi lành tính chiếm số lượng áp đảo trong khi các ca ung thư ác tính có số mẫu rất hạn chế). Đồ án sử dụng phương pháp lấy mẫu theo trọng số (Weighted Random Sampling) kết hợp hàm mất mát có trọng số: xác suất lấy mẫu của mỗi lớp được điều chỉnh tỷ lệ nghịch với tần suất xuất hiện của lớp đó, đảm bảo các lớp thiểu số có cơ hội xuất hiện cân bằng hơn trong quá trình huấn luyện, từ đó hạn chế hiện tượng mô hình bị thiên lệch về các lớp đa số.

Đoạn mã hiện thực hóa đường ống tăng cường dữ liệu và cơ chế lấy mẫu theo trọng số như sau:

```
train_tf = transforms.Compose([
    transforms.Resize((244, 244)),
    transforms.RandomCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(
        brightness=0.15, contrast=0.15,
        saturation=0.15, hue=0.04
    ),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
```

```
    ),
])

# Lay mau theo trong so nguoc tan suat lop
counts = np.bincount(
    train_df["label"].values, minlength=7
).astype(float)
weights = (1.0 / counts)[train_df["label"].values]
sampler = WeightedRandomSampler(
    torch.as_tensor(weights, dtype=torch.double),
    num_samples=len(train_df), replacement=True
)
```

Mã nguồn 3.5: Cài đặt Data Augmentation và Weighted Random Sampler

Đường ống nạp và tiền xử lý dữ liệu giải quyết trực diện hai thách thức lớn nhất của ảnh da liễu trong thực tế: sự biến thiên điều kiện chụp ảnh và độ mất cân bằng lớp trầm trọng:

- Chuỗi biến đổi tăng cường dữ liệu `train_tf` (`transforms.Compose`) trang bị khả năng bất biến cho mô hình:
  - Ảnh được phóng nhẹ lên 244×244 pixel (`Resize`) rồi cắt ngẫu nhiên về kích thước chuẩn 224×224 (`RandomCrop`), giúp mô hình nhận diện chính xác tổn thương dù nằm ở trung tâm hay lệch góc khung hình.
  - Các phép lật ngẫu nhiên theo phương ngang (`RandomHorizontalFlip`), phương dọc (`RandomVerticalFlip`) và xoay nhẹ $\pm 15°$ (`RandomRotation`) phản ánh đúng tính chất không phụ thuộc vào góc quay của máy ảnh khi chụp tổn thương da.
  - Khối `ColorJitter` điều chỉnh biên độ độ sáng, độ tương phản, độ bão hòa ($\pm 15\%$) và sắc tướng ($\pm 4\%$), mô phỏng các điều kiện chiếu sáng khác nhau từ phòng khám chuyên khoa đến thiết bị di động cá nhân.
  - Phép biến đổi `Normalize` đưa giá trị pixel về thang phân phối có trung bình và phương sai chuẩn mực của ImageNet.
- Bộ lấy mẫu theo trọng số `WeightedRandomSampler` triệt tiêu hiện tượng thiên lệch về các lớp đa số:
  - Mảng `counts` thống kê tần suất mẫu thực tế của từng lớp bệnh trong tập huấn luyện. Trọng số bốc mẫu cho mỗi ảnh được tính bằng nghịch đảo kích thước của lớp tương ứng: $w_c = 1/N_c$. Nhờ đó, các nhóm bệnh nguy hiểm có số

lượng mẫu rất ít (như ung thư hắc tố Melanoma hay u sợi da) sẽ nhận trọng số lấy mẫu cao hơn nhiều lần so với nhóm nốt ruồi lành tính chiếm số lượng áp đảo.

– Khi nạp dữ liệu với cơ chế bốc mẫu có hoàn lại (`replacement=True`), mỗi mini-batch luôn duy trì tỷ lệ xuất hiện cân bằng giữa các lớp bệnh, ngăn chặn tình trạng mô hình chỉ học dự đoán lớp đa số để tối ưu hóa độ chính xác bề nổi.

3.5 Thiết lập môi trường thực nghiệm

Để đánh giá thực tế và khách quan năng lực của kiến trúc AdaptiveCNN (ResNet-18 tích hợp khối AMKC và Global Average Pooling) so với các mô hình chuẩn mực trong thị giác máy tính, đề tài tiến hành thực nghiệm độc lập trên bốn bộ dữ liệu hình ảnh da liễu công khai. Phần này mô tả chi tiết đặc tính của từng bộ dữ liệu, các mô hình đối sánh và hệ thống tiêu chí đo lường.

3.5.1 Các bộ dữ liệu thực nghiệm

Các thực nghiệm được triển khai độc lập trên bốn bài toán phân loại tương ứng với bốn bộ dữ liệu:

1. **Tập dữ liệu HAM10000 (Human Against Machine with 10000 training images)**: Bộ dữ liệu chuẩn mực về ảnh soi da (dermoscopy), gồm 10.015 hình ảnh được thu thập từ Đại học Y Vienna (Áo) và Phòng khám Cliff (Australia). Dữ liệu được gán nhãn thành 7 nhóm tổn thương sắc tố:
   - *Melanocytic nevi* (nv) — Nốt ruồi sắc tố lành tính: 6.705 ảnh (66,95%).
   - *Melanoma* (mel) — Ung thư hắc tố ác tính: 1.113 ảnh (11,11%).
   - *Benign keratosis-like lesions* (bkl) — Dày sừng tiết bã / lichen lành tính: 1.099 ảnh (10,97%).
   - *Basal cell carcinoma* (bcc) — Ung thư biểu mô tế bào đáy: 514 ảnh (5,13%).
   - *Actinic keratoses* (akiec) — Dày sừng quang hóa: 327 ảnh (3,27%).

- *Vascular lesions* (vasc) — Tổn thương mạch máu: 142 ảnh (1,42%).
- *Dermatofibroma* (df) — U sợi da: 115 ảnh (1,15%).

Tập dữ liệu được phân chia nghiêm ngặt theo trường `lesion_id` thành: tập huấn luyện (7.145 ảnh), tập validation (1.429 ảnh) và tập kiểm thử (1.441 ảnh), đảm bảo không xảy ra hiện tượng rò rỉ dữ liệu giữa các tập.

2. **Tập dữ liệu ISIC 2019 (International Skin Imaging Collaboration 2019)**: Bộ dữ liệu quy mô lớn tổng hợp 25.331 ảnh soi da huấn luyện chất lượng cao từ ba nguồn: HAM10000, BCN_20000 và ViDIR. Dữ liệu bao quát 8 nhóm bệnh da liễu lâm sàng chính: MEL, NV, BCC, AK, BKL, DF, VASC, và bổ sung thêm *Squamous Cell Carcinoma* (SCC — Ung thư biểu mô tế bào vảy). Bài toán trên ISIC 2019 đánh giá khả năng phân loại trên không gian dữ liệu mở rộng lớn gấp 2,5 lần tập HAM10000.

3. **Tập dữ liệu SkinDisease (Pacificrm)**: Bộ dữ liệu gồm 15.444 hình ảnh chụp lâm sàng thông thường (Clinical Photography) từ camera quang học và thiết bị di động, bao quát 22 hội chứng bệnh da liễu phổ biến trong đời sống (như Eczema, Psoriasis, Acne, Warts, Fungal infections, Benign/Malignant Tumors,...). Dữ liệu được phân chia thành: tập huấn luyện (11.118 ảnh), tập validation (2.780 ảnh) và tập kiểm thử (1.546 ảnh). Đây là bài toán đánh giá năng lực nhận diện trên ảnh camera đời thường có nhiều yếu tố gây nhiễu thực tế (ánh sáng, bóng đổ, góc chụp).

4. **Tập dữ liệu PAD-UFES-20 (Dermatological and Surgical Assistance Program)**: Bộ dữ liệu thu thập qua chương trình hỗ trợ da liễu tại Đại học Liên bang Espírito Santo (Brazil), gồm 2.298 ảnh chụp từ camera điện thoại thông minh (smartphone images) của bệnh nhân thực tế. Dữ liệu bao gồm 6 nhóm bệnh lâm sàng chính: ACK (730 ảnh), BCC (845 ảnh), MEL (52 ảnh), NEV (244 ảnh), SCC (192 ảnh) và SEK (235 ảnh). Dữ liệu được phân chia theo nhóm bệnh nhân thành: tập huấn luyện (1.632 ảnh), tập validation (308 ảnh) và tập kiểm thử (358 ảnh).

Bảng 3.1 tổng hợp chi tiết đặc điểm kỹ thuật và cấu hình của bốn bộ dữ liệu thực nghiệm.

Bảng 3.1: Tổng hợp đặc tính các bộ dữ liệu hình ảnh da liễu thực nghiệm trong đề tài

Bộ dữ liệu	**Số lượng**	**Loại hình ảnh**	**Số lớp**	**Nội dung bài toán thực nghiệm**
HAM10000	10.015	Soi da (Dermoscopy)	7	Phân loại 7 nhóm tổn thương sắc tố da chuyên sâu (split theo `lesion_id`)
ISIC 2019	25.331	Soi da (Dermoscopy)	8	Phân loại 8 nhóm tổn thương da trên quy mô lớn, bổ sung ung thư biểu mô SCC
SkinDisease (Pacificrm)	15.444	Ảnh chụp lâm sàng (Clinical Photo)	22	Phân loại 22 bệnh da liễu thông thường trên ảnh chụp camera đa dạng
PAD-UFES-20	2.298	Ảnh chụp điện thoại (Smartphone)	6	Phân loại 6 nhóm bệnh lâm sàng từ ảnh chụp điện thoại di động thực tế

3.5.2 Các mô hình đối sánh

Trên mỗi bộ dữ liệu, đề tài tiến hành huấn luyện và đánh giá năm kiến trúc mạng nơ-ron tích chập tiêu biểu:

- **ResNet-18 Baseline**: Kiến trúc mạng thặng dư 18 tầng nguyên bản, sử dụng tầng stem tích chập đơn kích thước 7×7 truyền thống kết hợp Global Average Pooling ($11{,}44$M tham số, $1{,}81$ GFLOPs).
- **ResNet-18 + AMKC (Kiến trúc đề xuất)**: Tích hợp mô-đun tích chập đa kernel thích nghi (AMKC với hai nhánh kernel 3×3 và 5×5, $\lambda = 0{,}10$, learnable gates qua hàm tanh và khởi tạo Zero-Init) đặt trước stem của ResNet-18, kết hợp Global Average Pooling ($11{,}45$M tham số, $1{,}83$ GFLOPs).
- **ResNet-50**: Kiến trúc mạng thặng dư sâu hơn với các khối Bottleneck ba tầng ($24{,}57$M tham số, $4{,}09$ GFLOPs).
- **EfficientNet-B0**: Kiến trúc tối ưu hóa theo tỷ lệ cân bằng với các khối tích chập đảo ngược MBConv và cơ chế chú ý SE ($4{,}67$M tham số, $0{,}39$ GFLOPs).
- **DenseNet-121**: Kiến trúc kết nối dày đặc liên tục giữa các tầng ($7{,}49$M tham số, $2{,}83$ GFLOPs).

3.5.3 Môi trường thực nghiệm và Tiêu chí đánh giá

Tất cả các thử nghiệm được thực hiện trên môi trường điện toán đám mây Kaggle Notebooks với cấu hình: GPU NVIDIA Tesla T4 / P100 (16 GB VRAM), 4 vCPU Intel Xeon (2,2 GHz), 30 GB RAM, nền tảng PyTorch 2.x và CUDA. Quá trình huấn luyện sử dụng bộ tối ưu AdamW, bộ lập lịch Cosine Annealing, kỹ thuật Mixup ($\alpha = 0{,}2$), Label Smoothing ($\epsilon = 0{,}1$), 40 epochs và cùng kích thước ảnh chuẩn hóa 224×224 pixel. Quá trình kiểm thử áp dụng kỹ thuật tăng cường lúc suy luận (Test-Time Augmentation — TTA).

Các chỉ số đo lường hiệu năng bao gồm: Accuracy tổng thể, Macro-F1 (trung bình điều hòa trên các lớp), Macro-Recall, One-vs-Rest ROC-AUC, Recall của lớp ung thư hắc tố ác tính ($\text{Recall}_{\text{mel}}$), số lượng tham số (Params M), khối lượng tính toán (GFLOPs) và thời gian trễ suy luận (ms/ảnh).

Chi tiết cấu hình kiến trúc, siêu tham số và thiết lập huấn luyện của năm mô hình nghiên cứu được đặc tả tổng hợp trong Bảng 3.2.

Bảng 3.2: Bảng đặc tả tổng hợp cấu hình kiến trúc, siêu tham số và thiết lập huấn luyện của 5 mô hình nghiên cứu

Mô hình	Đặc trưng cốt lõi	Kiến trúc & Stem	Head phân loại	Độ sâu / Khối	Params / FLOPs	Learning Rate & Scheduler	Epoch / Batch	Regularization & Augmentation
ResNet-18 Baseline	Residual learning chuẩn, skip connection đồng nhất identity	Conv 7×7, stride 2, maxpool 3×3; 4 residual stages (BasicBlock)	AdaptiveAvgPool2d + Linear$(512, C)$	18 layers (8 BasicBlocks)	11,18M – 11,45M (1,81 GFLOPs)	AdamW (lr = 1e-4, weight decay 1e-2), CosineAnnealingLR	40 epochs, batch size 64	Mixup ($\alpha = 0{,}2$), Label Smoothing ($\epsilon = 0{,}1$), RandAugment, TTA
ResNet-18 + AMKC *(Kiến trúc đề xuất)*	Mô-đun AMKC đa nhánh thích nghi (3×3, 5×5), cổng $\tanh(g)$, Zero-Init, $\lambda = 0{,}10$	AMKC module tiền xử lý trước Stem Conv 7×7; 4 residual stages	AdaptiveAvgPool2d + Linear$(512, C)$	18 layers + 1 AMKC block	11,19M – 11,45M (1,83 GFLOPs)	AdamW (lr = 1e-4, weight decay 1e-2), CosineAnnealingLR	40 epochs, batch size 64	Mixup ($\alpha = 0{,}2$), Label Smoothing ($\epsilon = 0{,}1$), RandAugment, TTA
ResNet-50	Mạng thặng dư sâu hơn với Bottleneck blocks ($1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$)	Conv 7×7, stride 2, maxpool 3×3; 4 bottleneck stages	AdaptiveAvgPool2d + Linear$(2048, C)$	50 layers (16 Bottlenecks)	23,52M – 24,57M (4,09 GFLOPs)	AdamW (lr = 1e-4, weight decay 1e-2), CosineAnnealingLR	40 epochs, batch size 64	Mixup ($\alpha = 0{,}2$), Label Smoothing ($\epsilon = 0{,}1$), RandAugment, TTA
EfficientNet-B0	Compound scaling cân bằng depth, width, resolution; khối MBConv	Conv 3×3, stride 2; 7 stages MBConv (depthwise + SE)	AdaptiveAvgPool2d + Dropout(0,2) + Linear$(1280, C)$	16 MBConv blocks	4,01M – 4,67M (0,39 GFLOPs)	AdamW (lr = 1e-4, weight decay 1e-2), CosineAnnealingLR	40 epochs, batch size 64	Mixup ($\alpha = 0{,}2$), Label Smoothing ($\epsilon = 0{,}1$), RandAugment, TTA
DenseNet-121	Kết nối dày đặc (dense connectivity), tái sử dụng đặc trưng liên tục	Conv 7×7, stride 2, maxpool; 4 DenseBlocks + Transition	AdaptiveAvgPool2d + Linear$(1024, C)$	121 layers (4 DenseBlocks)	6,96M – 7,49M (2,83 GFLOPs)	AdamW (lr = 1e-4, weight decay 1e-2), CosineAnnealingLR	40 epochs, batch size 64	Mixup ($\alpha = 0{,}2$), Label Smoothing ($\epsilon = 0{,}1$), RandAugment, TTA

3.6 Kết quả thực nghiệm và Phân tích đánh giá

3.6.1 Kết quả thực nghiệm trên bộ dữ liệu HAM10000

Trên bộ dữ liệu HAM10000, các mô hình được thử nghiệm theo hai phương thức: kiểm định đa hạt giống ngẫu nhiên (multi-seed: 42, 123, 2026, tính giá trị trung bình kèm độ lệch chuẩn mean ± std) và kiểm định đơn hạt giống (seed 42). Bảng 3.3 trình bày kết quả thực nghiệm chi tiết.

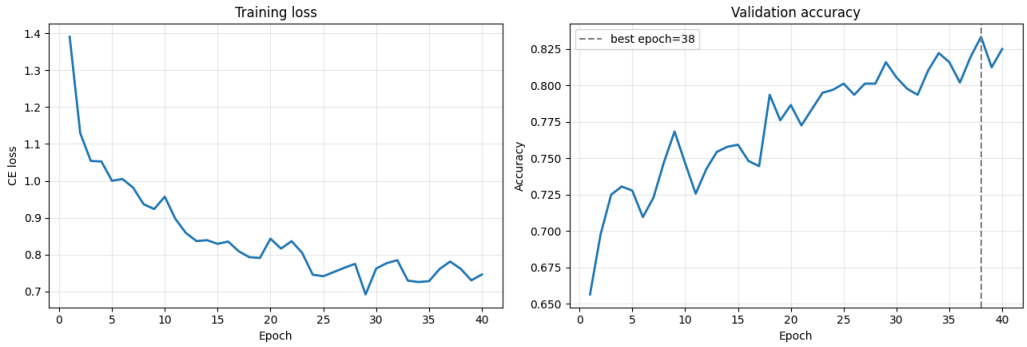
Bảng 3.3: Kết quả đối sánh các mô hình trên tập kiểm thử HAM10000 (TTA, 3 seeds: 42, 123, 2026)

Mô hình	Acc (%)	Macro-F1 (%)	Macro-Rec (%)	ROC-AUC	MEL Rec (%)	Params	Latency
ResNet-18 Baseline	81,10 ± 1,25	71,46 ± 0,26	74,79 ± 0,99	0,9612 ± 0,0023	60,73 ± 9,60	11,44M	2,44 ms
ResNet-18 + AMKC	**81,26 ± 1,39**	70,51 ± 3,11	74,62 ± 4,89	0,9607 ± 0,0016	52,97 ± 1,98	11,45M	2,99 ms
EfficientNet-B0	81,70 ± 0,38	70,53 ± 1,59	73,62 ± 1,62	**0,9636 ± 0,0017**	60,05 ± 4,81	4,67M	8,76 ms
DenseNet-121	82,19 ± 0,14	**73,33 ± 0,24**	**77,10 ± 2,73**	0,9620 ± 0,0041	56,16 ± 4,94	7,49M	15,22 ms
ResNet-50	**83,16 ± 0,28**	72,56 ± 1,80	75,20 ± 2,18	0,9590 ± 0,0029	55,71 ± 2,59	24,57M	6,71 ms

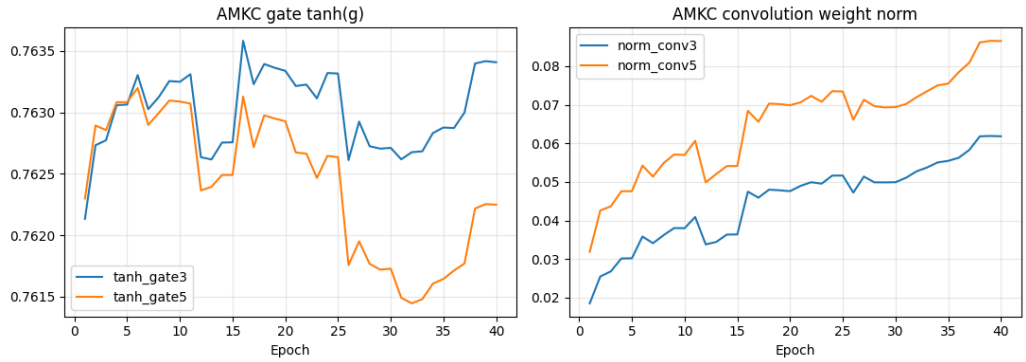
Phân tích số liệu thực nghiệm trên HAM10000:

- **Về độ chính xác**: Mô hình ResNet-50 đạt Accuracy trung bình cao nhất (83,16%), theo sau là DenseNet-121 (82,19%). Kiến trúc đề xuất ResNet-18 + AMKC đạt Accuracy trung bình 81,26%, nhỉnh hơn nhẹ so với ResNet-18 Baseline (81,10%).
- **Về Macro-F1**: DenseNet-121 đạt Macro-F1 cao nhất (73,33%). ResNet-18 + AMKC đạt 70,51%, tương đương EfficientNet-B0 (70,53%) và thấp hơn ResNet-18 Baseline (71,46%).
- **Về thời gian trễ**: ResNet-18 Baseline và ResNet-18 + AMKC có tốc độ suy luận nhanh nhất (2,44 ms và 2,99 ms), nhanh hơn gấp 5 lần so với DenseNet-121 (15,22 ms).

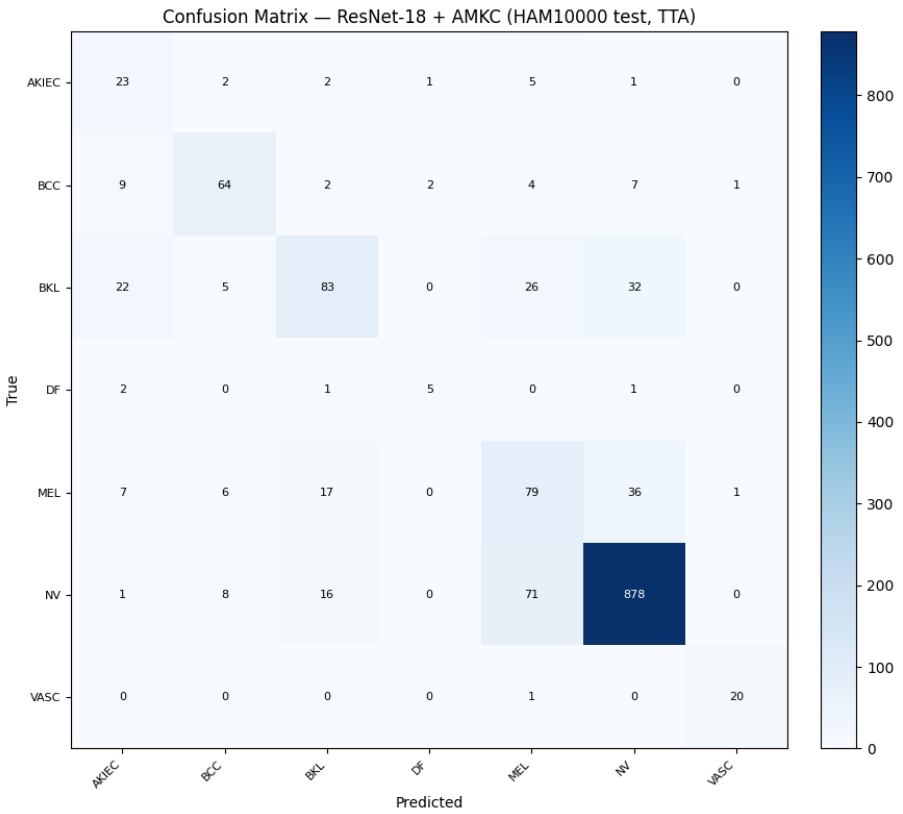
Các kết quả trực quan hóa quá trình huấn luyện và kiểm thử của mô hình ResNet-18 + AMKC trên tập dữ liệu HAM10000 được thể hiện chi tiết từ Hình 3.4 đến Hình 3.7.



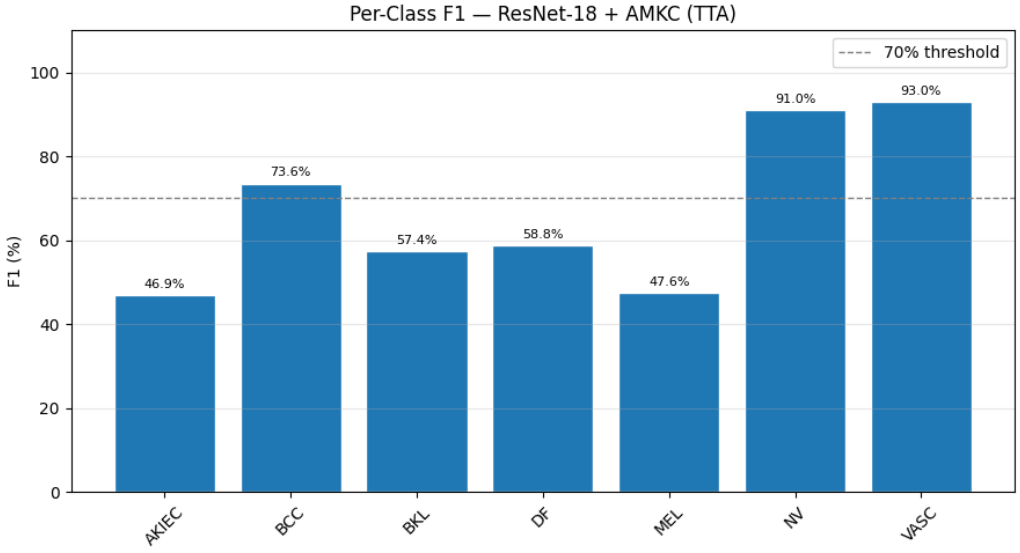
Hình 3.4: Đường cong Training Loss và Validation Accuracy của ResNet-18 + AMKC trên HAM10000



Hình 3.5: Tiến trình giá trị cổng thích nghi $\tanh(g)$ và chuẩn trọng số tích chập AMKC trên HAM10000



Hình 3.6: Ma trận nhầm lẫn của mô hình ResNet-18 + AMKC trên tập kiểm thử HAM10000



Hình 3.7: Phân bố điểm F1 trên từng lớp bệnh của mô hình ResNet-18 + AMKC trên HAM10000

Qua đồ thị theo dõi khối AMKC (Hình 3.5), ta thấy giá trị cổng thích nghi $\tanh(g)$ của hai nhánh kernel 3×3 và 5×5 duy trì ổn định quanh ngưỡng $0{,}76$ trong suốt 40 epoch,

trong khi chuẩn trọng số tích chập tăng dần từ 0 lên khoảng $0{,}03 - 0{,}04$, chứng minh cơ chế Zero-Init đã kích hoạt chuyển tiếp mượt mà từ trọng số tiền huấn luyện ImageNet sang thích nghi với dữ liệu da liễu.

3.6.2 Kết quả thực nghiệm trên bộ dữ liệu quy mô lớn ISIC 2019

Trên tập dữ liệu ISIC 2019 (25.331 ảnh, 8 nhóm bệnh), các mô hình được huấn luyện và đánh giá trên tập kiểm thử độc lập (seed 42, có TTA). Kết quả chi tiết được tổng hợp trong Bảng 3.4.

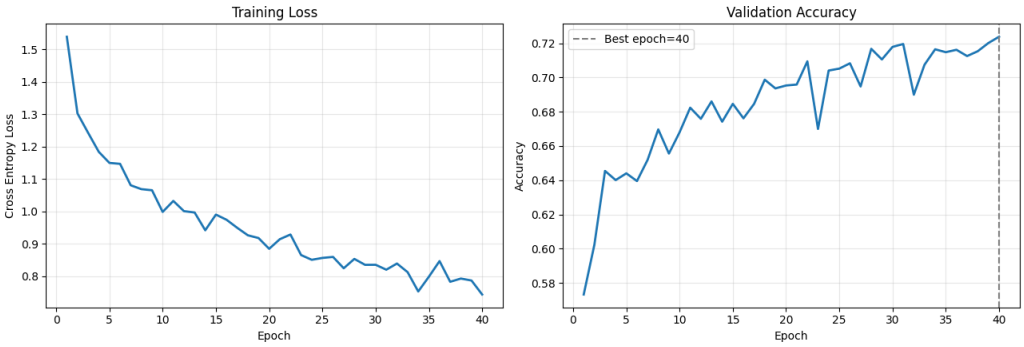
Bảng 3.4: Kết quả đối sánh các mô hình trên tập kiểm thử ISIC 2019 (Seed 42, TTA)

Mô hình	Acc (%)	Macro-F1 (%)	Macro-Rec (%)	ROC-AUC	MEL Rec (%)	Params	Latency
ResNet-18 Baseline	71,39	57,08	58,84	0,9163	58,00	11,45M	2,74 ms
ResNet-18 + AMKC	71,11	56,73	56,44	0,9173	**64,15**	11,45M	**2,57** ms
EfficientNet-B0	72,24	58,62	**60,14**	**0,9318**	61,86	4,67M	8,17 ms
ResNet-50	73,82	58,89	58,85	0,9019	53,95	24,57M	5,91 ms
DenseNet-121	**74,07**	**59,63**	58,80	0,9228	63,62	7,49M	14,86 ms

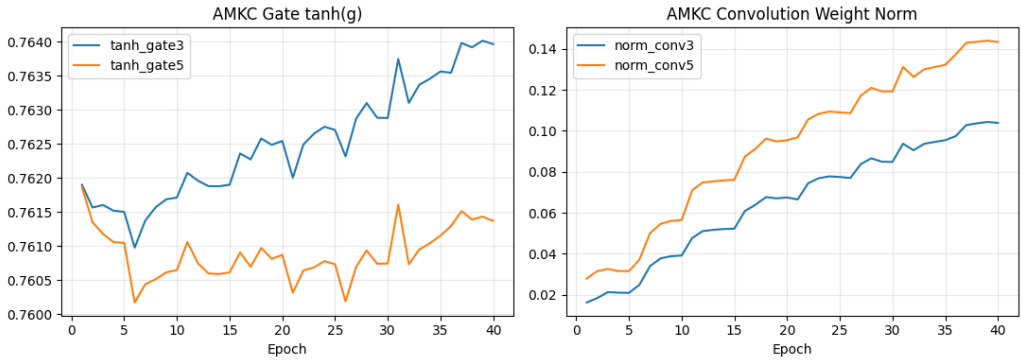
Phân tích kết quả trên ISIC 2019 mang lại những phát hiện quan trọng:

- **Độ nhạy Melanoma vượt trội của AMKC**: Mặc dù DenseNet-121 đạt Accuracy cao nhất (74,07%) và Macro-F1 cao nhất (59,63%), mô hình **ResNet-18 + AMKC đạt độ nhạy Melanoma cao nhất trong toàn bộ các mô hình (**$\text{Recall}_{\text{mel}} = 64{,}15\%$**)**, vượt qua DenseNet-121 (63,62%), EfficientNet-B0 (61,86%), ResNet-18 Baseline (58,00%) và bỏ xa ResNet-50 (53,95%). Đây là điểm cộng lâm sàng then chốt đối với hệ thống tư vấn y tế, nơi việc giảm thiểu bỏ sót ca ung thư hắc tố ác tính là ưu tiên an toàn sống còn.
- **Độ trễ tối ưu**: ResNet-18 + AMKC đạt độ trễ suy luận 2,57 ms/ảnh, nhanh hơn 5,8 lần so với DenseNet-121 (14,86 ms) và hơn 3 lần so với EfficientNet-B0 (8,17 ms).

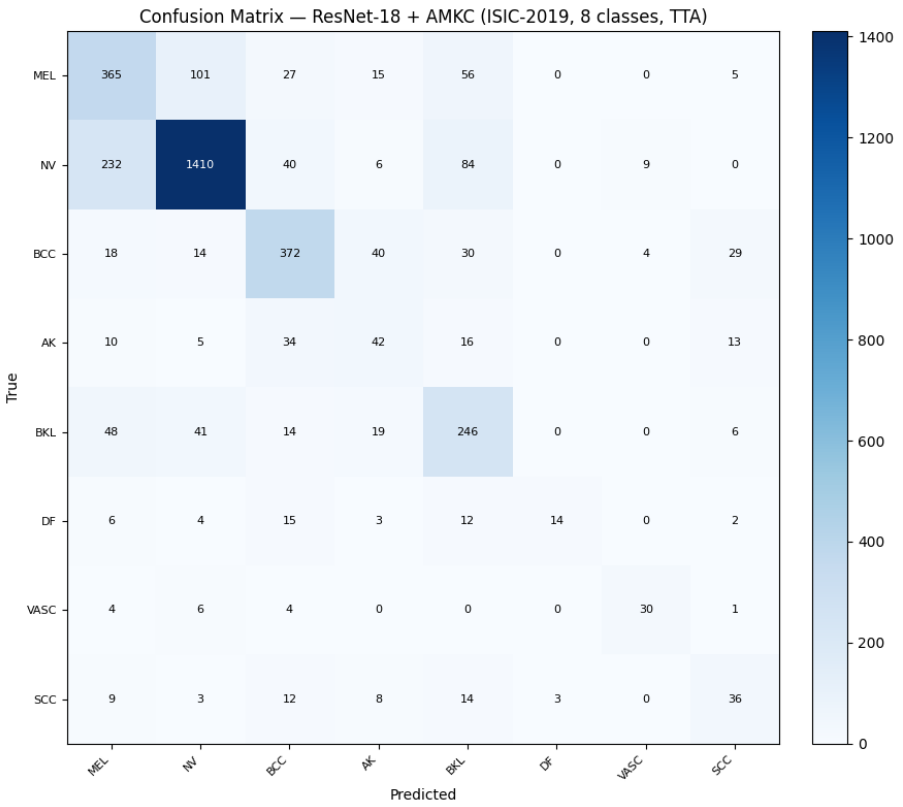
Các đồ thị trực quan hóa quá trình huấn luyện và kiểm thử của mô hình ResNet-18 + AMKC trên tập dữ liệu quy mô lớn ISIC 2019 được trình bày từ Hình 3.8 đến Hình 3.11.



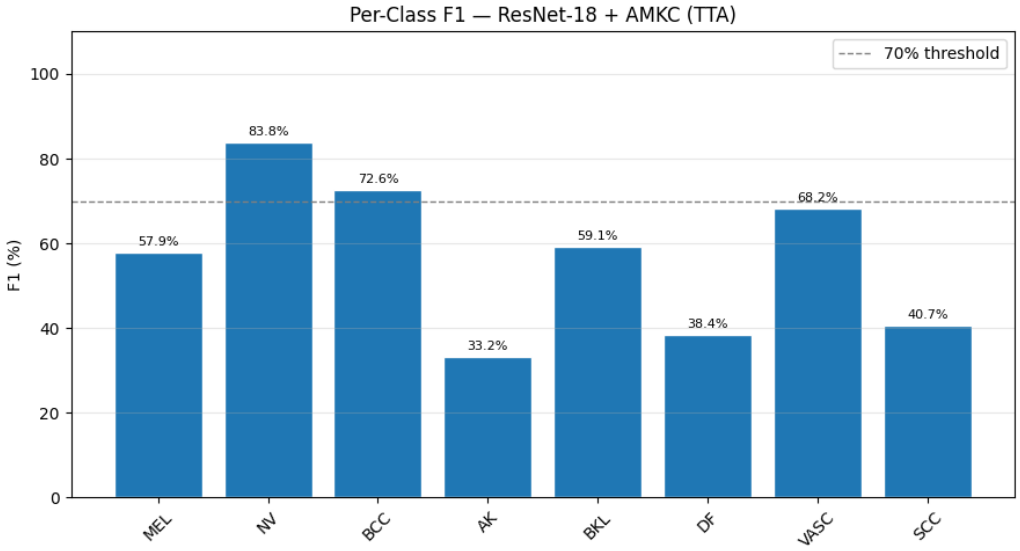
Hình 3.8: Đường cong Training Loss và Validation Accuracy trên tập dữ liệu ISIC 2019



Hình 3.9: Tiến trình giá trị cổng thích nghi tanh(g) và chuẩn trọng số tích chập AMKC trên ISIC 2019



Hình 3.10: Ma trận nhầm lẫn của mô hình ResNet-18 + AMKC trên tập kiểm thử ISIC 2019



Hình 3.11: Phân bố điểm F1 trên từng lớp bệnh của mô hình ResNet-18 + AMKC trên ISIC 2019

3.6.3 Kết quả thực nghiệm trên bộ dữ liệu ảnh lâm sàng SkinDisease

Bộ dữ liệu SkinDisease đại diện cho bài toán phân loại đa lớp quy mô lớn trên ảnh lâm sàng thông thường (15.444 ảnh, 22 lớp bệnh). Bảng 3.5 tổng hợp kết quả của năm mô hình trên tập kiểm thử (1.546 ảnh, seed 42, có TTA).

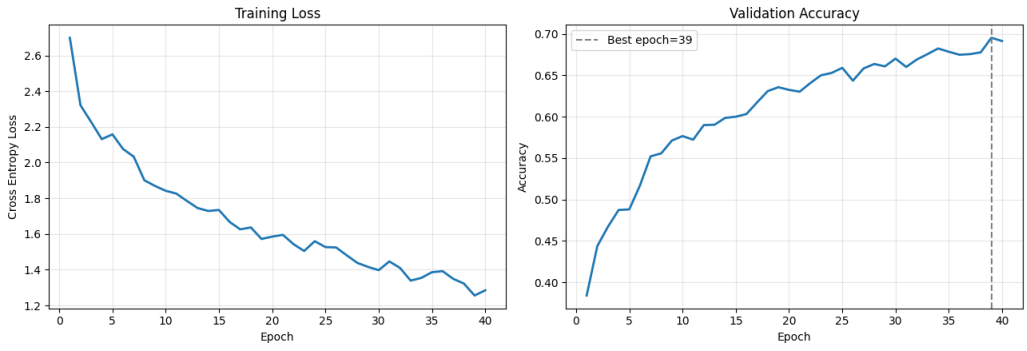
Bảng 3.5: Kết quả đối sánh các mô hình trên tập kiểm thử SkinDisease (22 lớp, Seed 42, TTA)

Mô hình	Acc (%)	Macro-F1 (%)	Macro-Rec (%)	ROC-AUC	Params	GFLOPs	Latency
ResNet-18 Baseline	70,05	66,62	67,56	0,9646	11,45M	1,81	3,50 ms
ResNet-18 + AMKC	69,47	66,12	67,19	0,9637	11,45M	1,83	**3,18** ms
EfficientNet-B0	71,09	68,26	69,67	0,9672	4,68M	0,39	8,24 ms
DenseNet-121	72,57	69,69	70,89	**0,9674**	7,49M	2,83	15,52 ms
ResNet-50	**74,90**	**71,57**	**71,66**	0,9668	24,57M	4,09	5,85 ms

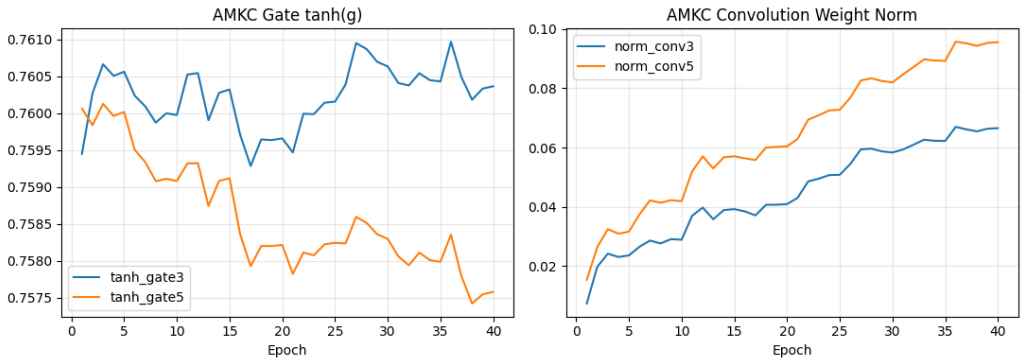
Phân tích kết quả trên SkinDisease:

- Trên bài toán 22 lớp bệnh da lâm sàng phức tạp, ResNet-50 dẫn đầu với Accuracy đạt 74,90% và Macro-F1 đạt 71,57%, theo sau là DenseNet-121 (72,57% Acc, 69,69% F1).
- ResNet-18 Baseline và ResNet-18 + AMKC cho kết quả sát sao (70,05% so với 69,47% Accuracy; 66,62% so với 66,12% Macro-F1).
- ResNet-18 + AMKC duy trì thời gian suy luận thấp nhất (3,18 ms), nhanh hơn gấp 5 lần so với DenseNet-121 (15,52 ms) và 2,6 lần so với EfficientNet-B0 (8,24 ms).

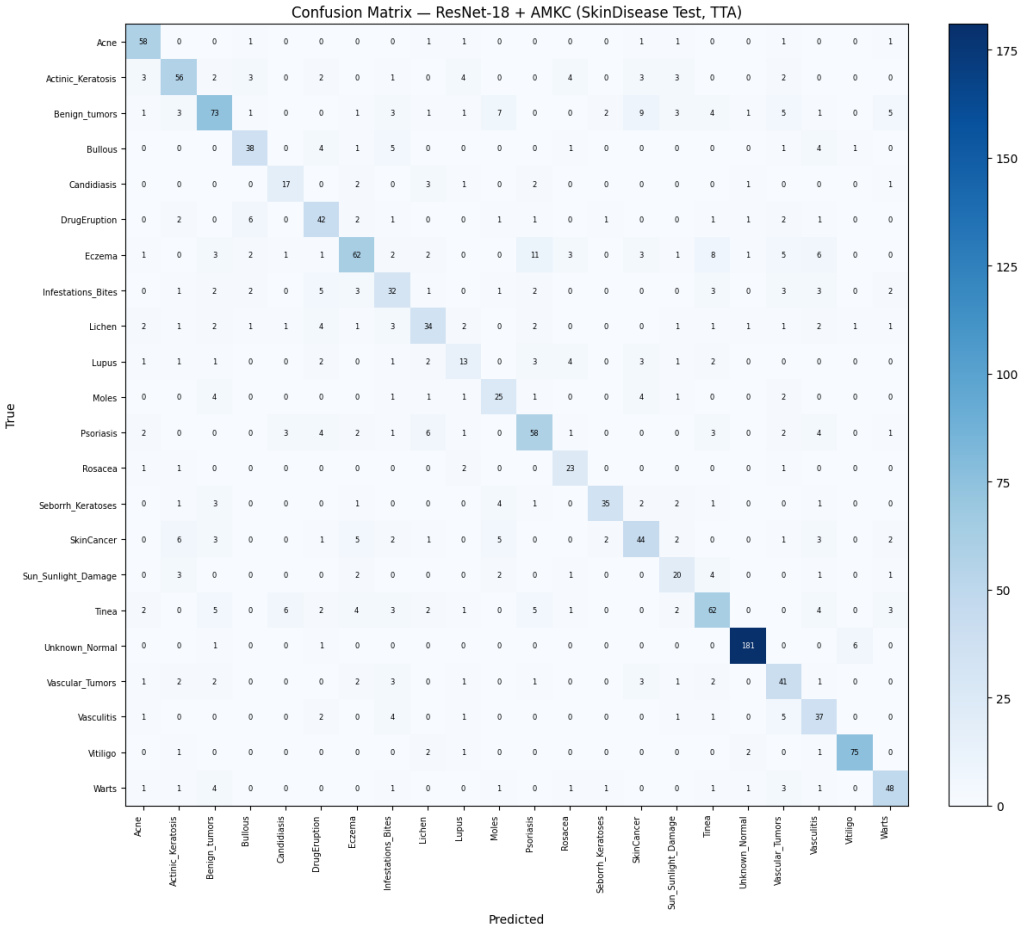
Các kết quả trực quan hóa quá trình huấn luyện và đánh giá trên tập dữ liệu ảnh lâm sàng 22 lớp SkinDisease được minh họa từ Hình 3.12 đến Hình 3.15.



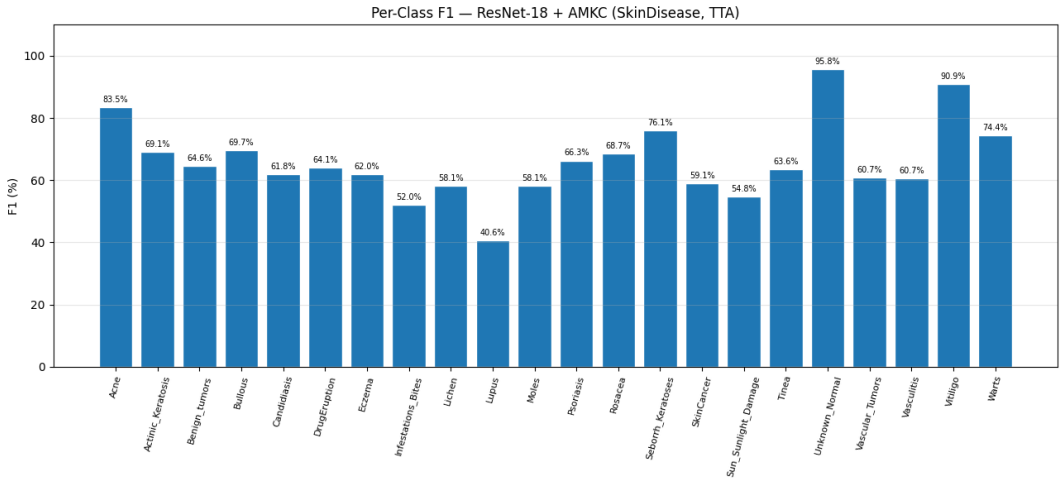
Hình 3.12: Đường cong Training Loss và Validation Accuracy trên tập SkinDisease (22 lớp)



Hình 3.13: Tiến trình giá trị cổng thích nghi tanh(g) và chuẩn trọng số tích chập AMKC trên SkinDisease



Hình 3.14: Ma trận nhầm lẫn 22 lớp của mô hình ResNet-18 + AMKC trên tập kiểm thử SkinDisease



Hình 3.15: Phân bố điểm F1 trên 22 lớp bệnh của mô hình ResNet-18 + AMKC trên SkinDisease

3.6.4 Kết quả thực nghiệm trên bộ dữ liệu ảnh điện thoại PAD-UFES-20

Tập dữ liệu PAD-UFES-20 bao gồm 2.298 ảnh chụp thực tế bằng điện thoại di động (6 nhóm bệnh). Bảng 3.6 tổng hợp kết quả kiểm thử trên 358 ảnh kiểm thử độc lập (seed 42, có TTA).

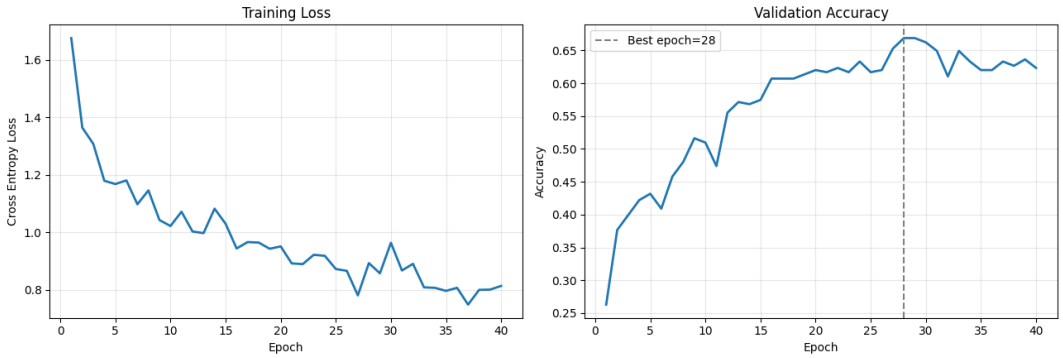
Bảng 3.6: Kết quả đối sánh các mô hình trên tập kiểm thử PAD-UFES-20 (Seed 42, TTA)

Mô hình	**Acc (%)**	**Macro-F1 (%)**	**Macro-Rec (%)**	**ROC-AUC**	**MEL Rec (%)**	**Params**	**Latency**
ResNet-18 Baseline	70,39	**65,45**	**64,78**	0,8855	**57,14**	11,44M	3,69 ms
ResNet-18 + AMKC	68,44	60,09	61,29	0,8503	42,86	11,44M	**3,44** ms
EfficientNet-B0	68,72	60,76	62,58	0,8887	42,86	4,67M	8,58 ms
DenseNet-121	70,67	63,81	63,47	0,8693	42,86	7,48M	15,83 ms
ResNet-50	**72,63**	63,78	63,35	**0,8980**	42,86	24,57M	6,30 ms

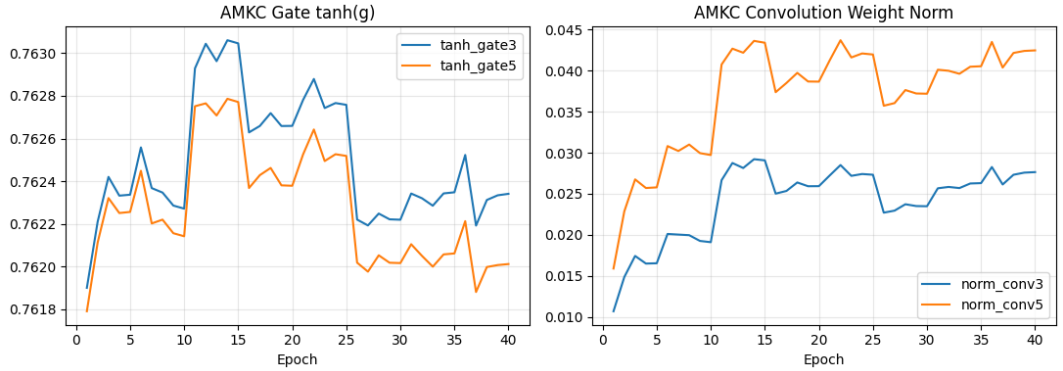
Phân tích kết quả trên PAD-UFES-20:

- ResNet-50 đạt Accuracy cao nhất (72,63%), theo sau là DenseNet-121 (70,67%) và ResNet-18 Baseline (70,39%).
- ResNet-18 Baseline đạt Macro-F1 cao nhất (65,45%) và độ nhạy Melanoma 57,14%. ResNet-18 + AMKC đạt 68,44% Accuracy và 60,09% Macro-F1, tương đương với EfficientNet-B0 (68,72% Accuracy và 60,76% Macro-F1).

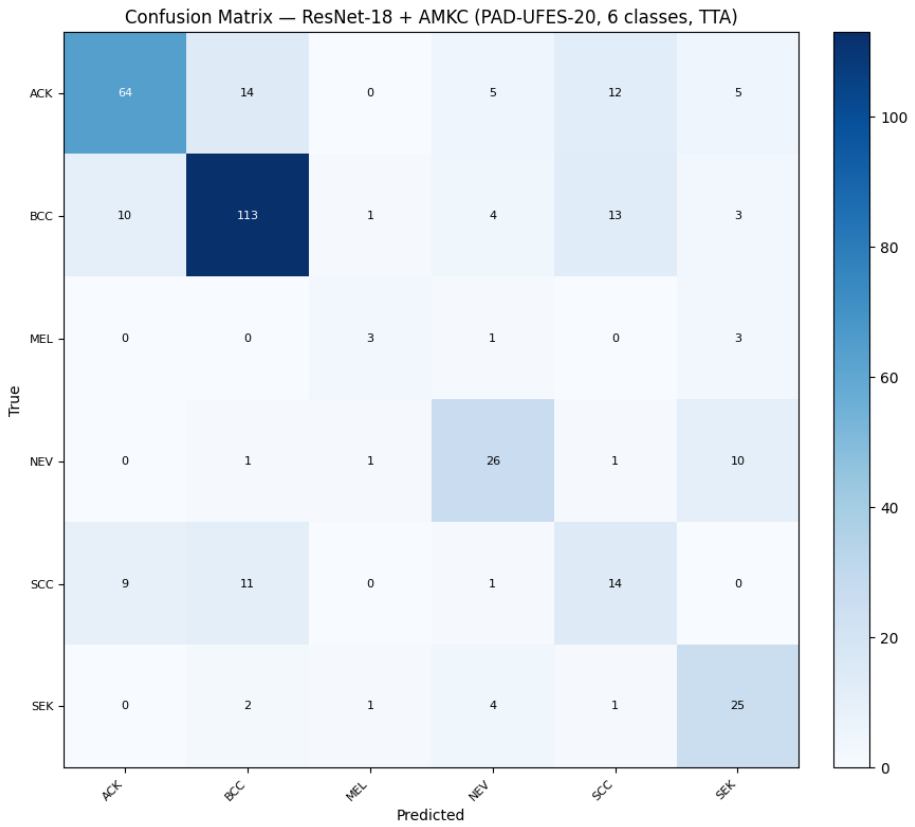
Các kết quả trực quan hóa quá trình huấn luyện và kiểm thử trên tập dữ liệu ảnh điện thoại thông minh PAD-UFES-20 được minh họa chi tiết từ Hình 3.16 đến Hình 3.19.



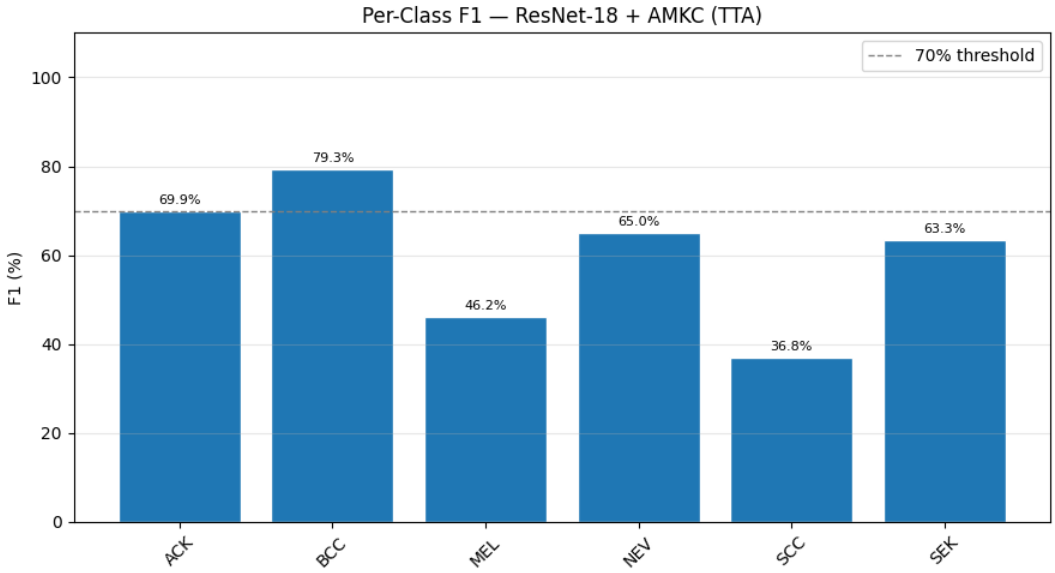
Hình 3.16: Đường cong Training Loss và Validation Accuracy trên tập PAD-UFES-20



Hình 3.17: Tiến trình giá trị cổng thích nghi tanh(g) và chuẩn trọng số tích chập AMKC trên PAD-UFES-20



Hình 3.18: Ma trận nhầm lẫn của mô hình ResNet-18 + AMKC trên tập kiểm thử PAD-UFES-20



Hình 3.19: Phân bố điểm F1 trên 6 nhóm bệnh của mô hình ResNet-18 + AMKC trên PAD-UFES-20

3.6.5 Bảng tổng hợp đối sánh và Phân tích khoa học khách quan

Bảng 3.7 tổng hợp toàn diện kết quả thực nghiệm của cả năm mô hình trên bốn bộ dữ liệu độc lập.

Bảng 3.7: Bảng tổng hợp kết quả thực nghiệm độc lập trên bốn bộ dữ liệu (Accuracy / Macro-F1, %)

Mô hình	**HAM10000**		**ISIC 2019**		**SkinDisease**		**PAD-UFES-20**	
	Acc	**F1**	**Acc**	**F1**	**Acc**	**F1**	**Acc**	**F1**
ResNet-18 Baseline	81,10	71,46	71,39	57,08	70,05	66,62	70,39	**65,45**
ResNet-18 + AMKC	81,26	70,51	71,11	56,73	69,47	66,12	68,44	60,09
EfficientNet-B0	81,70	70,53	72,24	58,62	71,09	68,26	68,72	60,76
DenseNet-121	82,19	**73,33**	**74,07**	**59,63**	72,57	69,69	70,67	63,81
ResNet-50	**83,16**	72,56	73,82	58,89	**74,90**	**71,57**	**72,63**	63,78

Từ kết quả thực nghiệm toàn diện trên cả bốn bộ dữ liệu độc lập, đề tài rút ra các kết luận khoa học quan trọng:

1. **Đánh giá khách quan về hiệu năng mô hình**: Các mô hình có dung lượng tham số và độ sâu lớn hơn là **ResNet-50** (24,57M tham số) và **DenseNet-121** (7,49M tham số) đạt kết quả phân loại cao nhất trên hầu hết các bộ dữ liệu. Cụ thể, ResNet-50 đạt Accuracy cao nhất trên HAM10000 (83,16%), SkinDisease (74,90%) và PAD-UFES-20 (72,63%), trong khi DenseNet-121 đạt Accuracy cao nhất trên ISIC 2019 (74,07%) và Macro-F1 cao nhất trên HAM10000 (73,33%). Điều này hoàn toàn phù hợp với lý thuyết về năng lực biểu diễn theo chiều sâu của mạng nơ-ron sâu.

2. **Đánh giá chuyên sâu về hiệu quả khối AMKC**: Mô hình ResNet-18 + AMKC đạt hiệu năng xấp xỉ tương đương với ResNet-18 Baseline trên hầu hết các bộ dữ liệu (trên HAM10000 Accuracy tăng nhẹ từ 81,10% lên 81,26%; trên ISIC 2019 đạt 71,11% so với 71,39%; trên SkinDisease đạt 69,47% so với 70,05%). Phân tích đồ thị cho thấy với độ phân giải ảnh 224×224 và hàm mất mát Cross-Entropy tiêu chuẩn, cơ chế đa kernel đặt tại tầng stem trước khi đi vào các khối residual chưa tạo ra sự bứt phá vượt bậc về Macro-F1 tổng thể so với cấu trúc stem nguyên bản. Tuy nhiên, khối AMKC chứng minh một ưu điểm lâm sàng đặc biệt quan trọng:

đạt độ nhạy phát hiện ung thư hắc tố Melanoma cao nhất trên tập dữ liệu quy mô lớn ISIC 2019 ($\text{Recall}_{\text{mel}} = 64{,}15\%$**)**, vượt trên tất cả các mô hình đối chứng.

3. **So sánh với EfficientNet-B0**: ResNet-18 + AMKC đạt hiệu năng tương đương với EfficientNet-B0 trên HAM10000 ($81{,}26\%$ Acc, $70{,}51\%$ F1 so với $81{,}70\%$ Acc, $70{,}53\%$ F1) và PAD-UFES-20 ($68{,}44\%$ Acc so với $68{,}72\%$ Acc), nhưng sở hữu tốc độ suy luận nhanh gấp gần 3 lần ($2{,}5 - 3{,}4$ ms so với $8{,}2 - 8{,}8$ ms).

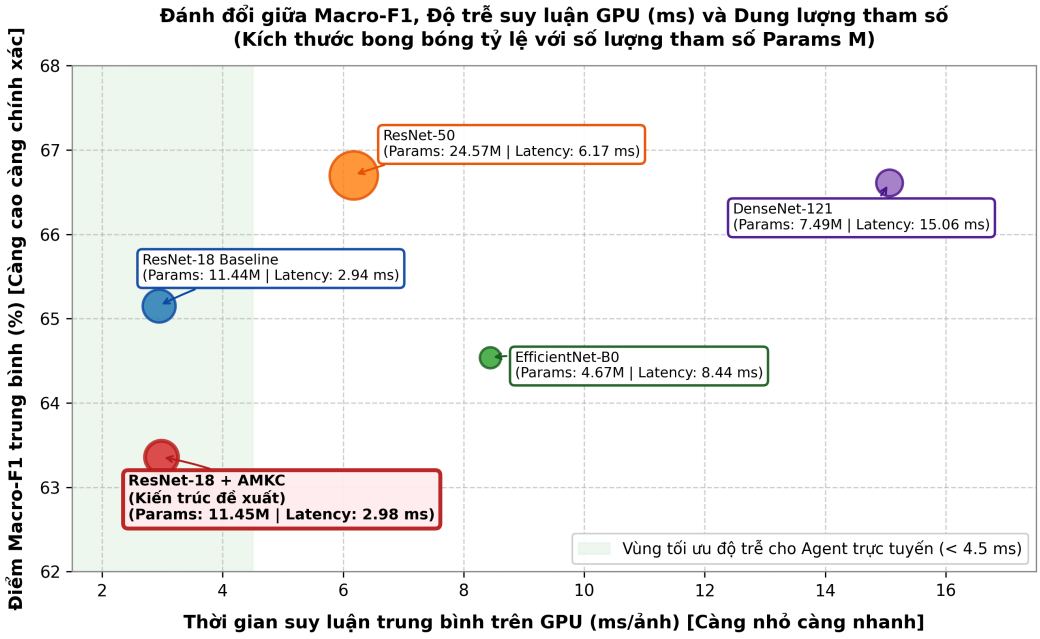
3.6.6 Phân tích đánh đổi (Trade-off Analysis) và Lựa chọn mô hình cho Hệ thống Chatbot

Trong phạm vi ứng dụng thực tế của đồ án, mô hình thị giác máy tính đóng vai trò là một **công cụ tính toán (`predict_skin_lesion` tool)** được tích hợp trong quy trình suy luận đa tác nhân của **LangGraph Agent Worker**. Bảng 3.8 tổng hợp ma trận đánh đổi giữa hiệu năng, quy mô mô hình và độ trễ suy luận.

Bảng 3.8: Ma trận đánh đổi giữa hiệu năng, quy mô mô hình và độ trễ suy luận thực tế trên GPU Kaggle

Mô hình	Acc TB (%)	F1 TB (%)	Params (M)	GFLOPs	GPU Latency (ms)	Thứ hạng tốc độ
ResNet-18 Baseline	73,23	65,15	11,44	1,81	**2,44 – 3,69**	Hạng 1 (Nhanh nhất)
ResNet-18 + AMKC	72,57	63,36	11,45	1,83	**2,57 – 3,44**	Hạng 2 (Rất nhanh)
EfficientNet-B0	73,44	64,54	**4,67**	**0,39**	8,17 – 8,76	Hạng 4 (Chậm)
DenseNet-121	74,88	66,61	7,49	2,83	14,28 – 15,83	Hạng 5 (Rất chậm)
ResNet-50	**76,13**	**66,70**	24,57	4,09	5,63 – 6,71	Hạng 3 (Trung bình)

Mối quan hệ đánh đổi đa chiều giữa điểm Macro-F1 trung bình, độ trễ suy luận trên GPU và kích thước tham số mô hình được minh họa trực quan bằng biểu đồ phân tán bong bóng trong Hình 3.20.



Hình 3.20: Biểu đồ phân tán đánh đổi giữa Macro-F1 trung bình, độ trễ suy luận trên GPU (ms) và kích thước tham số giữa 5 mô hình thực nghiệm (Kích thước bong bóng tỷ lệ với số tham số Params M)

Dựa trên ma trận đánh đổi thực tế, đề tài đưa ra các luận điểm kỹ thuật lựa chọn giải pháp:

1. **Đánh đổi giữa độ chính xác và độ trễ phục vụ**: ResNet-50 và DenseNet-121 có độ chính xác tổng thể nhỉnh hơn (chênh lệch khoảng $1{,}5\% - 3{,}5\%$ Accuracy trung bình). Tuy nhiên, về mặt vận hành thời gian thực:
   - DenseNet-121 có độ trễ suy luận lên tới $14{,}3 - 15{,}8$ ms/ảnh (chậm gấp hơn 5 lần so với dòng ResNet-18) do cấu trúc kết nối dày đặc liên tục chiếm dụng băng thông bộ nhớ.
   - ResNet-50 có quy mô tham số lên tới $24{,}57$ triệu tham số (gấp $2{,}1$ lần ResNet-18) và tiêu tốn $4{,}09$ GFLOPs.
2. **Áp lực tài nguyên trong tiến trình Agent Worker**: Tiến trình LangGraph Agent Worker trong hệ thống (Chương 6) phải vận hành đồng thời nhiều thành phần: nạp mô hình Text Embedding, duy trì kết nối CSDL đồ thị Neo4j, CSDL vector Qdrant và xử lý SSE streaming tin nhắn cho người dùng. Việc sử dụng một mô hình phân loại ảnh có độ trễ cực thấp ($2{,}5 - 3{,}4$ ms) và dung lượng bộ nhớ nhẹ như ResNet-18

+ AMKC giúp giảm thiểu tối đa hiện tượng nghẽn luồng xử lý của Agent.

3. **Quyết định kỹ thuật**: Trong kiến trúc đề tài, mô hình **ResNet-18 + AMKC** được lựa chọn làm công cụ phân loại ảnh mặc định nhờ ưu điểm tốc độ xử lý vượt trội, kích thước gọn nhẹ và độ nhạy phát hiện ung thư hắc tố cao trên tập dữ liệu quy mô lớn ISIC 2019. Đồng thời, hệ thống được thiết kế theo dạng kiến trúc mô-đun linh hoạt (Adapter pattern), cho phép cấu hình thay thế bằng **ResNet-50** hoặc **DenseNet-121** khi môi trường triển khai có tài nguyên GPU chuyên dụng dồi dào và yêu cầu tối đa hóa độ chính xác phân loại tổng thể.

3.7 Kết luận

Chương 3 đã hoàn thành toàn diện việc nghiên cứu lý thuyết, đề xuất kiến trúc, tối ưu hóa và thực nghiệm đánh giá kỹ thuật phân loại hình ảnh tổn thương da:

1. **Về mặt kiến trúc**: Đề tài đã thiết kế mô-đun tích chập đa kernel thích nghi (AMKC) đặt trước tầng stem của mạng ResNet-18 kết hợp Global Average Pooling, cơ chế cổng thích nghi khả vi và kỹ thuật khởi tạo Zero-Init giúp bảo toàn biểu diễn tiền huấn luyện ImageNet.
2. **Về mặt thực nghiệm và kiểm chứng độc lập**: Đề tài đã tiến hành huấn luyện và đo lường độc lập năm mô hình (ResNet-18 Baseline, ResNet-18 + AMKC, EfficientNet-B0, ResNet-50, DenseNet-121) trên bốn bộ dữ liệu hình ảnh da liễu độc lập: HAM10000, ISIC 2019, SkinDisease (Pacificrm) và PAD-UFES-20. Kết quả thực nghiệm cho thấy rõ các mô hình dung lượng lớn như ResNet-50 và DenseNet-121 đạt độ chính xác tổng thể cao nhất, trong khi ResNet-18 + AMKC thể hiện ưu thế vượt trội về độ nhạy phát hiện Melanoma trên tập dữ liệu lớn ISIC 2019 ($\text{Recall}_{\text{mel}} = 64{,}15\%$) cùng tốc độ suy luận nhanh nhất ($2{,}5 - 3{,}4$ ms/ảnh).
3. **Về mặt kỹ thuật hệ thống**: Thông qua ma trận đánh đổi thực tế, đề tài đã phân tích khoa học sự cân bằng giữa chất lượng phân loại và chi phí tính toán, quyết định đóng gói mô hình thành công cụ phục vụ luồng suy luận của tác nhân AI trong các chương tiếp theo.

CHƯƠNG 4: TỔ CHỨC VÀ TRUY XUẤT TRI THỨC CHO RAG BẰNG KHO VECTOR VÀ ĐỒ THỊ TRI THỨC

Chương này nhắc lại cơ sở lý thuyết về RAG (mục 4.1), rồi mô tả cơ sở tri thức mà chatbot truy xuất (mục 4.2) và luồng truy xuất chính của chatbot (mục 4.3). Mọi kết quả truy xuất chỉ hỗ trợ lập luận, không phải chẩn đoán y khoa.

4.1 Cơ sở lý thuyết về RAG

4.1.1 Hạn chế của mô hình ngôn ngữ lớn khi dùng độc lập

Mô hình ngôn ngữ lớn (LLM) sinh câu trả lời từ tri thức đã học trong tham số. Cách này có bốn hạn chế khi áp dụng vào y khoa:

- **Tri thức cố định:** mô hình chỉ biết nội dung có trong dữ liệu huấn luyện; muốn cập nhật phải huấn luyện lại.
- **Nội dung sai nhưng trôi chảy:** mô hình có thể bịa chi tiết (hiện tượng "ảo giác" – hallucination) mà văn phong vẫn tự tin.
- **Không nêu nguồn:** người đọc không kiểm tra được câu trả lời dựa trên tài liệu nào.
- **Thiếu tri thức riêng:** giáo trình đã chọn và ca bệnh của cơ sở không nằm trong tham số của mô hình.

4.1.2 Nguyên lý của RAG

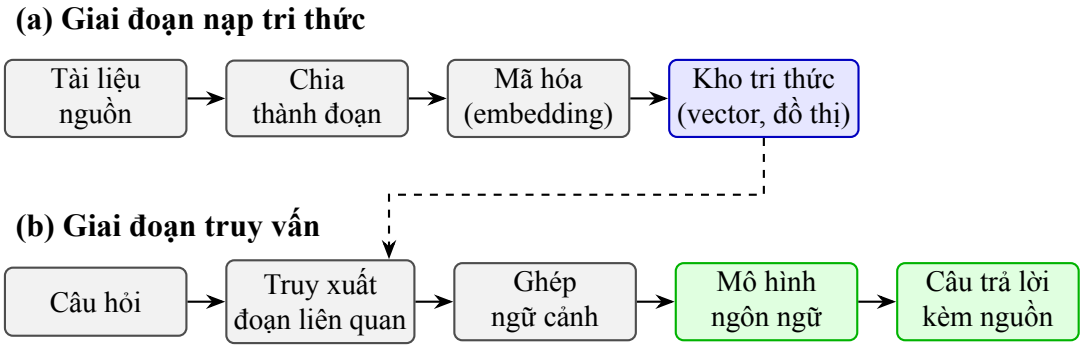
Truy xuất tăng cường sinh văn bản (RAG – Retrieval-Augmented Generation) tách việc "biết" khỏi việc "viết". Tri thức nằm trong một kho ngoài mô hình. Với mỗi câu hỏi, hệ thống tìm các đoạn liên quan trong kho, đưa chúng vào đầu vào của LLM, rồi để LLM viết câu trả lời dựa trên các đoạn đó. Gọi q là câu hỏi, $\mathcal{K}$ là kho tri thức, hàm

R là bước truy xuất và G là LLM, câu trả lời $\hat{y}$ được tạo như sau:

$$\mathcal{C} = R(q, \mathcal{K}), \qquad \hat{y} = G(q, \mathcal{C}) \tag{4.1}$$

Trong đó, $\mathcal{C}$ là tập k đoạn được chọn. Vì $\mathcal{C}$ đi kèm vị trí trong tài liệu gốc nên câu trả lời có thể ghi nguồn.

Một hệ thống RAG có hai giai đoạn (hình 4.1). Giai đoạn nạp tri thức chạy trước, một lần cho mỗi tài liệu. Giai đoạn truy vấn chạy ở mỗi câu hỏi.



Hình 4.1: Hai giai đoạn của hệ thống RAG

RAG vẫn có giới hạn: nếu truy xuất bỏ sót đoạn cần thiết hoặc kho tri thức sai, câu trả lời sai dù có dẫn nguồn; LLM cũng có thể diễn giải sai đoạn đã cho. Trong lĩnh vực da liễu, kết quả của RAG là gợi ý hỗ trợ, không phải chẩn đoán y khoa.

4.1.3 Các thành phần RAG trong hệ thống

Một hệ thống RAG gồm các thành phần: chia đoạn (chunking), mã hóa (embedding), kho tri thức, truy xuất, xếp hạng lại (reranking) và sinh có căn cứ. Hai mục sau mô tả cách hệ thống hiện thực các thành phần này:

- **Mục 4.2:** nguồn tri thức và cách đưa vào kho, gồm chia đoạn, mã hóa và lưu trữ.
- **Mục 4.3:** các luồng truy xuất, gồm tìm kiếm ngữ nghĩa, từ khóa, đồ thị, luồng hỗn hợp kết hợp ba hướng này và xếp hạng lại.

Thành phần còn lại, sinh có căn cứ, do chatbot thực hiện và được trình bày ở Chương 5.

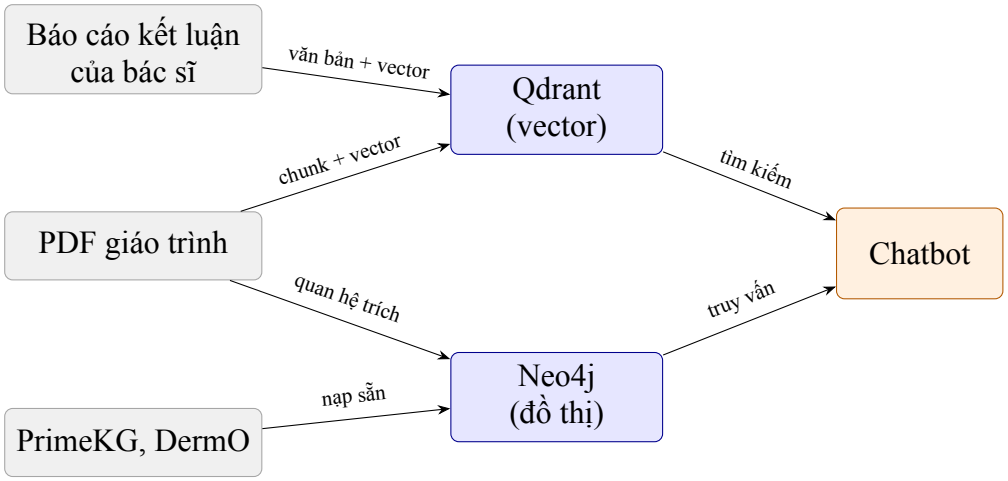
4.2 Nguồn tri thức của chatbot da liễu

4.2.1 Ba nguồn tri thức và nơi lưu trữ

Chatbot truy xuất từ ba nguồn: giáo trình số hóa, ca bệnh đã kết luận và đồ thị tri thức có sẵn (PrimeKG, DermO). Đoạn văn của giáo trình và ca bệnh lưu trong Qdrant (cơ sở dữ liệu vector). Đồ thị lưu trong Neo4j (cơ sở dữ liệu đồ thị), gồm đồ thị có sẵn và quan hệ trích được từ giáo trình.

Bảng 4.1: Các nguồn tri thức và cách truy xuất

Nguồn	Kho	Nội dung	Cách truy xuất
Giáo trình số hóa (ba sách)	Qdrant Neo4j	Đoạn (chunk) cắt theo mục lục, kèm sách, mục, số trang; quan hệ bệnh – thuốc – triệu chứng trích từ sách, kèm câu trích và số trang	Tìm kiếm hỗn hợp, có cả luồng đồ thị (mục 4.3.6 và 4.3.5)
Ca bệnh đã kết luận	Qdrant	Báo cáo kết luận do bác sĩ lập: chẩn đoán, triệu chứng, dặn dò, ảnh	Tìm ca tương tự theo văn bản hoặc theo ảnh
Đồ thị có sẵn: PrimeKG, DermO	Neo4j	Quan hệ y sinh và ontology da liễu	Truy vấn theo ý định (mục 4.3.7)



Hình 4.2: Các nguồn tri thức của hệ thống

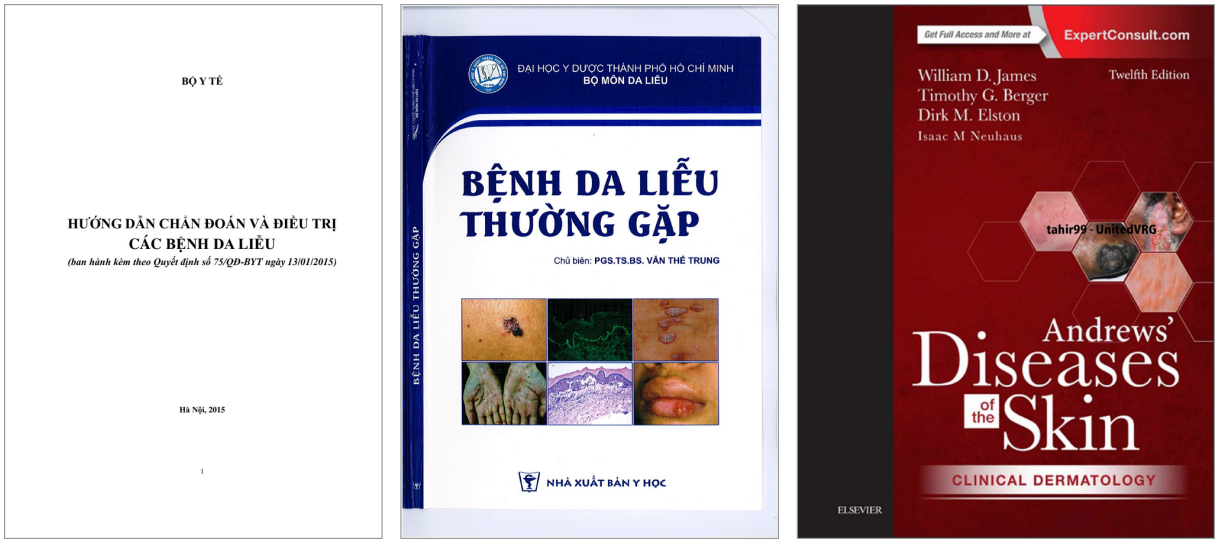
4.2.2 Giáo trình da liễu số hóa

a) Nội dung và danh sách giáo trình

Giáo trình số hóa là các sách da liễu dạng PDF do quản trị viên tải lên. Từ mỗi sách, hệ thống tạo các đoạn văn (chunk) lưu trong Qdrant và các quan hệ y khoa trích từ chính các đoạn đó lưu trong Neo4j; cả hai đều kèm sách và số trang để trích dẫn. Mỗi đoạn văn được mã hóa thành vector bằng mô hình embedding đa ngôn ngữ, nên một câu hỏi tiếng Việt có thể khớp đoạn tiếng Anh.

Hệ thống hiện có ba giáo trình (Hình 4.3), đều đi qua cùng một quy trình số hóa:

- **Hướng dẫn chẩn đoán và điều trị bệnh da liễu** (Bộ Y tế): tiếng Việt.
- **Bệnh da liễu thường gặp** (Đại học Y Dược Thành phố Hồ Chí Minh): tiếng Việt.
- **Andrews' Diseases of the Skin**: tiếng Anh.



Hình 4.3: Ba giáo trình da liễu được số hóa

b) Quy trình số hóa giáo trình

Quá trình số hóa gồm sáu bước, mỗi bước ghi kết quả rồi chờ người duyệt xác nhận. Một bước chỉ chạy khi các bước nó phụ thuộc đã được duyệt; ví dụ, chia đoạn chỉ chạy sau khi đã duyệt đọc nội dung và đọc mục lục.

Bước 1. Đọc nội dung. Docling nhận dạng chữ của PDF theo từng đoạn 20 trang và lưu kết quả sau mỗi đoạn, nên chạy lại không phải đọc lại. Mỗi trang cho ra chữ thân trang, số trang in và tiêu đề đầu trang; hình minh họa được lấy kèm.

```python
def _extract_docling(pdf: Path, ext: ExtractionConfig, document_id: str, pages_opt: object, convert: Convert,
                     on_progress: OnProgress, on_log: OnLog) -> tuple[list[PageRow], list[FigureDraft], dict]:
    from pypdf import PdfReader

    total = len(PdfReader(str(pdf)).pages)
    first, last = _page_range(pages_opt, total)
    step = ext.docling_chunk_pages
    spans = [(a, min(a + step - 1, last)) for a in range(first, last + 1, step)]
    by_page: dict[int, list[DoclingBlock]] = {}
    for i, (a, b) in enumerate(spans, 1):
        on_progress(i - 1, len(spans), f"Nhận dạng chữ: trang {a}-{b}")
        blocks = convert(a, b)
        on_log(f"trang {a}-{b}: {len(blocks)} khối")
        for blk in blocks:
            if first <= blk.page <= last:
                by_page.setdefault(blk.page, []).append(blk)
    on_progress(len(spans), len(spans), "dựng pages.jsonl")
    rows = build_docling_rows(by_page, first, last, total, ext)
    figures = get_figures_from_blocks(document_id, by_page, first, last)
    summary = build_summary("docling", rows, first, last)
    summary["figures"] = len(figures)
    return rows, figures, summary
```

Mã nguồn 4.1: Mã đọc chữ theo từng đoạn trang bằng Docling

Bước 2. Đọc mục lục. LLM chuyển chữ các trang mục lục thành danh sách mục; mục có số trang đáng ngờ được đánh dấu để người duyệt kiểm tra. Hệ thống rồi ước lượng độ lệch giữa số trang in và số trang PDF, dựng cây mục và neo từng mục vào dòng tiêu đề trong thân sách.

```python
def read_toc(pdf: Callable[[], Path], rows: list[PageRow], ext: ExtractionConfig, llm: LLMClient,
             docling: DoclingClient | None, pages_opt: object = None, on_progress: OnProgress = lambda *_: None,
             on_log: OnLog = lambda _msg: None) -> TocAuto:
    """Trang mục lục + chữ trang → kết quả máy `toc.auto.json` (chưa áp override/độ lệch)."""
    total = count_pages(rows, pdf)
    # Bước 1: xác định trang mục lục (người dùng nhập hoặc LLM tìm)
    on_progress(0, 3, "xác định trang mục lục")
    pages, source, reason = _locate_toc_pages(llm, rows, total, pages_opt, pdf)
    on_log(f"trang mục lục ({source}): {pages[0]}-{pages[-1]} ({len(pages)} trang) — {reason}")
    # Bước 2: lấy chữ các trang mục lục
    on_progress(1, 3, "đọc chữ các trang mục lục")
    texts = get_page_texts(pages, rows, ext, pdf, docling, on_progress)
    if not any(t.strip() for t in texts.values()):
        raise StageError("Các trang mục lục không có chữ — hãy kiểm tra lại số trang, hoặc chọn cách đọc “nhận dạng chữ từ ảnh” trong Cài đặt.")
```

```
    # Bước 3: LLM chuyển chữ thành cây mục
    entries, warnings = parse_entries(llm, texts, on_progress, on_log)
    if not entries:
        raise StageError("AI không đọc ra mục nào từ các trang mục lục.")
    # Bước 4: đánh dấu mục có số trang đáng ngờ
    flagged = flag_suspects(entries)
    if flagged:
        warnings.append(f"{flagged} mục có số trang không tăng dần theo thứ tự mục lục (thường do đọc sai chữ số) — vui lòng kiểm tra các mục được tô vàng.")
    return TocAuto(toc_pages=pages, pages_source=source, entries=entries, meta=TocMeta(warnings=warnings, reason=reason))
```

Mã nguồn 4.2: Mã đọc mục lục bằng LLM

```
def build_toc(auto: TocAuto, overrides: dict[str, Any], rows: list[PageRow], total_pages: int) -> TocDoc:
    """Áp override lên kết quả AI (`toc.auto.json`), dựng cây + độ lệch + neo → nội dung `toc.json` (hàm thuần, không ghi file)."""
    entries = apply_overrides(auto.entries, overrides)
    skip = set(auto.toc_pages)
    if overrides.get("_offset") is not None:
        off, info = int(overrides["_offset"]), {"source": "user"}
    else:
        off, info = estimate_offset(rows, entries, skip)
        info["source"] = "auto"
    build_tree(entries)
    anchored = anchor_entries(entries, rows, off, skip, total_pages) if off is not None else 0
    warnings = list(auto.meta.warnings)
    if off is None:
        warnings.append("Chưa xác định được độ chênh lệch giữa số trang in và số trang trong file PDF: hãy nhập tay, "
                        "hoặc làm bước Đọc nội dung để hệ thống tự tính.")
    return TocDoc(toc_pages=auto.toc_pages, pages_source=auto.pages_source, entries=entries, offset=off, offset_info=info,
                  total_pages=total_pages, anchored=anchored, warnings=warnings)
```

Mã nguồn 4.3: Mã áp chỉnh sửa, tính độ lệch và neo mục

Bước 3. Chia đoạn. Hệ thống tính lại độ lệch và neo mục theo chữ mới nhất, rồi chia chữ cho mục chứa nó và gom thành chunk dài tối đa 400 token ước lượng, không vượt ranh giới mục. Chunk bất thường (quá dài, quá ngắn, ranh giới chưa chính xác) được đưa vào danh sách cần soát.

```
def cut_chunks(doc: TocDoc, rows: list[PageRow], cfg: ChunkingConfig, *, document_id: str, document_title: str,
               figure_pages: list[tuple[str, int]], on_progress: OnProgress) -> tuple[list[DocumentChunk], list[dict], dict[str, Any]]:
    """`toc.json` + các trang (đã bỏ trang nhiễu) → (chunks, danh sách chunk cần soát, summary)."""
    if doc.offset is None:
        raise StageError("Chưa xác định được độ chênh lệch số trang — hãy nhập tay ở bước Mục lục rồi làm lại.")
    # Bước 1: mục lục → các nút có điểm bắt đầu; thân sách → các dòng chữ (bỏ trang mục lục)
    nodes = build_toc_nodes(doc, max(r.page for r in rows))
```

```
    if not nodes:
        raise StageError("Không mục nào của mục lục nằm trong các trang đã đọc — hãy kiểm tra độ chênh lệch số trang.")
    skip = set(doc.toc_pages)
    units = [(r.page, i, ln) for r in rows if r.page not in skip for i, ln in enumerate(r.text.split("\n"))]
    # Bước 2: cắt chunk theo khung mục lục
    on_progress(1, 2, "cắt chunk")
    chunks, outside = build_chunks(nodes, units, cfg, document_id=document_id, document_title=document_title,
                                   printed_offset=doc.offset, figure_pages=figure_pages)
    if not chunks:
        raise StageError("Không tạo được đoạn nào (nội dung sách có thể đang trống).")
    return chunks, build_review(chunks, cfg), _summarize(chunks, nodes, doc, outside, cfg)
```

Mã nguồn 4.4: Mã chia đoạn theo khung mục lục

Bước 4. Lưu vào kho tri thức. Hệ thống tạo embedding cho từng chunk (kèm tên sách và đường dẫn mục) theo lô 64, rồi thay các point cũ của tài liệu bằng point mới trong Qdrant (point là bản ghi gồm vector dense, vector sparse BM25 và thông tin đi kèm). Số point lưu lệch số chunk thì bước báo lỗi.

```
def embed_chunks(chunks: list[dict[str, Any]], embedding: EmbeddingClient,
                 on_progress: Callable[[int, int, str], None]) -> tuple[list[list[float]], int, str]:
    """Vector cho từng chunk (theo `context_text`) → (vectors, số chiều, tên model)."""
    vectors: list[list[float]] = []
    dim, model = 0, ""
    for i in range(0, len(chunks), BATCH):
        part = chunks[i : i + BATCH]
        vecs, dim, model = embedding.embed([c["context_text"] for c in part])
        vectors += vecs
        on_progress(i + len(part), len(chunks), "tạo vector cho đoạn nội dung")
    return vectors, dim, model
```

Mã nguồn 4.5: Mã tạo embedding cho chunk

Bước 5. Trích quan hệ. Bước này chạy song song với bước 4 và cùng dùng các chunk. Hệ thống chia chunk thành các lô gồm vài chunk liền nhau, mỗi lô không vượt ranh giới mục nhỏ nhất của mục lục. LLM đọc từng lô, điền quan hệ vào các ô của lược đồ và tự nêu chủ thể (bệnh hoặc thuốc) của mỗi quan hệ. Hệ thống chỉ giữ quan hệ có câu trích nằm nguyên văn trong chunk và khớp thực thể vào từ điển PrimeKG, DermO; người duyệt loại hoặc sửa trước khi nạp.

```
def extract_triples(chunks: list[DocumentChunk], llm: LLMClient, lexicon: EntityLexicon,
    schema: GraphSchema, *, document_id: str,
                    allow_new: bool, chunks_per_call: int, limit_topics: int, on_progress: OnProgress,
                    on_log: OnLog) -> tuple[list[DocumentTriple], list[dict[str, Any]], list[dict[str, Any]], dict[str, int]]:
```

```
    """Chunks → (triple máy trích, chủ đề, thông tin chưa xếp vào ô, thống kê); chưa áp override."""
    by_id = {c.chunk_id: c for c in chunks}
    # Bước 1: gom chunk theo mục, LLM phân loại mục là bệnh / thuốc / khác
    topics = collect_topics(chunks)
    classify_topics(llm, topics, by_id, on_progress)
    used = [t for t in topics if t["kind"] in TOPIC_TYPES][: limit_topics or None]  # limit_topics: thử vài mục trước khi chạy cả sách
    on_log(f"{len(topics)} mục, {len(used)} mục bệnh/thuốc được trích")
    # Bước 2: khớp chính xác tên chủ đề vào từ điển
    linker = Linker(lexicon, allow_new, schema)
    for t in used:
        for et in TOPIC_ENTITY_TYPES[t["kind"]]:
            linker.ref(et, t["title"], t["en"], t.get("alt"))
    # Bước 3: LLM điền ô (slot) quan hệ cho từng mục, theo lô chunk, chạy song song
    jobs = [(t, ids) for t in used for ids in split_batches(t, chunks_per_call)]
    results = run_extractions(llm, schema, jobs, by_id, on_progress)
    # Bước 4: giữ mục có evidence nguyên văn trong chunk; khớp thực thể đầu kia
    stats: Counter[str] = Counter()
    unmapped: list[dict[str, Any]] = []
    grounded: list[tuple[dict[str, Any], dict[str, Any], str]] = []
    for t, ids, res in results:
        if res.get("error"):
            stats["llm_failed_calls"] += 1
            on_log(f"lỗi trích mục {t['title']!r} ({ids[0]}...): {res['error']}")
        unmapped += [{**u, "topic": t["title"], "chunk_id": ids[0]} for u in res["unmapped"]]
        for it in res["items"]:
            cid = find_grounded_chunk(it, ids, by_id)
            if cid is None:
                stats["evidence_dropped"] += 1
                continue
            grounded.append((t, it, cid))
            slot = schema.slot(it["slot"])
            assert slot is not None  # `extract` chỉ giữ ô có trong schema
            for rel in slot.relations:
                linker.ref(rel.object_type, it["text"], it["en"])
    # Bước 5: LLM chọn nút cho thuật ngữ chưa khớp chính xác
    linker.resolve_pending(llm, on_progress, on_log)
    # Bước 6: dựng triple (gộp trùng)
    triples = merge_triples([tr for t, it, cid in grounded
                             for tr in build_triples(document_id, t, it, cid,
                             by_id[cid].page_start, linker)])
    for t in topics:
        t["links"] = {et: linker.final(et, t["title"], t["en"]).model_dump()
                      for et in TOPIC_ENTITY_TYPES.get(t["kind"], ()) if linker.key(et, t["title"], t["en"]) in linker.refs}
    return triples, topics, unmapped, dict(stats)
```

Mã nguồn 4.6: Mã trích quan hệ từ chunk

Bước 6. Lưu vào đồ thị tri thức. Hệ thống lấy các quan hệ đã duyệt và khớp đủ hai đầu, dựng nút và cạnh (cạnh trùng cặp nút được gộp, nguồn ghi sách và số trang in), thay toàn bộ nút và cạnh của sách trong Neo4j rồi đếm lại; số cạnh lệch thì bước báo lỗi.

```
def run_graph_index(self, run: StageRunState) -> dict[str, Any]:
    if self._graph is None or not self._graph.configured:
        raise StageError("Chưa cấu hình đồ thị tri thức (Neo4j).")
    files, graph, lexicon = run.files, self._graph, self._get_lexicon()
    schema = self._get_schema(run.stage_id)
    # triple đã duyệt (status ok) + trang in của chunk để ghi nguồn
```

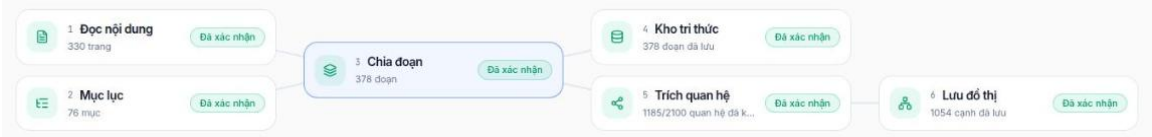
```
    ok = [t for t in (DocumentTriple.model_validate(x) for x in files.read_jsonl("triples.jsonl")) if t.status == "ok"]
    if not ok:
        raise StageError("Chưa có quan hệ nào đã khớp đủ hai đầu để lưu — hãy duyệt bước Trích quan hệ trước.")
    printed = {c["chunk_id"]: c["page_printed_start"] for c in files.read_jsonl("chunks.jsonl")}  # chỉ cần 2 field: không dựng cả DocumentChunk
    run.progress(0, 3, "kiểm tra nút trong đồ thị")
    existing = graph.existing_nodes(step6_graph_index.get_primekg_pairs(ok))
    nodes, edges, skipped, merged = step6_graph_index.build_graph_payload(
        ok, printed, self._repo.meta(run.document_id).get("title", run.document_id), schema, lexicon, existing,
    )
    run.progress(1, 3, "ghi vào Neo4j")
    graph.replace_document(run.document_id, nodes, edges)
    run.progress(2, 3, "đếm lại cạnh đã lưu")
    stored = graph.count_edges(run.document_id)
    if stored != len(edges):
        raise StageError(f"Số quan hệ đã lưu ({stored}) khác số cần lưu ({len(edges)}) — kiểm tra nút DermO/PrimeKG có trong Neo4j "
                         "(thiếu nút thì cạnh không được tạo).")
    summary = step6_graph_index.summarize_graph_payload(nodes, edges, skipped, merged, stored)
    info = {k: summary[k] for k in ("edges", "nodes_added", "by_relation", "new_entities")}
    files.write_json("graph_index.json", {**info, "indexed_at": datetime.now(UTC).isoformat(timespec="seconds")})
    return summary
```

Mã nguồn 4.7: Mã nạp quan hệ vào đồ thị

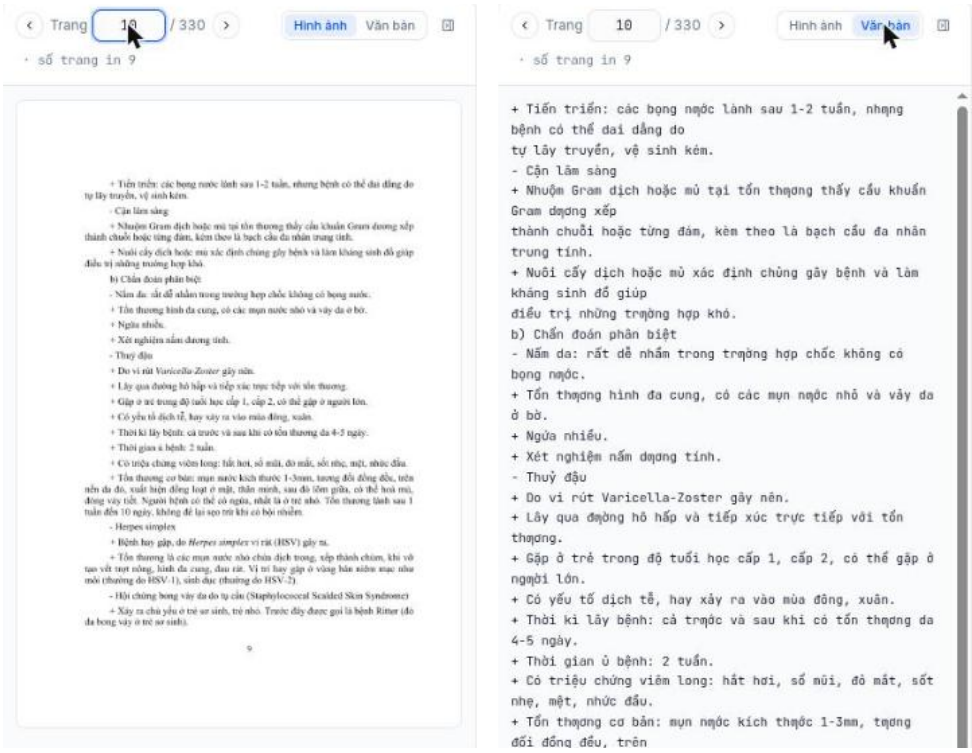
c) Kết quả số hóa giáo trình

Kết quả trực quan.

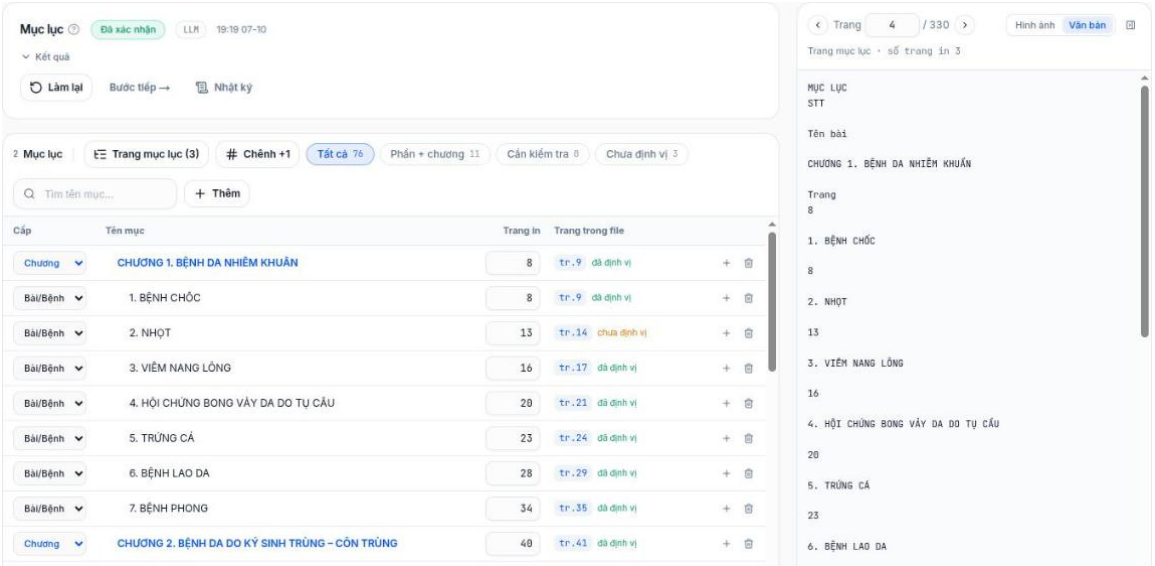
Hình 4.4 là tổng quan quy trình số hóa; các hình tiếp theo là kết quả của từng bước.



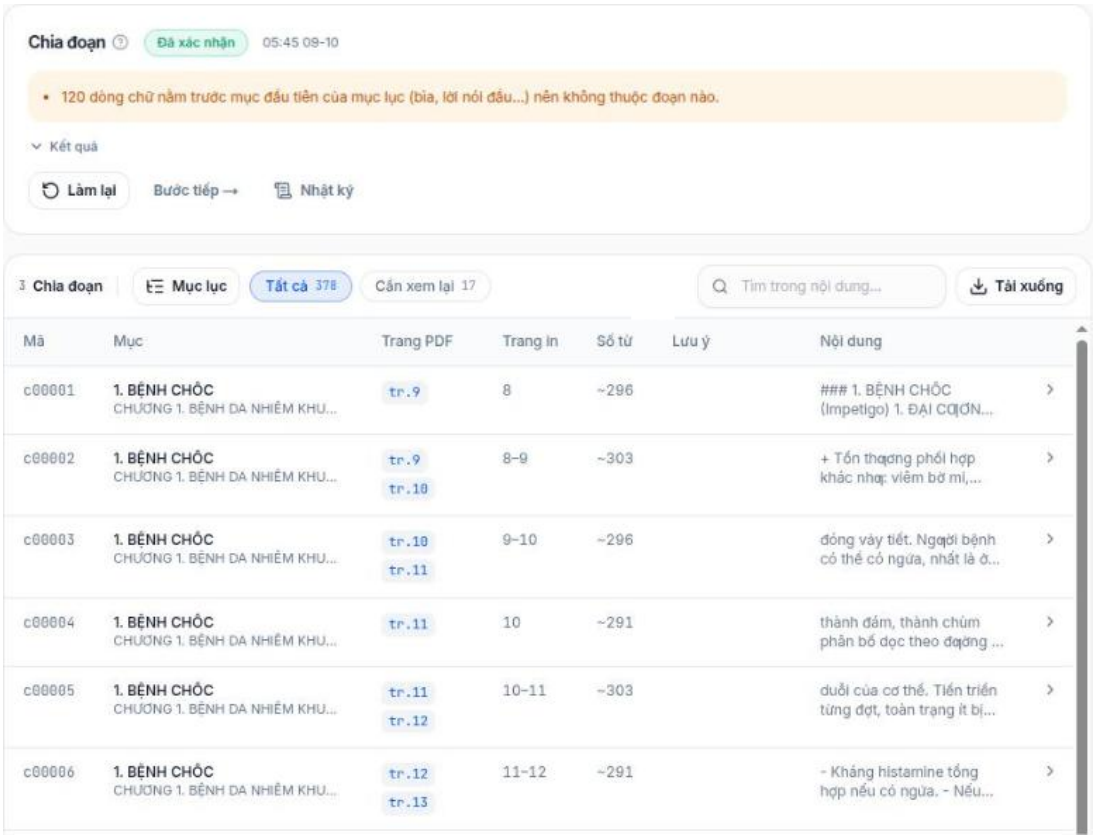
Hình 4.4: Tổng quan quy trình số hóa giáo trình



Hình 4.5: Kết quả đọc nội dung của một trang



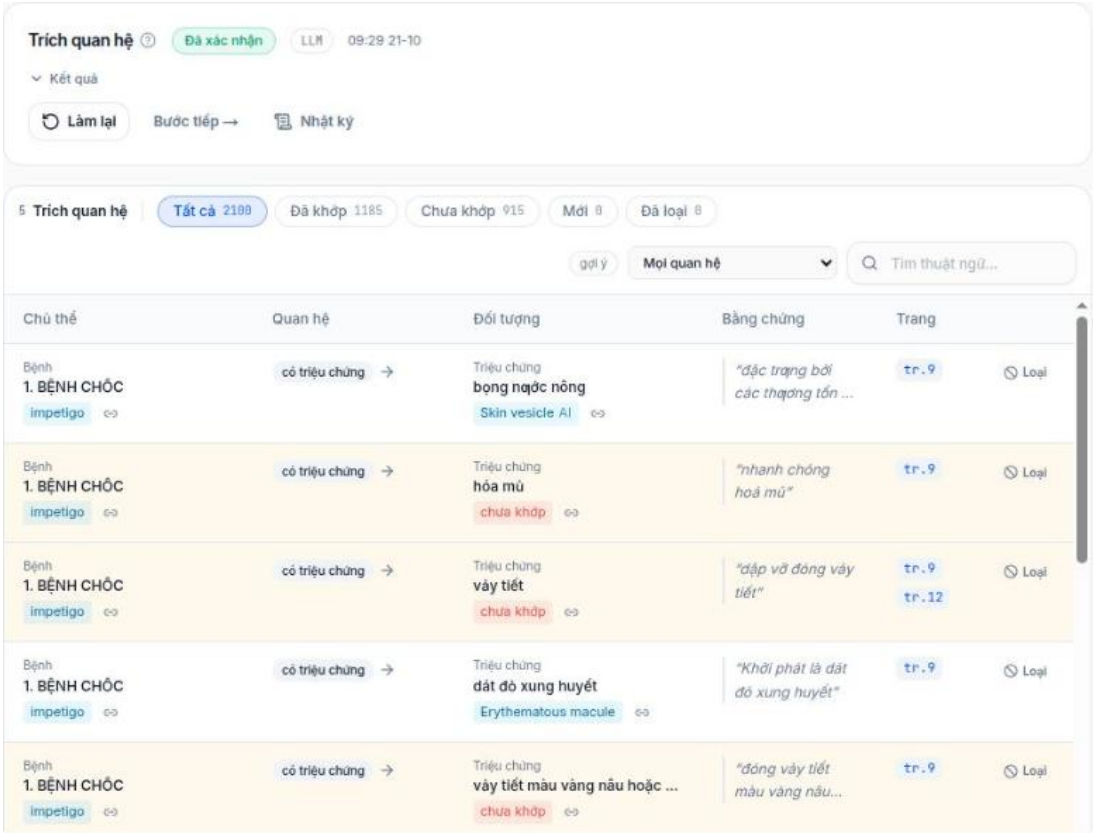
Hình 4.6: Kết quả đọc mục lục của một sách



Hình 4.7: Kết quả chia đoạn: chunk mẫu và danh sách cần soát



Hình 4.8: Kết quả lưu vào kho tri thức: số đoạn đã lưu, số chiều vector, kho lưu trữ và mô hình tạo vector



Hình 4.9: Kết quả trích quan hệ: giao diện duyệt

Thông tin dữ liệu.

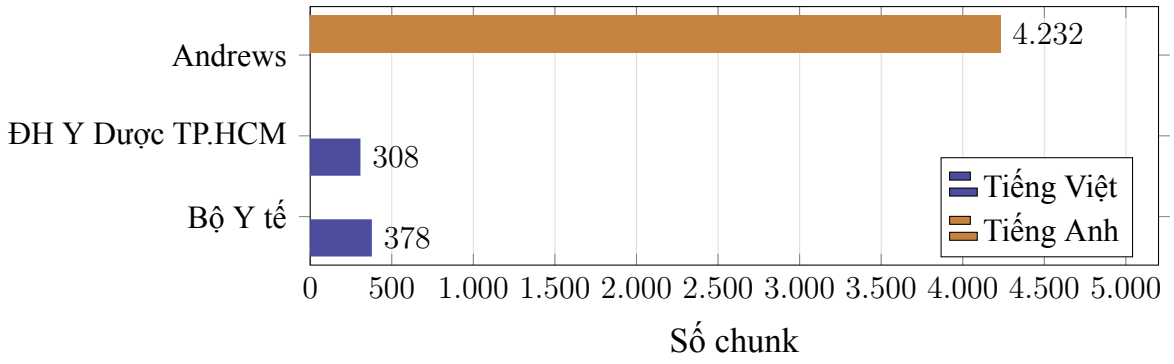
Mỗi chunk là một point trong Qdrant, mang hai vector:

- **Vector dense:** 4096 chiều, từ mô hình `qwen/qwen3-embedding-8b`, dùng cho tìm kiếm ngữ nghĩa.
- **Vector sparse:** tần suất từ theo BM25, dùng cho tìm kiếm từ khóa.

Thông tin đi kèm (payload) của point ghi `document_id`, part, section, mục, số trang, mã các hình trong khoảng trang (`figure_ids`) và đường dẫn file PDF gốc (`source_pdf`).

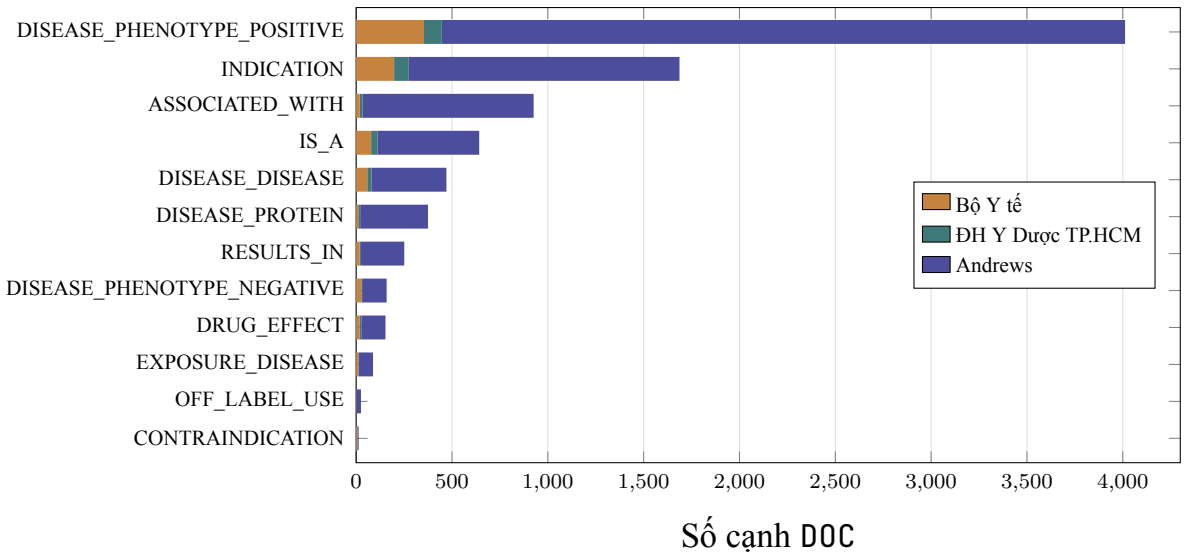
Bảng 4.2: Thống kê ba giáo trình đã nạp vào kho

Sách	Ngôn ngữ	Số trang	Số chunk	Số cạnh DOC
Hướng dẫn chẩn đoán điều trị da liễu (Bộ Y tế)	Việt	330	378	798
Bệnh da liễu thường gặp (ĐH Y Dược TP.HCM)	Việt	249	308	273
Andrews' Diseases of the Skin	Anh	1.083	4.232	7.731



Hình 4.10: Số chunk của ba giáo trình trong Qdrant

Bước 6 đã nạp 8.802 cạnh từ ba sách (hình 4.11). Bước trích quan hệ cho ra 21.754 quan hệ, trong đó 10.267 quan hệ khớp đủ hai đầu vào PrimeKG hoặc DermO. Hệ thống bỏ 887 quan hệ có hai đầu trùng nút và gộp 578 quan hệ trùng cặp nút, còn 8.802 cạnh. Sách Fitzpatrick's Color Atlas mới qua bước chia đoạn (370 chunk), chưa lưu vào kho nên không nằm trong thống kê.



Hình 4.11: Số cạnh trích từ giáo trình theo loại quan hệ và theo sách (tổng 8.802 cạnh, ba sách)

Hai loại quan hệ chiếm nhiều nhất là `DISEASE_PHENOTYPE_POSITIVE` (4.012 cạnh) và `INDICATION` (1.687 cạnh); hai loại này phục vụ các câu hỏi về triệu chứng và điều trị của bệnh.

4.2.3 Ca bệnh đã kết luận của bác sĩ

a) Nội dung và vai trò của ca bệnh

Ca bệnh đã kết luận là báo cáo kết luận do bác sĩ chốt khi đóng ca một phiên tư vấn. Mỗi phiên có một báo cáo và bác sĩ không sửa sau khi chốt. Chatbot dùng kho này để đối chiếu biểu hiện của người bệnh với các ca thực tế tương tự; kết quả chỉ để tham khảo, không phải chẩn đoán y khoa.

b) Quy trình nạp ca bệnh vào Qdrant

Hệ thống nạp ca vào một bộ lưu riêng trong Qdrant ngay sau khi bác sĩ lưu báo cáo kết luận.

Bước 1. Tổng hợp văn bản. Hệ thống đọc bản chụp (snapshot) của báo cáo lúc bác sĩ chốt và ghép thành một văn bản gồm chẩn đoán, thông tin bệnh nhân, dữ kiện lâm

sàng, nhận định, dặn dò và thông tin ảnh, cùng thông tin đi kèm (payload), không chứa thông tin cá nhân (họ tên, số điện thoại).

```python
def build_report_text(
    self,
    report: FinalizeReport,
    profile: PatientProfile | None = None,
    images: list[tuple[Any, Any]] | None = None,
) -> tuple[str, dict[str, Any]]:
    """Tổng hợp thông tin ca bệnh lâm sàng (đã ẩn danh họ tên, sđt) thành văn bản để embedding và payload."""
    snapshot = report.snapshot or {}
    # ... (ghép giới tính, tuổi, tiền sử, dị ứng, dữ kiện lâm sàng, nhận định, dặn dò, nhãn ảnh thành các dòng văn bản)
    text = "\n".join(lines)

    payload: dict[str, Any] = {
        "case_id": report.id,
        "diagnosis": report.diagnosis,
        "notes": report.notes or "",
        "gender": gender,
        "age": age,
        "allergies": allergies,
        "medical_history": medical_history,
        "clinical_facts": facts,
        "reasoning": reasoning,
        "urgency": urgency,
        "instructions": instructions,
        "is_reference_case": bool(report.is_reference_case),
        "text": text,
        "created_at": report.created_at.isoformat() if report.created_at else None,
        "images": image_infos,
    }
    return text, payload
```

Mã nguồn 4.8: Mã tổng hợp văn bản và payload của ca bệnh

Bước 2. Tạo vector và nạp point. Hệ thống tính embedding dense và vector sparse BM25 (chế độ hỗn hợp Việt – Anh) từ cùng văn bản, rồi ghi một point vào Qdrant.

```python
def index_report_sync(
    self,
    report: FinalizeReport,
    profile: PatientProfile | None = None,
    images: list[tuple[Any, Any]] | None = None,
) -> None:
    """Đồng bộ một ca bệnh vào Qdrant (dùng trong script hoặc thread đồng bộ)."""
    self.ensure_collection()
    text, payload = self.build_report_text(report, profile, images=images)
    vectors, _, _ = self._embedding.embed([text])
    dense_vec = vectors[0]
    sparse_vec = encode_document(text, "mixed")

    point = PointStruct(
        id=self.point_id(report.id),
        vector={"": dense_vec, SPARSE_NAME: sparse_vec},
        payload=payload,
    )
    self._qdrant.upsert_points_sync(self._collection, [point])
```

Mã nguồn 4.9: Mã nạp một ca bệnh vào Qdrant

c) Kết quả nạp ca bệnh

Kết quả trực quan.


```
{
  "case_id": "22d444e7-8fcd-4e38-9546-2d250c32e86e",
  "diagnosis": "Viêm da cơ địa vùng mặt",
  "notes": "Tái khám sau 4 tuần.",
  "gender": "male",
  "age": 31,
  "allergies": "Penicillin",
  "medical_history": "Viêm da cơ địa mạn tính; Tiếp xúc xà phòng thơm (hay dùng xà phòng
    thơm)",
  "clinical_facts": [
    {
      "label": "Đỏ, khô, bong vảy mịn",
      "detail": "đỏ, khô, bong vảy mịn và ngứa nhiều nhất là ban đêm, bị 5 ngày",
      "fact_type": "lesion",
      "origin": "patient_reported"
    },
    {
      "label": "Ngứa",
      "detail": "ngứa nhiều nhất là ban đêm, bị 5 ngày",
      "fact_type": "symptom",
      "origin": "patient_reported"
    },
    "…"
  ],
  "reasoning": "Da khô đỏ, ngứa về đêm, tiền sử viêm da cơ địa từ nhỏ.",
  "urgency": "monitor",
  "instructions": [
    "Dưỡng ẩm 2 lần/ngày.",
    "Tránh xà phòng thơm."
  ],
  "is_reference_case": false,
  "text": "Chẩn đoán ca bệnh: Viêm da cơ địa vùng mặt ⏎ Đặc điểm bệnh nhân: Giới tính: male,
    Tuổi: 31, Tiền sử bệnh: Viêm da cơ địa mạn tính; Tiếp xúc xà phòng thơm (hay dùng xà
    phòng thơm), Tiền sử dị ứng: Penicillin ⏎ Dữ kiện lâm sàng & Triệu chứng ghi nhận: ⏎ …",
  "created_at": "2026-11-03T18:55:55.807884+00:00",
  "images": []
}
```


Hình 4.12: Thông tin đi kèm của một ca bệnh trong Qdrant

Thông tin dữ liệu. Mọi báo cáo đã chốt đều được nạp, kể cả ca chưa đánh dấu là ca tham khảo; dấu này hiện chỉ được ghi kèm, chưa dùng để lọc khi tra. Bảng 4.3 liệt kê thông tin đi kèm mỗi point.

Bảng 4.3: Thông tin đi kèm mỗi ca bệnh trong Qdrant

Trường	**Ý nghĩa**
`case_id`	Mã ca; mã point được sinh từ mã này
`diagnosis`	Chẩn đoán do bác sĩ chốt
`is_reference_case`	Dấu ca tham khảo do bác sĩ đặt
`created_at`	Thời điểm tạo báo cáo
`gender`	Giới tính
`age`	Tuổi, tính từ năm sinh
`allergies`	Dị ứng
`medical_history`	Tiền sử bệnh
`clinical_facts`	Dữ kiện lâm sàng ghi nhận trong phiên
`reasoning`	Nhận định của bác sĩ
`urgency`	Mức khẩn cấp
`instructions`	Dặn dò và hướng xử trí
`notes`	Ghi chú bổ sung
`text`	Văn bản dùng để tạo vector
`images`	Ảnh gắn vào ca: nhãn đã xác nhận, vị trí, mô tả, tên và đường dẫn file

Hiện có 14 ca bệnh đã kết luận trong kho.

4.2.4 Đồ thị tri thức có sẵn: PrimeKG và DermO

a) Nội dung và vai trò của đồ thị

Neo4j chứa đồ thị có sẵn gồm hai phần, cùng quan hệ trích từ giáo trình (mục 4.2.2):

- **PrimeKG:** đồ thị y sinh có sẵn. Hệ thống giữ 5 loại nút (bệnh, thuốc, phenotype, gen/protein, phơi nhiễm) và 11 loại quan hệ, khoảng 3,46 triệu cạnh.
- **DermO:** ontology da liễu, 3.427 thuật ngữ, mỗi thuật ngữ có từ đồng nghĩa và thuật ngữ cha. Dùng để chuẩn hóa tên, không dùng để suy ra quan hệ lâm sàng.

PrimeKG và DermO không có cạnh nối nhau; hệ thống nối chúng bằng so khớp tên lúc truy vấn.

```
# edges.csv
relation,display_relation,x_type,x_id,y_type,y_id
disease_phenotype_positive,phenotype present,disease,11292,effect/phenotype,1047
indication,indication,drug,DB09096,disease,11292
indication,indication,drug,DB05219,disease,11292

# nodes.csv
id,name,type,source
11292,"dermatitis, atopic",disease,MONDO
DB09096,Benzoyl peroxide,drug,DrugBank
DB05219,Crisaborole,drug,DrugBank
```

Hình 4.13: Dữ liệu PrimeKG sau khi lọc: bảng cạnh và bảng nút

```
[Term]
id: DERMO:0000122
name: atopic dermatitis
xref: SNOMEDCT_US_2015_03_01:156331003 "atopic dermatitis"
xref: SNOMEDCT_US_2015_03_01:24079001 "atopic dermatitis"
comment: Could put in atopic stigmata, ch 12, table 12.5.
synonym: "AD" EXACT []
synonym: "atopic eczema" EXACT []
synonym: "eczema" EXACT []
xref: DERMLEX:2100 "atopic dermatitis"
xref: DOID:10652 "atopic dermatitis"
xref: DOID:3310 "atopic dermatitis"
xref: HP:0000964 "atopic dermatitis"
xref: HP:0001047 "atopic dermatitis"
xref: ICD10:L20 "atopic dermatitis"
is_a: DERMO:0000121 ! inflammatory dermatoses
is_a: DERMO:0000432 ! causes of secondary erythroderma
is_a: DERMO:0000487 ! eczematous eruptive disease
relationship: results_in DERMO:0001613 ! nail abnormality in dermatologic disease
created_by: Hannah
```

Hình 4.14: Dữ liệu nguồn của DermO: một khối thuật ngữ

b) Quy trình nạp đồ thị vào Neo4j

Hai phần có sẵn được dựng như sau.

PrimeKG, bước 1. Lọc. Nguồn là bảng quan hệ của PrimeKG (Harvard Dataverse, phiên bản 2). Hệ thống chỉ giữ 5 loại nút và 11 loại quan hệ trong phạm vi, bỏ dòng thiếu mã và cạnh trùng, rồi ghi danh sách nút và cạnh trung gian.

```
def main() -> None:
    # ... (mở kg.csv, tạo edges.csv, khai báo bộ đếm); duyệt từng dòng quan hệ:
```

```
    for r in csv.DictReader(f):
        rel, xt, yt, xi, yi = r["relation"], r["x_type"], r["y_type"], r["x_id"], r["y_id"]
        if not (xt and yt and xi and yi):
            skipped["dòng thiếu type/id"] += 1
            continue
        if rel not in RELATIONS or xt not in NODE_LABELS or yt not in NODE_LABELS:
            skipped["ngoài phạm vi"] += 1
            continue
        key = hash((rel, xt, xi, yt, yi))
        if key in seen:
            skipped["cạnh trùng"] += 1
            continue
        seen.add(key)
        nodes.setdefault((xt, xi), (r["x_name"], r["x_source"]))
        nodes.setdefault((yt, yi), (r["y_name"], r["y_source"]))
        w.writerow([rel, r["display_relation"], xt, xi, yt, yi])
        edge_counts[(rel, xt, yt)] += 1
```

Mã nguồn 4.10: Mã lọc quan hệ PrimeKG

PrimeKG, bước 2. Nạp vào Neo4j. Hệ thống nạp cạnh theo lô; quan hệ đã có cạnh được gộp (merge) để nạp lại không nhân đôi.

```
def load_edges(driver: Any) -> None:
    # quan hệ chưa có cạnh nào thì CREATE (nhanh, MERGE trên nút bậc cao của drug_drug rất chậm); đã có thì MERGE để chạy lại không nhân đôi
    fresh = {rel for rel in RELATIONS.values()} - existing_relations(driver)
    buffers: dict[tuple[str, str, str], list[dict[str, str]]] = {}
    totals: dict[str, int] = {}
    with driver.session() as s, (IN_DIR / "edges.csv").open(encoding="utf-8", newline="") as f:

        def flush(key: tuple[str, str, str]) -> None:
            relation, xt, yt = key
            rel = RELATIONS[relation]
            verb = "CREATE" if rel in fresh else "MERGE"
            q = (f"UNWIND $batch AS row MATCH (x:{NODE_LABELS[xt]} {{id: row.x_id}}), "
                 f"(y:{NODE_LABELS[yt]} {{id: row.y_id}}) "
                 f"{verb} (x)-[r:{rel}]→(y) SET r.display_relation = row.display_relation")
            s.run(q, batch=buffers[key]).consume()
            totals[rel] = totals.get(rel, 0) + len(buffers[key])
            buffers[key] = []

        for i, r in enumerate(csv.DictReader(f), 1):
            key = (r["relation"], r["x_type"], r["y_type"])
            buffers.setdefault(key, []).append({"x_id": r["x_id"], "y_id": r["y_id"], "display_relation": r["display_relation"]})
            if len(buffers[key]) >= BATCH_SIZE:
                flush(key)
            if i % 500_000 == 0:
                print(f"  ... {i} dòng cạnh", flush=True)
        for key in list(buffers):
            if buffers[key]:
                flush(key)
    for rel, n in sorted(totals.items()):
        print(f"  [cạnh] {rel}: {n} ({'mới' if rel in fresh else 'upsert'})")
```

Mã nguồn 4.11: Mã nạp cạnh PrimeKG

DermO, bước 1. Đọc tệp ontology. Tệp DermO (định dạng OBO, phiên bản

1.19) gồm các khối thuật ngữ; hệ thống đọc mỗi khối thành một thuật ngữ có tên, định nghĩa, từ đồng nghĩa, thuật ngữ cha và các quan hệ.

```python
def parse(lines: list[str]) -> tuple[dict[str, str], list[dict[str, Any]]]:
    header: dict[str, str] = {}
    terms: list[dict[str, Any]] = []
    cur: dict[str, Any] | None = None
    in_term = False
    for raw in lines:
        line = raw.rstrip("\n")
        if line.startswith("["):
            in_term = line == "[Term]"
            cur = new_term() if in_term else None
            if cur is not None:
                terms.append(cur)
            continue
        if ":" not in line:
            continue
        tag, _, value = line.partition(":")
        value = value.strip()
        if cur is None:
            if not in_term and not terms:
                header[tag] = value
            continue
        if tag == "id":
            cur["id"] = value
        elif tag == "name":
            cur["name"] = value
        elif tag == "def" and (m := DEF_RE.match(value)):
            cur["def"] = m.group(1)
        elif tag == "synonym" and (m := SYN_RE.match(value)):
            text = m.group(1).removeprefix(NEW_SYN_TAG).strip()
            if text:
                cur["synonyms"].append({"text": text, "scope": m.group(2)})
        elif tag == "xref":
            cur["xrefs"].append(value.split()[0])
        elif tag == "alt_id":
            cur["alt_ids"].append(value)
        elif tag == "is_a":
            cur["is_a"].append(value.split()[0])
        elif tag == "relationship":
            rel, target = value.split()[:2]
            cur["relationships"].append({"type": rel, "target": target})
        elif tag == "is_obsolete":
            cur["is_obsolete"] = value == "true"
    return header, terms
```

Mã nguồn 4.12: Mã đọc tệp ontology DermO

DermO, bước 2. Nạp vào Neo4j. Mỗi thuật ngữ thành một nút `DermoTerm`. Quan hệ "là một" (`is_a`) thành cạnh `IS_A`; các quan hệ khác giữ tên gốc viết hoa (ví dụ `HAS_SYMPTOM`, `RESULTS_IN`).

```python
def load_edges(driver: Any, terms: list[dict[str, Any]]) -> None:
    # (loại cạnh OBO, từ, đến); `is_a` thành IS_A, còn lại giữ tên quan hệ gốc (viết hoa)
    ↪  và chiều gốc
    by_type: dict[str, list[dict[str, str]]] = {"IS_A": []}
```

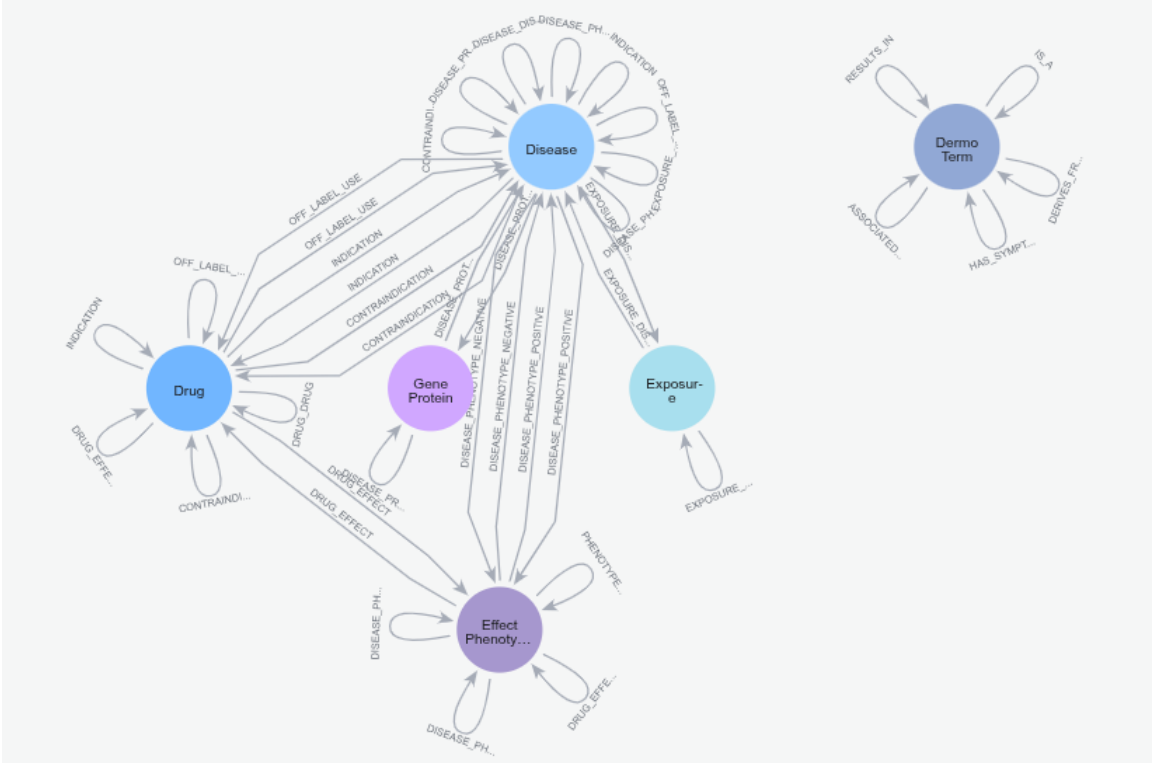
```
    for t in terms:
        by_type["IS_A"] += [{"x": t["id"], "y": p} for p in t["is_a"]]
        for r in t["relationships"]:
            if not REL_RE.match(r["type"]):
                raise SystemExit(f"loại quan hệ lạ trong OBO: {r['type']!r}")  # type ghép
                ↪ thẳng vào Cypher nên phải chặn
            by_type.setdefault(r["type"].upper(), []).append({"x": t["id"], "y":
            ↪ r["target"]})
    with driver.session() as s:
        for rel, rows in by_type.items():
            q = f"UNWIND $batch AS row MATCH (x:DermoTerm {{id: row.x}}), (y:DermoTerm
            ↪ {{id: row.y}}) MERGE (x)-[:{rel}]→(y)"
            for batch in batched(rows):
                s.run(q, batch=batch).consume()
            print(f"  [cạnh] {rel}: {len(rows)}")
```

Mã nguồn 4.13: Mã nạp cạnh DermO

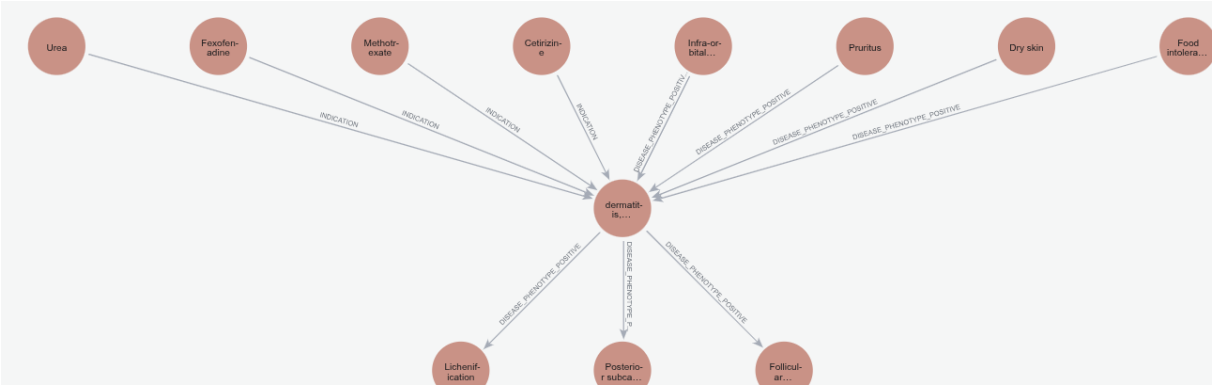
c) Kết quả nạp đồ thị

Kết quả trực quan.

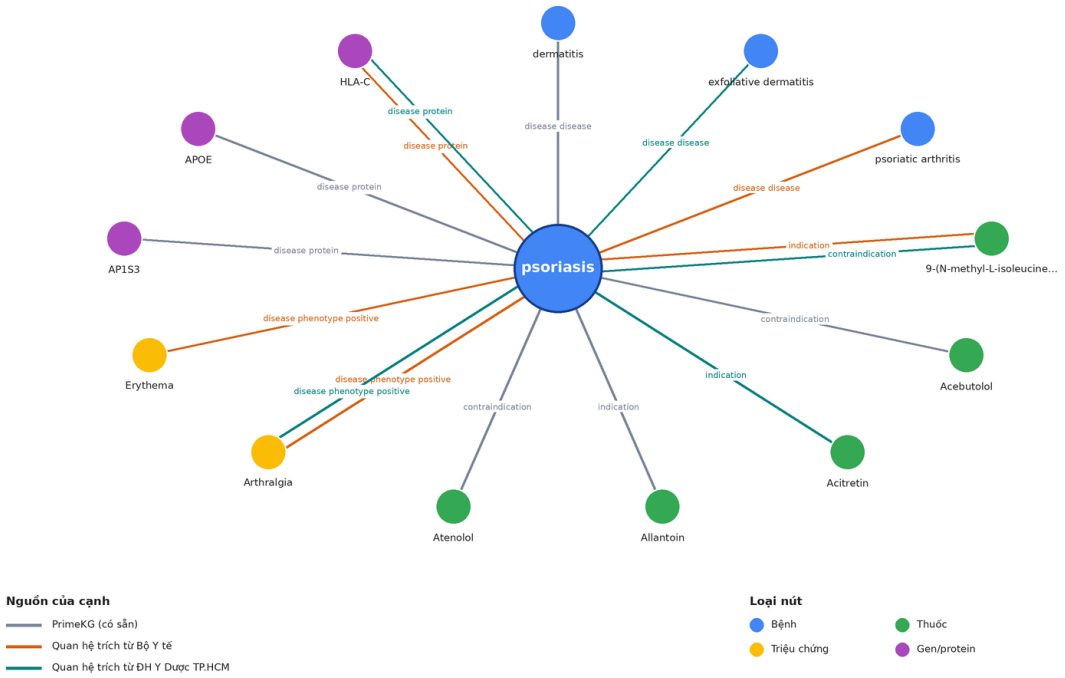
Hình 4.15 là lược đồ (schema) của đồ thị tri thức, lấy từ Neo4j Browser bằng `db.schema.visualization()`, gồm năm loại nút của PrimeKG và nút `DermoTerm` của DermO. Hình đã lọc bỏ các nhãn của đồ thị hành trình bệnh nhân. Mỗi vòng tự nối là một loại quan hệ giữa hai nút cùng loại; các quan hệ nối `DermoTerm` (`IS_A`, `HAS_SYMPTOM`, …) gồm cả cạnh trích từ giáo trình.



Hình 4.15: Lược đồ đồ thị tri thức trong Neo4j Browser (sáu loại nút, không gồm đồ thị hành trình bệnh nhân)



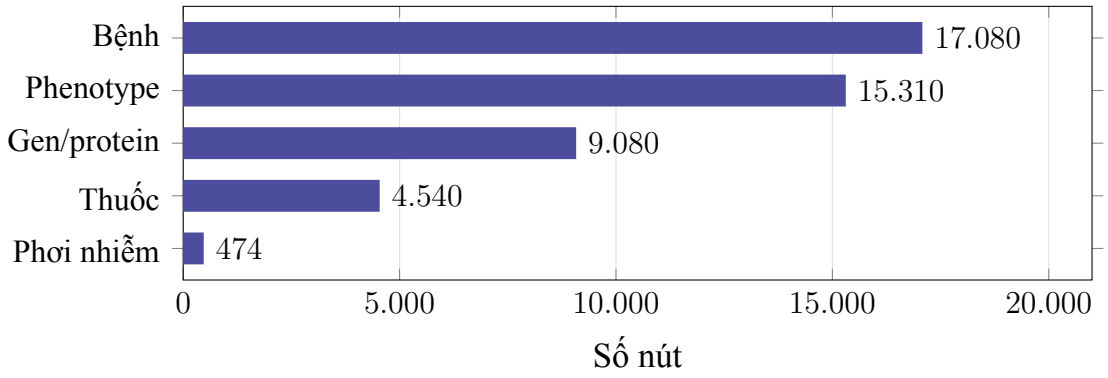
Hình 4.16: Đồ thị quanh bệnh *dermatitis, atopic* trong Neo4j Browser: thuốc chỉ định (cạnh `INDICATION`) và triệu chứng, dấu hiệu lâm sàng của bệnh (cạnh `DISEASE_PHENOTYPE_POSITIVE`)



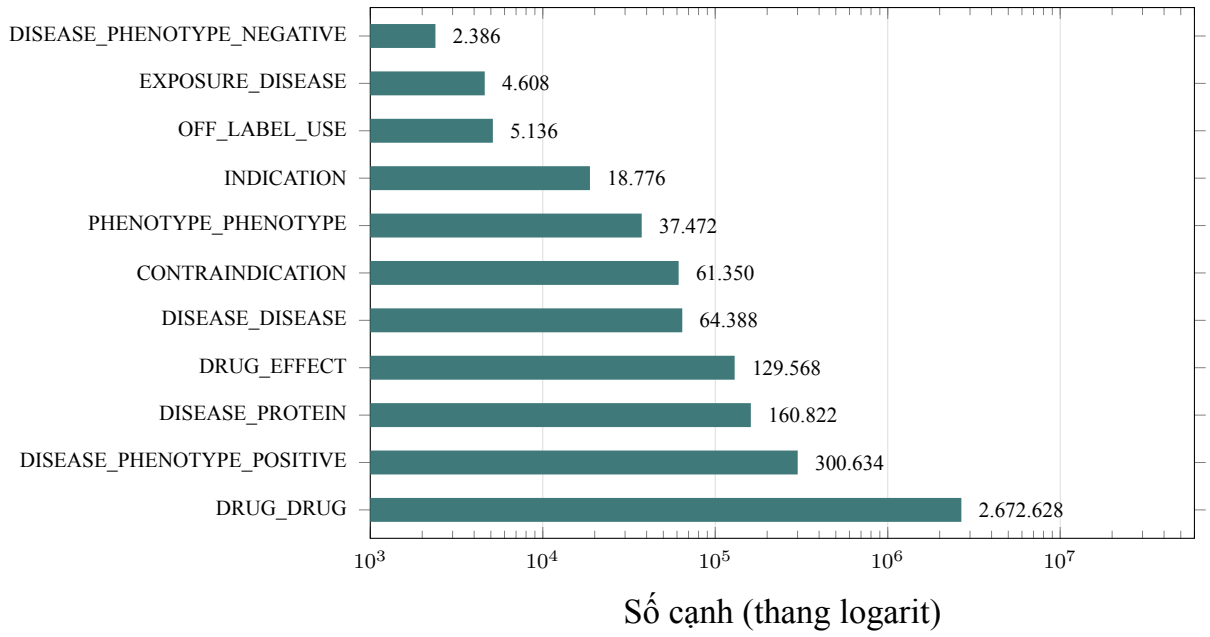
Hình 4.17: Cạnh PrimeKG và cạnh trích từ hai giáo trình quanh bệnh psoriasis

Thông tin dữ liệu.

Hình 4.18 và 4.19 thống kê PrimeKG đã nạp. Loại cạnh `DRUG_DRUG` chiếm khoảng 77% tổng số cạnh nên trục số cạnh dùng thang logarit. Các truy vấn của chatbot không duyệt loại cạnh này.



Hình 4.18: Số nút của PrimeKG theo loại (bệnh gồm cả nhóm `MONDO_grouped`)



Hình 4.19: Số cạnh của PrimeKG theo loại quan hệ

4.3 Các luồng truy xuất tri thức của chatbot

4.3.1 Truy xuất trong luồng RAG của chatbot

Truy xuất tăng cường sinh văn bản (RAG) gồm hai phần: tìm bằng chứng trong kho tri thức, rồi để mô hình ngôn ngữ sinh câu trả lời dựa trên bằng chứng đó. Chatbot thực hiện bốn bước:

1. **Nạp tri thức:** giáo trình được cắt thành đoạn và nạp vào Qdrant; quan hệ trích từ sách nạp vào Neo4j; ca bệnh nạp vào kho riêng (mục 4.2).
2. **Truy xuất:** agent gọi công cụ tra cứu, công cụ chạy ba hướng tìm kiếm ngữ nghĩa, từ khóa, đồ thị và gộp kết quả bằng RRF (các mục từ 4.3.3 đến 4.3.6).
3. **Đưa bằng chứng vào ngữ cảnh:** mỗi kết quả nhận một mã `E#` kèm sách, mục và số trang (Chương 5).
4. **Sinh câu trả lời:** agent chỉ giữ giả thuyết có ít nhất một mã `E#` hợp lệ; câu trả lời là gợi ý hỗ trợ, không phải chẩn đoán y khoa (Chương 5).

Chương này mô tả hai bước đầu. Hai bước sau thuộc luồng lập luận của agent ở

Chương 5.

4.3.2 Các luồng truy xuất trên văn bản giáo trình

Trên các đoạn giáo trình (chunk), hệ thống có ba hướng tìm kiếm. Hai hướng đầu, ngữ nghĩa và từ khóa, bổ sung cho nhau. Hướng thứ ba đi qua đồ thị: từ thực thể trong câu hỏi tới quan hệ đã trích từ sách, rồi tới các đoạn chứa câu trích của quan hệ đó (bảng 4.4). Ca bệnh đã kết luận dùng hai hướng đầu trên kho riêng; hướng đồ thị chỉ áp dụng cho giáo trình.

Bảng 4.4: Ba hướng tìm kiếm bổ sung cho nhau

Tiêu chí	**Ngữ nghĩa (dense)**	**Từ khóa (BM25, sparse)**	**Đồ thị (liên kết tới đoạn)**
So khớp	Cosine giữa vector câu hỏi và vector đoạn	Từ trùng, có trọng số theo độ hiếm	Tên thực thể trùng tên hoặc từ đồng nghĩa của nút; lấy đoạn gắn với quan hệ của nút
Bắt được	Diễn đạt khác nghĩa cùng; câu hỏi tiếng Việt, sách tiếng Anh	Tên thuốc, hoạt chất, mã bệnh, liều	Đoạn mà sách đã trích thành quan hệ với thực thể được hỏi, kể cả khi đoạn dùng từ khác
Bỏ sót	Token hiếm bị làm mờ trong vector	Câu hỏi và đoạn không có từ chung	Thực thể chưa có quan hệ trích từ sách; không đi qua nút lân cận

Ví dụ: câu hỏi "da tôi đỏ và ngứa, nổi thành mảng" và đoạn chứa "erythematous pruritic plaques" không có từ chung. BM25 cho điểm 0, tìm kiếm ngữ nghĩa vẫn khớp. Ngược lại, câu hỏi về một tên thuốc cụ thể được BM25 khớp đúng chữ, trong khi vector có thể gộp thuốc đó với các thuốc cùng nhóm.

Mã nguồn đặc trưng của từng luồng được trích ở các mã nguồn 4.14, 4.15, 4.16, 4.17, 4.18 và 4.20. Phần sau mô tả từng luồng đơn (mục 4.3.3, 4.3.4 và 4.3.5), rồi luồng hỗn hợp ghép ba luồng đó (mục 4.3.6).

4.3.3 Luồng tìm kiếm ngữ nghĩa

a) Nguyên lý tìm kiếm ngữ nghĩa

Tìm kiếm ngữ nghĩa so khớp theo nghĩa của câu thay vì theo ký tự. Mô hình embedding đổi mỗi đoạn văn bản thành một vector thực d chiều. Bài toán tra cứu trở thành bài toán hình học: tìm các vector đoạn gần vector câu hỏi nhất.

Hệ thống dùng mô hình `qwen/qwen3-embedding-8b` (8 tỷ tham số), đầu ra $d = 4096$ chiều. Mô hình có hai đặc tính:

- **Instruction-tuned:** câu hỏi được thêm tiền tố chỉ dẫn; đoạn trong kho tri thức được mã hóa ở dạng nguyên bản.
- **Đa ngôn ngữ:** câu hỏi tiếng Việt và đoạn tiếng Anh nằm chung một không gian vector.

b) Cơ chế tìm kiếm ngữ nghĩa

Gọi f là hàm embedding. Quy trình có hai giai đoạn:

- **Nạp dữ liệu (indexing):** mỗi đoạn được mã hóa một lần, $d = f(\text{đoạn})$, rồi lưu vào Qdrant.
- **Truy vấn (querying):** câu hỏi được mã hóa một lần, $q = f(\text{câu hỏi})$. Kiến trúc Bi-Encoder mã hóa câu hỏi và đoạn độc lập, nên vector đoạn được tính trước và độ trễ lúc truy vấn thấp.

Độ liên quan giữa q và d là độ tương đồng cosine:

$$\text{sim}(q, d) = \frac{q \cdot d}{\|q\|\|d\|} \tag{4.2}$$

Trong đó, $\text{sim}(q, d) \in [-1, 1]$. Phép đo chỉ phụ thuộc hướng vector nên không bị lệch bởi độ dài đoạn.

Hệ thống xếp các đoạn theo sim giảm dần, lấy k đoạn đầu, đọc nội dung và siêu dữ liệu (sách, mục, trang) theo ID vector rồi trả về.

```python
_QUERY_INSTRUCTION = "Given a Vietnamese or English dermatology question, retrieve relevant textbook passages that answer it"


def embed_query(text: str) -> list[float]:
    # câu hỏi thêm tiền tố chỉ dẫn; đoạn trong kho được mã hóa ở dạng nguyên bản
    return _embed([f"Instruct: {_QUERY_INSTRUCTION}\nQuery: {text}"])[0]


# QdrantVectorClient.search(): truy vấn vector gần nhất theo cosine
async def search(self, name: str, vector: list[float], *, limit: int, query_filter: Filter | None = None,
                 score_threshold: float | None = None) -> list[ScoredPoint]:
    result = await self.client.query_points(name, query=vector, query_filter=query_filter, limit=limit,
                                            score_threshold=score_threshold)
    return result.points
```

Mã nguồn 4.14: Mã tìm kiếm ngữ nghĩa: mã hóa câu hỏi (có tiền tố chỉ dẫn) và truy vấn Qdrant

4.3.4 Luồng tìm kiếm từ khóa

a) Nguyên lý tìm kiếm từ khóa

Tìm kiếm từ khóa khớp chính xác chuỗi ký tự. Luồng này bù cho điểm yếu của vector: tên thuốc, hoạt chất, mã bệnh và liều lượng thường bị làm mờ trong embedding.

TF-IDF là nền tảng của phương pháp. Điểm của tài liệu D với câu hỏi Q là tổng, trên các từ $t \in Q$, của tích tần suất từ trong tài liệu (tf) và độ hiếm của từ trong kho (idf):

$$\text{score}_{\text{TF-IDF}}(D, Q) = \sum_{t \in Q} \text{tf}(t, D) \cdot \text{idf}(t) \tag{4.3}$$

Hai thành phần được tính như sau:

$$\text{tf}(t, D) = f(t, D) \tag{4.4}$$

$$\text{idf}(t) = \ln \frac{N}{n(t)} \tag{4.5}$$

Trong đó $f(t, D)$ là số lần từ t xuất hiện trong đoạn D, N là tổng số đoạn trong kho và $n(t)$ là số đoạn chứa t. Từ xuất hiện trong ít đoạn có idf lớn nên đóng góp nhiều vào điểm. Từ có mặt ở mọi đoạn ($n(t) = N$) có idf $= 0$ và không ảnh hưởng đến điểm.

TF-IDF có hai hạn chế:

- **Không bão hòa:** điểm tăng tuyến tính theo tần suất từ, dù từ lặp lại nhiều lần không làm đoạn liên quan hơn tương ứng.
- **Không chuẩn hóa độ dài:** đoạn dài chứa nhiều từ hơn nên được điểm cao hơn.

b) Thuật toán xếp hạng BM25

BM25 (Best Matching 25) thêm hai tham số để sửa hai hạn chế trên:

$$\text{score}(D,Q) = \sum_{t \in Q} \text{IDF}(t) \cdot \frac{\text{tf} \cdot (k_1 + 1)}{\text{tf} + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)} \tag{4.6}$$

$$\text{IDF}(t) = \ln\left(1 + \frac{N - n(t) + 0{,}5}{n(t) + 0{,}5}\right) \tag{4.7}$$

Trong đó:

- $\text{tf}(t, D)$: số lần từ t xuất hiện trong D; $n(t)$: số đoạn chứa t; N: tổng số đoạn.
- $k_1 = 1{,}2$: giới hạn mức tăng điểm theo tần suất từ (độ bão hòa).
- $b = 0{,}75$: mức chuẩn hóa theo tỷ số giữa độ dài đoạn $|D|$ và độ dài trung bình avgdl.

Qdrant lưu phần tf của mỗi đoạn dưới dạng vector thưa (sparse) và chấm điểm khi truy vấn. Cấu trúc này hoạt động như chỉ mục đảo: mỗi từ trỏ tới các đoạn chứa nó, nên truy vấn không phải duyệt toàn bộ kho.

c) Cơ chế tìm kiếm từ khóa

- **Mở rộng câu hỏi Việt – Anh:** LLM trích thực thể y khoa từ câu hỏi và thêm tên tiếng Anh, tiếng Việt vào câu hỏi BM25, để khớp với sách viết khác ngôn ngữ câu hỏi. Khi LLM lỗi, BM25 dùng câu hỏi gốc và hệ thống ghi chú lại.
- **Tiền xử lý tiếng Việt:** tách từ ghép bằng underthesea, loại stopword nhưng giữ từ phủ định và đơn vị liều. Thêm token bỏ dấu với trọng số thấp ($0{,}3$) để câu hỏi không dấu vẫn khớp.
- **Cách dùng:** chạy độc lập khi cần tra tên chính xác, và là một nhánh của luồng hỗn hợp.

```python
_K1 = 1.2
_B = 0.75
_AVG_DOC_LEN = 256  # độ dài trung bình giả định của một chunk (số từ)
_FOLD_WEIGHT = 0.3  # trọng số token bỏ dấu



def _token_id(token: str) -> int:
    # Qdrant chỉ nhận chỉ số uint32; crc32 ổn định giữa các tiến trình (hash() của Python thì không). Va chạm hiếm với
    # vài chục nghìn từ vựng và chỉ làm hai từ chia sẻ điểm, chấp nhận được.
    return zlib.crc32(token.encode("utf-8"))


def encode_document(text: str, language: str = "mixed") -> SparseVector:
    """Phần TF của BM25 cho một chunk: tf * (k1 + 1) / (tf + k1 * (1 - b + b * dl / avgdl))."""
    tokens, folds = tokenize_with_folds(text, language)
    if not tokens:
        return SparseVector(indices=[], values=[])
    norm = _K1 * (1 - _B + _B * len(tokens) / _AVG_DOC_LEN)  # độ dài tính theo token chính, token bỏ dấu không làm đoạn "dài" hơn
    weights: dict[int, float] = {}
    for token, tf in Counter([*tokens, *folds]).items():
        # cộng dồn khi hai từ va chạm cùng chỉ số, vì Qdrant yêu cầu chỉ số không trùng
        idx = _token_id(token)
        weights[idx] = weights.get(idx, 0.0) + tf * (_K1 + 1) / (tf + norm)
    return SparseVector(indices=list(weights), values=list(weights.values()))


def encode_query(text: str, language: str = "mixed") -> SparseVector:
    """Từ chính trọng số 1, từ bỏ dấu `_FOLD_WEIGHT`; Qdrant nhân IDF của từ đó khi tính điểm."""
    tokens, folds = tokenize_with_folds(text, language, all_tokens=True)
    weights: dict[int, float] = {}
    for token, weight in [*((t, 1.0) for t in tokens), *((t, _FOLD_WEIGHT) for t in folds)]:
        idx = _token_id(token)
        weights[idx] = max(weights.get(idx, 0.0), weight)
    return SparseVector(indices=list(weights), values=list(weights.values()))
```

Mã nguồn 4.15: Mã chuyển văn bản thành vector thưa BM25: `encode_document` lưu vào Qdrant, `encode_query` dùng khi truy vấn

4.3.5 Luồng tìm kiếm đồ thị

a) Nguyên lý tìm kiếm đồ thị

Hai luồng trước so khớp câu hỏi với nội dung đoạn. Luồng đồ thị so khớp theo thực thể y khoa và đi ngược từ thực thể về đoạn. Quan hệ trích từ sách (mục 4.2.2) lưu mã các đoạn chứa câu trích, nên từ một thực thể có thể lấy lại các đoạn đó, kể cả khi đoạn dùng từ khác với câu hỏi.

Điểm của đoạn c là số quan hệ trỏ tới nó:

$$\text{score}_G(c) = \left|\{\, r \in R(E) \;:\; c \in \text{chunks}(r) \,\}\right| \tag{4.8}$$

Trong đó E là tập nút đồ thị khớp với thực thể trong câu hỏi, $R(E)$ là tập quan hệ trích từ sách nối trực tiếp với một nút thuộc E, và $\text{chunks}(r)$ là tập đoạn chứa câu trích của quan hệ r. Đoạn cùng điểm được xếp theo mã đoạn.

Luồng này trả về đoạn giáo trình. Việc truy vấn đồ thị để chọn bệnh ứng viên là chức năng khác, trình bày ở mục 4.3.7.

b) Cơ chế tìm kiếm đồ thị

Luồng chỉ chạy với kho giáo trình; với kho ca bệnh, luồng trả danh sách rỗng.

1. **Lấy tên thực thể:** tên tiếng Anh và tiếng Việt của các thực thể y khoa trong câu hỏi, bỏ loại khác.
2. **Tìm nút:** tối đa 10 nút của đồ thị có tên hoặc từ đồng nghĩa khớp, không phân biệt hoa thường. Hệ thống khớp chính xác trước, chỉ khi không có kết quả mới khớp theo chuỗi con.
3. **Lấy đoạn từ quan hệ:** các quan hệ trích từ sách nối trực tiếp với những nút đó (một bước, không phân biệt hướng); mỗi quan hệ cho các đoạn chứa câu trích của nó.
4. **Xếp hạng và lấy đoạn:** đoạn có nhiều quan hệ trỏ tới xếp trước, rồi lấy nội dung đoạn từ Qdrant.

```python
async def graph_search(
    entities: list[ExtractedEntity],
    limit: int,
    corpus: RetrievalCorpus = DOCUMENT_CORPUS,
) -> list[dict[str, Any]]:
    """Payload chunk theo số cạnh trỏ tới giảm dần; rỗng khi không phải corpus sách, không có thực thể y khoa hoặc chưa có cạnh DOC."""
    if corpus.name != DOCUMENT_CORPUS.name:
        return []
    names = [
        t for e in entities if e.type != "other" for t in (e.text, e.vi) if t
    ]
```

```
    chunk_ids = await get_document_graph_service().chunks_for_entities(
        names, limit
    )
    return await get_retrieval_service().fetch(corpus, chunk_ids)

async def chunks_for_entities(self, names: list[str], limit: int = 20) -> list[str]:
    """`chunk_id` mà cạnh DOC của các thực thể trỏ tới, nhiều cạnh nhất trước. Khớp tên chính xác trước, không có mới khớp chứa."""
    names = [n.strip().lower() for n in dict.fromkeys(names) if n and n.strip()]
    if not self.configured or not names:
        return []
    query = """
        MATCH (n) WHERE (n:Entity OR n:DermoTerm) AND {match}
        WITH n LIMIT 10
        MATCH (n)-[r]-()
        WHERE r.source = 'DOC'
        UNWIND coalesce(r.chunk_ids, []) AS chunk_id
        RETURN chunk_id, count(DISTINCT r) AS weight
        ORDER BY weight DESC, chunk_id
        LIMIT $lim
    """
    exact = "(toLower(coalesce(n.name,'')) IN $names OR any(s IN coalesce(n.synonyms, []) WHERE toLower(s) IN $names))"
    contains = "any(q IN $names WHERE toLower(coalesce(n.name,'')) CONTAINS q OR any(s IN coalesce(n.synonyms, []) WHERE toLower(s) CONTAINS q))"
    for match in (exact, contains):
        rows = await self.neo4j.run(_cypher(query.replace("{match}", match)), names=names, lim=limit)
        if rows:
            return [str(r["chunk_id"]) for r in rows]
    return []
```

Mã nguồn 4.16: Mã luồng đồ thị: từ thực thể tới đoạn giáo trình

Đoạn do luồng này đề xuất được gắn dấu để chatbot biết nguồn là đồ thị. Phạm vi của luồng hẹp: chỉ xét quan hệ nối trực tiếp với thực thể và chỉ dùng quan hệ trích từ sách, vì quan hệ của hai đồ thị có sẵn không gắn với đoạn nào. Luồng trả rỗng khi sách chưa trích quan hệ hoặc khi đoạn đã chia lại mà chưa trích lại; lúc đó luồng hỗn hợp chạy với hai nhánh còn lại.

4.3.6 Luồng hỗn hợp (`hybrid_retrieval`)

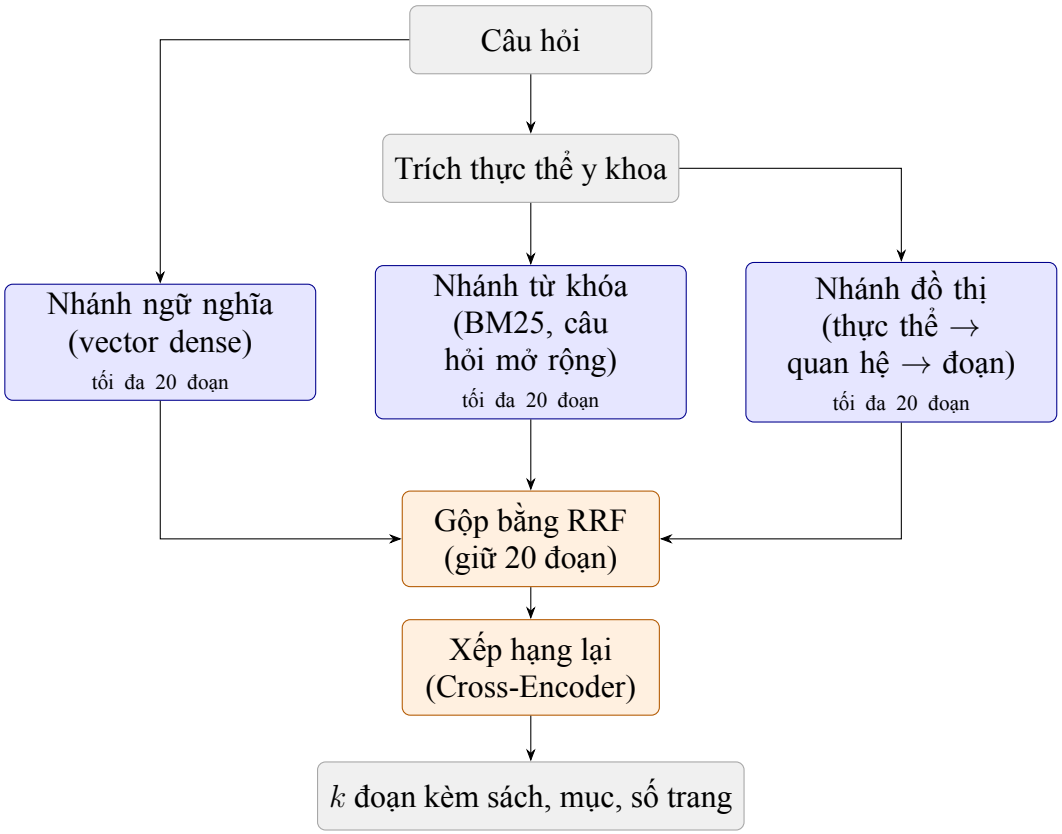
a) Quy trình truy xuất hỗn hợp

Luồng hỗn hợp ghép ba luồng ở trên theo bốn bước (hình 4.20). Mục tiêu của hai bước đầu là không bỏ sót đoạn liên quan (recall). Mục tiêu của hai bước sau là đưa đoạn đúng lên đầu (precision).

1. **Chuẩn bị câu hỏi:** trích thực thể y khoa; thực thể dùng để mở rộng câu hỏi cho nhánh BM25 và làm đầu vào cho nhánh đồ thị.

2. **Ba nhánh song song:** nhánh ngữ nghĩa, nhánh BM25 và nhánh đồ thị, mỗi nhánh lấy tối đa 20 đoạn.
3. **Gộp bằng RRF,** khử trùng theo ID đoạn, giữ 20 đoạn đầu.
4. **Xếp hạng lại** bằng Cross-Encoder, trả về k đoạn đầu kèm sách, mục, số trang.

Khi một nhánh lỗi, công cụ dùng kết quả của các nhánh còn lại. Khi reranker không dùng được, công cụ giữ thứ tự sau RRF. Cả hai trường hợp đều được ghi chú trong kết quả trả về để chatbot báo rõ nguồn nào chưa tra được.



Hình 4.20: Luồng truy xuất hỗn hợp

b) Gộp kết quả bằng RRF

RRF (Reciprocal Rank Fusion) gộp theo thứ hạng thay vì điểm thô, vì điểm cosine và điểm BM25 có thang khác nhau, còn nhánh đồ thị chỉ có thứ hạng theo số quan hệ:

$$\mathrm{RRF}(d) = \sum_{i \in \text{lists}} \frac{1}{k + \mathrm{rank}_i(d)}, \quad k = 60 \tag{4.9}$$

Trong đó $\text{rank}_i(d)$ là thứ hạng của đoạn d trong danh sách i. Đoạn xuất hiện ở nhiều danh sách được cộng nhiều số hạng nên lên cao. Hằng số $k = 60$ làm giảm chênh lệch giữa các vị trí đầu, nên không nhánh nào chi phối hoàn toàn. Hệ thống không dùng chuẩn hóa điểm có trọng số α vì phương pháp này nhạy với giá trị ngoại lai trong từng truy vấn và cần chỉnh α bằng thực nghiệm, trong khi chưa có tập đánh giá đủ lớn.

```python
_RRF_K = 60  # hằng số chuẩn của RRF: giảm ảnh hưởng chênh lệch thứ hạng ở đầu danh sách


def rrf(*rankings: list[str]) -> dict[str, float]:
    """Điểm RRF theo id"""
    scores: dict[str, float] = {}
    for ranking in rankings:
        for rank, key in enumerate(ranking, start=1):
            scores[key] = scores.get(key, 0.0) + 1.0 / (_RRF_K + rank)
    return scores

def merge_by_id(
    semantic: list[dict[str, Any]], bm25: list[dict[str, Any]], graph: list[dict[str, Any]], limit: int, id_key: str = "chunk_id"
) -> list[dict[str, Any]]:
    """Payload của ba nhánh, khử trùng theo `id_key`, xếp theo điểm RRF giảm dần, lấy `limit` đầu."""
    legs = [semantic, bm25, graph]
    payloads: dict[str, dict[str, Any]] = {}
    for payload in (p for leg in legs for p in leg):
        payloads.setdefault(str(payload.get(id_key)), payload)
    scores = rrf(*([str(p.get(id_key)) for p in leg] for leg in legs))
    return [payloads[k] for k in sorted(scores, key=lambda k: scores[k], reverse=True)[:limit]]
```

Mã nguồn 4.17: Mã gộp kết quả ba nhánh bằng RRF

c) *Xếp hạng lại bằng Cross-Encoder*

Mô hình `qwen/qwen3-reranker-8b` nhận từng cặp (câu hỏi, đoạn) và cho điểm từ 0 đến 1. Khác Bi-Encoder, Cross-Encoder đọc câu hỏi và đoạn cùng lúc nên đánh giá được tương tác giữa hai chuỗi, nhưng chậm hơn. Vì vậy mô hình chỉ chạy trên 20 đoạn sau RRF.

```python
async def run_hybrid(
    query: str,
    *,
    corpus: RetrievalCorpus = DOCUMENT_CORPUS,
    filter: RetrievalFilter | None = None,
    semantic_fn: Callable[[str, int], Awaitable[Payloads]] | None = None,
    bm25_fn: Callable[[str, int], Awaitable[Payloads]] | None = None,
    entities_fn: Callable[[str], Awaitable[list[ExtractedEntity]]] | None = None,
    graph_fn: Callable[[list[ExtractedEntity], int], Awaitable[Payloads]] | None = None,
    rerank_fn: Callable[[str, list[str]], Awaitable[list[float] | None]] | None = None,
```

```
) -> HybridResult:
    notes: list[str] = []
    semantic_fn = semantic_fn or (lambda q, n: semantic_leg(q, n, corpus, filter))
    bm25_fn = bm25_fn or (lambda q, n: bm25_search(q, n, corpus, filter))
    graph_fn = graph_fn or (lambda e, n: graph_search(e, n, corpus))
    # trích thực thể một lần để mở rộng query BM25 (Việt-Anh)
    try:
        entities = await (entities_fn or extract_entities)(query)
    except Exception as exc:  # noqa: BLE001
        logger.error("[hybrid] không trích được thực thể để mở rộng query: %s", exc,
        ↪  exc_info=True)
        entities = []
        notes.append(f"Không mở rộng được query Việt-Anh ({type(exc).__name__}) — BM25 dùng
        ↪  query gốc.")
    bm25_query = expand_query(query, entities)
    semantic_r, bm25_r, graph_r = await asyncio.gather(
        semantic_fn(query, LEG_LIMIT), bm25_fn(bm25_query, LEG_LIMIT), graph_fn(entities,
        ↪  LEG_LIMIT), return_exceptions=True
    )
    semantic = _unwrap("semantic", semantic_r, notes)
    bm25 = _unwrap("bm25", bm25_r, notes)
    graph = _unwrap("graph", graph_r, notes)

    # trộn các nhánh theo RRF, khử trùng theo id của nguồn
    candidates = to_candidates(merge_by_id(semantic, bm25, graph, POOL, corpus.id_key),
    ↪  corpus)
    from_graph = {str(p.get(corpus.id_key)) for p in graph}
    candidates = [{**c, "via_graph": True} if c["_id"] in from_graph else c for c in
    ↪  candidates]  # agent thấy chunk nào do đồ thị đề xuất
    reranked = await rank_candidates(query, candidates, rerank_fn) if candidates else None
    return HybridResult(
        bm25_query=bm25_query,
        notes=notes,
        semantic=semantic,
        bm25=bm25,
        graph=graph,
        candidates=candidates,
        ranked=reranked if reranked is not None else candidates,
        reranked=reranked is not None,
    )
```

Mã nguồn 4.18: Mã điều phối luồng hỗn hợp

d) Đánh giá luồng hỗn hợp

Bộ kiểm thử lấy từ 378 chunk của sách Bộ Y tế; hai sách còn lại (cùng 4.540 chunk) chưa nằm trong bộ kiểm thử. RAGAS sinh câu hỏi theo vai bệnh nhân từ 120 chunk hợp lệ (loại chunk ngắn hơn 300 ký tự, mục lục, phụ lục), kết quả 24 câu (10 đơn bước, 14 đa bước) và 36 nhãn chunk.

Bảng 4.5: Kết quả đánh giá các luồng truy xuất trên bộ kiểm thử (n = 24)

Luồng truy xuất	hit@1	hit@3	hit@5	hit@10	recall@10	MRR@10
Ngữ nghĩa	0,58	0,75	0,83	0,96	0,90	0,70
Từ khóa BM25	0,50	0,67	0,83	0,88	0,73	0,63
Hỗn hợp	**0,83**	**0,92**	**1,00**	**1,00**	**0,94**	**0,90**

Luồng hỗn hợp cao nhất ở mọi chỉ số. So với luồng ngữ nghĩa, hit@5 tăng 0,17 (khoảng tin cậy 95% [0,04; 0,33]). Rerank nâng hit@1 từ 0,67 (sau RRF) lên 0,83.

Bộ kiểm thử có các hạn chế:

- Cỡ mẫu nhỏ nên khoảng tin cậy rộng.
- Câu hỏi sinh từ chính chunk nên văn phong gần văn bản gốc, thiên lợi cho BM25.
- Chưa kiểm tra thủ công toàn bộ nhãn.
- Bộ kiểm thử chỉ dùng sách tiếng Việt, chưa dùng sách Andrews (tiếng Anh) nên chưa đo được truy vấn xuyên ngôn ngữ.
- Bảng 4.5 chỉ gồm nhánh ngữ nghĩa và BM25.
- Chưa đánh giá kho ca bệnh và truy xuất đồ thị.

4.3.7 Truy xuất trực tiếp trên PrimeKG và DermO

Ngoài luồng đồ thị đi về đoạn giáo trình (mục 4.3.5), chatbot truy vấn trực tiếp đồ thị có sẵn (PrimeKG, DermO) để lấy quan hệ y khoa và bệnh ứng viên.

a) Giải quyết thực thể

Chatbot nêu tên thực thể (tối đa 6) khi truy vấn. Hệ thống ánh xạ các tên này về nút đồ thị:

1. Tra DermO (khớp tên hoặc từ đồng nghĩa) để lấy tên chuẩn.
2. Dùng tập tên chuẩn khớp nút PrimeKG loại bệnh, triệu chứng hoặc thuốc. Hệ thống khớp chính xác trước; nếu không có thì khớp một phần, tối đa 3 nút.
3. Tên không khớp (thường là tiếng Việt hoặc viết tắt) được LLM dịch sang tên y khoa chuẩn và thử lại một lần.

Bước 1 và 2 dùng hai câu truy vấn ở mã nguồn 4.19.

```
EXACT_SEED_QUERY = """
MATCH (n:Entity)
```

```
WHERE n.type IN $types AND toLower(n.name) IN $names
RETURN elementId(n) AS id, n.name AS name, n.type AS type
"""

LOOSE_SEED_QUERY = """
MATCH (n:Entity)
WHERE n.type IN $types AND toLower(n.name) CONTAINS $name
RETURN elementId(n) AS id, n.name AS name, n.type AS type
ORDER BY size(n.name)
LIMIT 3
"""
```

Mã nguồn 4.19: Câu Cypher khớp nút gốc: khớp chính xác, rồi khớp một phần

```
async def resolve_seeds(self, entities: list[str]) -> tuple[list[dict[str, Any]],
↪  list[str], set[str]]:
    """Trả (các nút PrimeKG khớp, input không khớp nút nào, DermO id của các input)."""
    seeds: dict[str, dict[str, Any]] = {}
    unmatched: list[str] = []
    dermo_ids: set[str] = set()
    for entity in entities:
        names = {entity.lower()}
        record = await self.exact_dermo(entity)
        if record:
            dermo_ids.add(record["id"])
            names |= {str(record["name"]).lower(), *(s.lower() for s in
            ↪  record.get("synonyms") or [])}
        rows = await self.run(EXACT_SEED_QUERY, types=SEED_TYPES, names=sorted(names))
        if not rows:
            rows = await self.run(LOOSE_SEED_QUERY, types=SEED_TYPES, name=entity.lower())
        if not rows:
            unmatched.append(entity)
        for row in rows:
            seeds[row["id"]] = row
    return list(seeds.values()), unmatched, dermo_ids
```

Mã nguồn 4.20: Mã ánh xạ thực thể sang nút PrimeKG

b) Truy vấn theo ý định

Chatbot chọn một trong tám ý định (intent) và nêu tên thực thể. Mỗi ý định ứng với một truy vấn cố định nên LLM không tự sinh câu truy vấn. Kết quả là các dữ kiện chủ thể – quan hệ – đối tượng, xếp theo ưu tiên của loại quan hệ.

Bảng 4.6: Ý định truy vấn đồ thị

Ý định	**Nội dung truy vấn**	**Đầu vào**
`disease_overview`	Tổng quan về bệnh	Bệnh
`symptoms_of_disease`	Triệu chứng có và không có	Bệnh
`red_flags_risk`	Dấu hiệu cảnh báo, yếu tố nguy cơ	Bệnh
`treatments`	Thuốc chỉ định và dùng ngoài chỉ định	Bệnh
`contraindications`	Chống chỉ định	Bệnh hoặc thuốc
`drug_profile`	Chỉ định, chống chỉ định, tác dụng phụ, tương tác thuốc	Thuốc
`diseases_with_symptom`	Bệnh ứng viên theo triệu chứng	Triệu chứng
`differential`	Triệu chứng phân biệt các bệnh dễ nhầm	2–5 bệnh

4.4 Kết luận

Hệ thống truy xuất từ ba nguồn: giáo trình số hóa (ba sách: Bộ Y tế, Đại học Y Dược TP.HCM, Andrews) và ca bệnh đã kết luận (Qdrant), đồ thị tri thức (Neo4j). Luồng chính dùng `hybrid_retrieval` (ngữ nghĩa + BM25 + đồ thị liên kết tới đoạn, gộp RRF, xếp hạng lại Cross-Encoder) làm công cụ mặc định, kết hợp đồ thị để chọn bệnh ứng viên và câu hỏi phân biệt. Trên bộ kiểm thử 24 câu, luồng hỗn hợp hai nhánh (ngữ nghĩa và BM25) đạt hit@5 = 1,00 và MRR@10 = 0,90, cao hơn từng luồng đơn; chưa đo lại sau khi thêm nhánh đồ thị.